COMP2011: Programming with C++

WU Jinjun

October 14, 2026

Contents

1 C++ Basics I **3**
1.1 Part I: A Simple C++ Program . . . 3
1.2 Part II: Simple C++ Data Types: Integers, Characters, and Strings . . . 5
1.3 Part III: C++ Variables . . . 6
1.4 Part IV: Operators . . . 8

2 Control Flow I **10**
2.1 You Have a Choice: `if` . . . 10
2.2 Loops or Iterations . . . 14

3 C++ Basics II **18**
3.1 More Basic Data Types in C++ . . . 18
3.2 Type Checking and Type Conversion . . . 20
3.3 Constants . . . 22
3.4 Further Readings and Examples . . . 23

4 Control Flow II **25**
4.1 Nested Loops . . . 25
4.2 Let's `switch`: C++ Multiple Choices . . . 29
4.3 Further Readings and Examples . . . 34

5 Problems for Basics and Control **36**
5.1 Problem 1: Guess the Number . . . 37
5.2 Problem 2: Drawing Right-Angled Triangles (RATs) . . . 40
5.3 Problem 3: Count Animals . . . 43
5.4 Problem 4: GPA Calculator . . . 46

6 Function I **52**
6.1 Function Basics . . . 52
6.2 Modular Programming . . . 55
6.3 Parameter Passing Methods . . . 58
6.4 Further Examples . . . 63

7 Array **66**
7.1 1-Dimensional Array . . . 66
7.2 Multi-dimensional Array . . . 72
7.3 C String: Special 1D Character Array . . . 75

8 Function II: Recursion **78**
8.1 Think Recursively! . . . 78
8.2 Programming Recursion in C++ . . . 79
8.3 Further Examples . . . 87

9 Scope **87**
9.1 What is the Scope of an Identifier? . . . 87
9.2 Example: File/Function/Block Scope . . . 87
9.3 File Scope . . . 88
9.4 Function Scope . . . 88
9.5 Block Scope . . . 89
9.6 Example: Problems with a Global Variable . . . 89
9.7 Identifiers of the Same Name . . . 89
9.8 Example: Scope Resolution . . . 90

10 Struct **91**
10.1 C++ Structure . . . 91
10.2 Array of Structures . . . 95

11 Function III: Miscellaneous **98**
11.1 Part I: Function Declaration and Function Definition . . . 98
11.2 Part II: Function Overloading . . . 102
11.3 Default Function Arguments . . . 104

12 Pointer **105**
12.1 lvalue (Address) and rvalue (Content) . . . 105
12.2 What is a Pointer? . . . 107
12.3 Pointer to Structure . . . 112
12.4 Dynamic Memory/Objects Allocation and Deallocation . . . 115
12.5 A Dynamic Data Structure: Linked List . . . 118
12.6 Array as a Pointer . . . 123
12.7 Multi-dimensional Array and Pointer . . . 127
12.8 Further Reading 0: Relation between Dynamic 2D Array & Pointer . . . 128
12.9 Further Reading I: Arguments for the `main()` Function . . . 129
12.10 Further Reading II: Array of Pointers to Structures . . . 130
12.11 Further Reading III: Dynamic Data Structure: (Binary) Tree . . . 132

13 C++ Class **134**
13.1 What is a C++ Class? . . . 134
13.2 OOP! . . . 136
13.3 Inline Class Member Functions; Classes With Dynamic Data; Interaction Between Classes . . . 141
13.4 C++ Class Basics, and this Pointer . . . 145

14 Stack & Queue **149**
14.1 Data Structures . . . 149
14.2 Part I: Stack . . . 149
14.3 Part II: Queue . . . 152

1 C++ Basics I

Just like human languages (Chinese, English, Japanese, ...), each programming language (Pascal, C++, Java, ...) has its own:

- **vocabulary** — the set of legal "words";
- **grammar (syntax)** — how "words" are put together to make a legal "sentence".

A **program** consists of a sequence of **statements** (c.f. sentences in human language). Some parts of a statement are called **expressions** (c.f. phrases), e.g. the logical expression `x > y` and the arithmetic expression `5 + 4`.

1.1 Part I: A Simple C++ Program

1.1.1 A First Program: `hello-world.cpp`

The classic first program simply prints `Hello World!` to the screen.

File: hello-world.cpp — the common demo program used when learning a new language; prints "Hello World!".

```
/*
 * File: hello-world.cpp
 * A common program used to demo a new language
 */

#include <iostream>     // Load info of a Standard C++ library
using namespace std;    // Standard C++ namespace

int main()              // Program's entry point
{
    /* Major program codes */
    cout << "Hello World!" << endl;

    return 0;           // A nice ending
}
```

1.1.2 Write, Compile, and Run a Program in a Terminal

1. **Write** the program using an editor, e.g. Visual Studio Code (Microsoft), Eclipse, vi (Unix/Linux), or even MS Word.
2. **Save** the program into a file called `hello-world.cpp`.
3. **Compile** the program using the `g++` compiler:

   ```
   g++ -o hello-world hello-world.cpp
   ```

   If you don't specify the output filename using the `-o` option, the default is `a.out`:

   ```
   g++ hello-world.cpp
   ```

4. **Run** the program in a terminal (command window):

   ```
   linux:: hello-world
   Hello World!
   ```

Example: File: add.cpp — computes and prints the sum of two numbers that are hard-coded into the program.

```
/* File: add.cpp */
#include <iostream>     // Load info of a Standard C++ library
using namespace std;    // Standard C++ namespace

int main()              // Program's entry point
{
```

```
    /* Major program codes */
    cout << "123 + 456 = " << 123 + 456 << endl;

    return 0;            // A nice ending
}
```

1.1.3 Anatomy of a Simple C++ Program: `main`, Comments, and `#include`

Exactly One `main()` Function

Every program must have exactly one and only one `main()` function.

- Simple form of the `main` function: `int main() { ... }`
- General form of the `main` function: `int main(int argc, char** argv) { ... }` (We will talk about `argc` and `argv` later.)
- Between the braces "{" and "}" are the program codes consisting of zero or more program statements.
- Each simple C++ statement ends in a semicolon ";".

Comments.

- Use `/* ... */` for multiple-line comments:

```
/*
 * A common program used to demo a new language
 */
```

- Single-line comments start with `//`:

```
// Program's entry point
```

- Comments are just for humans to read. They will not be translated by the compiler into machine codes.

`#include` and Standard C++ Libraries.

- `#include` will include information of a **library** — a collection of sub-programs.
- E.g. `#include <iostream>` gets the information of the standard C++ library called `iostream` that deals with I/O:
  - `cin`: an object to read from, e.g. keyboard or file;
  - `cout`: an object to print to, e.g. screen or file;
  - `cerr`: an object to print error messages to, e.g. screen or file.

Examples:

```
// endl means "end of a line"
cout << "Einstein: God does not play dice." << endl;

// You may also break down the message in several lines
cerr << "Error: "
     << "There is no stress and tension in HKUST!"
     << endl;
```

These library information files are called **header files**.

User-defined Header Files.

- `#include` causes the contents of the specified header file to be inserted into the original file.
- You may create your own header files, which contain the definition or declaration of some objects and data structures; again you use `#include` to include them into your sub-programs.
- Example: `#include "drawing.h"` gets the information of a user-defined C++ library about drawing.
- By convention, the name of a header file ends in ".`h`" or ".`hpp`", while Standard C++ library header files have no file suffix.

- Also by convention, a user-created header file is delimited by double-quotes `"..."`, while Standard C++ library header files are delimited by `<...>`.

1.1.4 Free Format and Good Programming Style

- Extra blanks, tabs, and lines are ignored; thus codes may be indented in any way to enhance readability.
- More than one statement can be on one line.
- A single statement may also be spread over several lines:

```
cout << "Hello World!"
     << endl;
```

File: hello-world-too.cpp — the same Hello World program, showing that C++ is free format (multiple statements crammed onto few lines still compile).

```
#include <iostream> /* File: hello-world-too.cpp */
using       namespace std; int main   (int argc,
   char** argv) { cout<<"Hello World!"<<endl;return   0;}
```

Good Programming Style

- Place each statement on a line by itself.
- For long statements: if possible, break it down into several shorter statements; wrap it around with proper indentation (since extra space doesn't matter!).
- Use blank lines to separate sections of related codes that together perform some action.
- Indent consistently: use the same indentation for the same block of codes.

1.2 Part II: Simple C++ Data Types: Integers, Characters, and Strings

1.2.1 Basic Data Types: Integers, Characters, and Strings

- The Web has to deal with different multimedia data (text, sound/music, image, video, ...), and they can only be read/viewed with different softwares such as MS Notepad, Acrobat Reader, RealPlayer, etc.
- Similarly, a computer program has to deal with different types of data; in a programming language, data are categorized into different **types**.
- Each data type comes with a set of **operations** for manipulating its values. Operations on basic data types are built into a programming language.
- **Integers**: examples: $\ldots, -2, -1, 0, 1, 2, \ldots$; C++ type name: `int`.
- **Characters**: examples: `'a'`, `'b'`, `'4'`. Represent a single character by delimiting it in single quotes. For special characters, use the escape character \, e.g. `'\t'` = tab, `'\n'` = newline, `'\b'` = backspace, `'\0'` = null character. C++ type name: `char`.
- **Character strings**: examples: `"hkust"`, `"How are you?"`, `"500 dollars"`. Character strings are not a basic data type in C++; they are sequences of basic `char` data.

The `string` Library

There is a `string` library that defines `string` objects which are more than a character string. (More about it later.)

1.2.2 Binary Representation of Numbers and Characters

- A computer uses **binary numbers** (base 2) to represent data.
- In the decimal system: $423_{10} = 4 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$.

- In the binary system, a digit has only 2 possibilities: $\{0, 1\}$. Example: $101_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$.
- Thus, the maximum N-digit number in base 2 is $2^N - 1$.
- A binary digit is aka **bit**. 8 bits = 1 **byte** (the smallest amount of data that a computer can "bite" at once).

Relation between Characters and Integers.

- In C++, a `char` datum is represented by 1 byte (8 bits). Question: how many different characters can 8 bits represent? ($2^8 = 256$.)
- Put it in another way, a `char` datum is encoded by one of the possible 8-bit patterns.
- The most common encoding scheme is called **ASCII** (American Standard Code for Information Interchange).
- Since a computer only recognizes bits, a `char` datum may also be interpreted as an integer!

Character	ASCII Code	Integral Value
`'0'`	00110000	48
`'1'`	00110001	49
`'9'`	00111001	57
`'?'`	00111111	63
`'A'`	01000001	65
`'B'`	01000010	66
`'Z'`	01011010	90
`'a'`	01100000	97
`'b'`	01100001	98
`'z'`	01111010	122

1.3 Part III: C++ Variables

In the old `add.cpp` example, the 2 numbers to be added are hard-coded into the program file. Can we write a program that takes 2 arbitrary numbers to add? — We need variables.

1.3.1 Identifiers and Reserved Words

In math, $f(x) = x^2 + c$ involves: f — name of a function; x — name of a variable; c — name of a constant. In programming languages, these "names" are called **identifiers**.

Rules for Making up Identifier Names.

- Only the following characters may appear in an identifier: `0--9`, `a--z`, `A--Z`, _ (underscore).
- The first character cannot be a digit (`0--9`).
- C++ **keywords** — reserved words — are not allowed.
- Examples: `amount`, `COMP2011`, `myname`.
- C++ identifiers are **case-sensitive**: lowercase and uppercase letters are considered different. ⇒ `hkust`, `HKUST`, `HkUst`, `HKust` are different identifiers.

Guidelines:

- Use meaningful names, e.g. `amount` instead of `a`.
- For long names consisting of several words, use '_' to separate them or capitalize them, e.g. `num_of_students` or `numOfStudents` instead of `numofstudents`.
- Usually identifiers starting with '_' are used for system variables.

C++ keywords (reserved words) include:

asm	auto	bool	break	case
catch	char	class	const	const_cast
continue	default	delete	do	double
dynamic_cast	else	enum	explicit	extern
false	float	for	friend	goto
if	inline	int	long	mutable
namespace	new	operator	private	public
protected	register	reinterpret_cast	return	short
signed	sizeof	static	static_cast	struct
switch	template	this	throw	true
try	typedef	typeid	typename	union
unsigned	using	virtual	void	volatile
wchar_t	while			

1.3.2 Variables: Declaration and Definition

A variable has (analogy: Chan Tai Man's home):

- **Name**: Chan Tai Man's home;
- **Address**: Unit X, Kam Wah House, Choi Hung Estate, Kowloon;
- **Content**: Chan's family.
- A **variable** is a named memory location for a value that we can write to, retrieve from, and manipulate. It can be thought of as a container/box for a value.
- A variable must be **declared** and/or **defined** before it can be used.

  Syntax (variable definition): `<data-type> <identifier>;`

  Example: `int radius = 10, sum = 0;`

  Syntax (defining several variables of the same type at once): `<data-type> <identifier1>, <identifier2>, ...;`

  Examples:

```
int radius, num_of_words;
char choice, gender, pass_or_fail;
```

- When a variable is defined, the compiler allocates memory for it. The amount of memory is equal to the size of its data type.
- Some books call this variable declaration. Actually there is a big difference between declaration and definition (talked about later). When a variable is defined, it is also declared; the other way is not true.

1.3.3 Variable Initialization

Syntax (initialize variables while they are defined): `<data-type> <identifier> = <value>;`

Several variables of the same type may also be initialized at the same time:

```
int radius = 10, sum = 0;
```

A variable may also be initialized by a separate assignment statement after it is defined:

```
int radius;      // Variable definition
radius = 5;      // Initialization by assignment
```

Uninitialized Variables Contain Garbage

ANSI C++ does not require compilers to initialize variables. Thus, in general, if you do not explicitly initialize variables while defining them, their initial contents may be garbage. (Global variables are an exception.)

Example: File: add-var.cpp — reads 2 arbitrary integers from the user with `cin` and prints their sum (no longer hard-coded).

```
#include <iostream>      /* File: add-var.cpp */
using namespace std;
```

```
int main()                // Program's entry point
{
    int x, y;             // Define 2 variables to hold the int values \
   to add

    cin >> x;             // You may also shorten the 2 statements \
   into one:
    cin >> y;             // cin >> x >> y;

    cout << x << " + " << y << " = " << x+y << endl;

    return 0;             // A nice ending
}
```

1.4 Part IV: Operators

1.4.1 Assignment Operator

Syntax (assignment): `<variable> = <value>;`

In C++, the "=" sign is used to assign a value to a variable; it is the **assignment operator**.

Examples:

```
int a, b, x = 2, y = 3, z = 4;
a = 10*x;
b = a - (100*y - 1000*z);
a = a + b;
```

'=' is Not Math Equality

Don't try to understand the assignment statement `a = a + b;` using normal math notation — otherwise it doesn't make sense. Nor should you treat it as a boolean relational "equality" sign.

1.4.2 Arithmetic Operators

Assuming `x = 100`, `y = 67`, the arithmetic operators on `int` are:

Operation	Operator
unary minus	`-`
addition	`+`
subtraction	`-`
multiplication	`*`
division	`/`
modulus	`%`
increment	`++`
decrement	`--`

1.4.3 Pre- and Post- Increment, Decrement

- The unary increment operator `++` adds 1 to its operand; the unary decrement operator `--` subtracts 1 from its operand.
- There are 2 ways to call them: **pre-increment** `++x` vs **post-increment** `x++` (similarly `--x` vs `x--`).
- If used alone, they are equivalent to `x = x + 1` and `x = x - 1`.
- But if used with other operands, there is a big difference:
  – `++x` ⇒ add 1 to `x`, and use the result for further operation;

– `x++` ⇒ use the current value of `x` for some operation, and then add 1 to `x`.

```
cout << ++x;            /* same as */   x = x + 1;   cout << x;
cout << x++;            /* same as */   cout << x;   x = x + 1;
```

Example: File: inc-mod.cpp — demonstrates pre/post increment and decrement, and the trickiness of the mod operator with negative operands.

```
#include <iostream>     /* File: inc-mod.cpp */
using namespace std;

int main()
{
    int x = 100, y = 100; // Variable definitions and initialization
    int a = 10, b = 10, c = 10, d = 10;

    b = a + (x++) + ++y;

    cout << ++x << "\t"; cout << "x = " << x << endl; // Pre-increment
    cout << y++ << "\t"; cout << "y = " << y << endl; // \
   Post-increment

    a = ++b; cout << "a = " << a << "\t" << "b = " << b << endl;
    c = d++; cout << "c = " << c << "\t" << "d = " << d << endl;

    cout << 17%5 << endl; // Trickiness of the mod function
    cout << (-17)%5 << endl;
    cout << 17%(-5) << endl;
    cout << (-17)%(-5) << endl;

    return 0;
}
```

1.4.4 Shorthand Assignment Operators

Shorthand Notation	Normal Notation
`n += 2`	`n = n + 2`
`n -= 2`	`n = n - 2`
`n *= 2`	`n = n * 2`
`n /= 2`	`n = n / 2`
`n %= 2`	`n = n % 2`

1.4.5 Precedence and Associativity

Operator	Description	Associativity
`-`	minus (unary)	Right-to-Left
`++`	increment	
`--`	decrement	
`*`	multiply	Left-to-Right
`/`	divide	
`%`	mod	
`+`	add	Left-to-Right
`-`	subtract	
`=`	assignment	Right-to-Left

Precedence Rules Decide Which Operators Run First

Example: $1/2 + 3 * 4 = (1/2) + (3 * 4)$ because `*`, `/` have a higher precedence over `+`, `-`. In general, $x\ P\ y\ Q\ z = x\ P\ (y\ Q\ z)$ if operator Q is at a higher precedence level than operator P.

Associativity decides the grouping of operands with operators of the same level of precedence. Example: $1 - 2 + 3 - 4 = ((1 - 2) + 3) - 4$ because `+`, `-` are left associative. In general, if binary operators P, Q are of the same precedence level:

- if P, Q are both right associative, then $x\ P\ y\ Q\ z = x\ P\ (y\ Q\ z)$;
- if P, Q are both left associative, then $x\ P\ y\ Q\ z = (x\ P\ y)\ Q\ z$.

Cascading Assignments. C++ allows assigning the same value to multiple variables at once (assignment is right-associative):

```
int w, x, y, z;
y = z = 5;          // Same as y = (z = 5);
w = x = y + z;      // Same as w = (x = (y+z));
```

1.4.6 Expression and Statement

- An **expression** has a value which is the result of some operation(s) on its operand(s). Examples: `4`, `x - y`, `2 - a - (b * c)`.
- A **statement** is a sentence that acts as a command. It does not have a value and always ends in a '`;`'. Examples:
  - input statement: `cin >> x;`
  - output statement: `cout << x;`
  - assignment statement: `x = 5;`
  - variable definition: `int x;`
- For the first 3 statement examples above, if we take out the ending '`;`', they become input/output/assignment expressions! (More about this later.)

2 Control Flow I

So far, our C++ program consists of only the `main()` function. Inside `main()` is a sequence of statements, and all statements are executed once and exactly once. Such sequential computation can be a big limitation on what can be computed. Therefore, we have:

- **selection**
- **iteration**

2.1 You Have a Choice: `if`

2.1.1 `if` Statements

- Syntax:

```
if (<boolean expression>) <statement>
if (<boolean expression>) { <sequence of statements> }
```

- If the boolean expression evaluates to `true`, the statement(s) are executed; if it evaluates to `false`, they are skipped.
- Example: absolute value $|x|$ of x.

```
int x;
cin >> x;
if (x < 0)
{
    x = -x;
}
```

File: swap.cpp — sorts 2 numbers so that the 2nd one is larger, using an `if` statement and a temporary variable to swap.

```
#include <iostream>     /* File: swap.cpp */
using namespace std;
```

```
int main() /* To sort 2 numbers so that the 2nd one is larger */
{
    int x, y;           // The input numbers
    int temp;           // A dummy variable for manipulation

    cout << "Enter two integers (separated by whitespaces): ";
    cin >> x >> y;

    if (x > y)
    {
        temp = x;       // Save the original value of x
        x = y;          // Replace x by y
        y = temp;       // Put the original value of x to y
    }

    cout << x << '\t' << y << endl;
    return 0;
}
```

The `if-else` statement:

- Syntax:

```
if (<bool-exp>) <stmt> else <stmt>
if (<bool-exp>) { <stmts> } else { <stmts> }
```

- If the boolean expression is `true`, the first statement(s) run; otherwise the `else` statement(s) run.
- Example: to find the larger value.

```
int x, y, larger;
cin >> x >> y;
if (x > y)
    larger = x;
else
    larger = y;
```

The `if-else-if` statement:

- Syntax:

```
if (<bool-exp>) <stmt>
else if ( <bool-exp>) <stmt>
else if ( <bool-exp>) <stmt>
...
else <stmt>
```

```
if (<bool-exp>) { <stmts> }
else if ( <bool-exp>) { <stmts> }
else if ( <bool-exp>) { <stmts> }
...
else { <stmts> }
```

File: if-elseif-grade.cpp — converts a numerical mark to a letter grade using an `if-else-if` chain.

```
#include <iostream>     /* File: if-elseif-grade.cpp */
using namespace std;

int main()              /* To determine your grade (fictitious) */
{
    char grade;         // Letter grade
    int mark;           // Numerical mark between 0 and 100
    cin >> mark;

    if (mark >= 90)
```

```
        grade = 'A';     // mark >= 90
    else if (mark >= 60)
        grade = 'B';     // 90 > mark >= 60
    else if (mark >= 20)
        grade = 'C';     // 60 > mark >= 20
    else if (mark >= 10)
        grade = 'D';     // 20 > mark >= 10
    else
       grade = 'F';      // 10 > mark

    cout << "Your letter grade is " << grade << endl;
    return 0;
}
```

2.1.2 Relational Operators

Relational operators are used to compare two values. The result is boolean, indicating if the relationship is true or false.

Math	C++	Meaning
$=$	`==`	equal to
$<$	`<`	less than
$\leq$	`<=`	less than or equal to
$>$	`>`	greater than
$\geq$	`>=`	greater than or equal to
$\neq$	`!=`	not equal to

Don't mix up the 2 following different expressions:

```
x = y   // This is an assignment
x == y  // This is an equality comparison
```

2.1.3 Logical Operators and Operator Precedence

- Logical operators are used to modify or combine boolean values.
- C++ has 3 logical operators:
  - `!`: logical NOT
  - `||`: logical OR
  - `&&`: logical AND
- Boolean values:
  - `true`: internally represented by 1; ANY non-zero number is also considered `true`.
  - `false`: internally represented by 0.

p	q	`!`p	p `&&` q	p `||` q
T	T	F	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	F

Precedence and associativity of boolean operators. Operators below are shown in decreasing order of precedence.

Operator	Description	Associativity
`()`	parentheses	—
`++ -- ! -`	increment, decrement, logical NOT, unary minus	Right-to-Left
`* / %`	multiply, divide, mod	Left-to-Right
`+ -`	add, subtract	Left-to-Right
`> >= < <=`	relational operator	Left-to-Right
`== !=`	equal, not equal	Left-to-Right
`&&`	logical AND	Left-to-Right
`||`	logical OR	Left-to-Right
`=`	assignment	Right-to-Left

When in Doubt, Use Parentheses

When you are in doubt of the precedence or associativity, use extra parentheses to enforce the order of operations.

2.1.4 `=` versus `==`

- Both `x = y` and `x == y` are valid C++ expressions.
- `x = y` is an assignment expression, assigning the value of `y` to `x`. The expression has a result which is the final value of `x`. (That is why the cascading assignment works.)
- `x == y` is a boolean expression, testing if `x` and `y` are equal, and the result is either `true` or `false`.
- But since C++ also interprets integers as boolean:
  - In `if (x = 3) { <stmts> }`, `<stmts>` are always executed because `(x = 3)` evaluates to 3 — a non-zero value — which is interpreted as `true`.
  - In `if (x = 0) { <stmts> }`, `<stmts>` are always NOT executed because `(x = 0)` evaluates to 0 which is interpreted as `false`.

Assignment Used as a Boolean Expression

It is not recommended to use an assignment expression as a boolean expression. Writing `if (x = 3)` when you mean `if (x == 3)` compiles fine but silently makes the condition always true.

2.1.5 `if-else` Operator: `?:`

- Syntax:

```
(<bool-exp>) ? <then-exp> : <else-exp>;
```

- The ternary if-else operator `?:` is used.
- Unlike an `if-else` statement, an `if-else` expression has a value!

```
/* Example: get the larger of two numbers */
larger = (x > y) ? x : y;
```

```
/* Example: get the letter grade from the percentile */
grade = (percentile >= 85) ? 'A'
      : ((percentile >= 60) ? 'B'
      : ((percentile >= 15) ? 'C'
      : ((percentile >= 5)  ? 'D': 'F'
        )
      )
    );
```

2.1.6 Nested `if` and the "Dangling `else`" Problem

- In the `if` or `if-else` statement, the `<stmts>` in the if-part or else-part can be any statement, including another `if` or `if-else` statement. In the latter case, it is called a nested `if` statement.
- "Nested" means that a complete statement is inside another.

```
if (condition1)
{
    if (condition2)
        if (condition3)
            cout << "conditions 1,2,3 are true." << endl;
        else
```

```
            cout << "conditions 1,2 are true." << endl;
    else
        cout << "condition1 true; condition2 false." << endl;
}
```

The "dangling `else`" problem: What is the value of `x` after the following code is executed?

Program code:

```
int x = 15;
if (x > 20)
    if (x > 30)
        x = 8;
    else
        x = 9;
```

Interpretation 1:

```
int x = 15;
if (x > 20)
{
    if (x > 30)
        x = 8;
    else
        x = 9;
}
```

Interpretation 2:

```
int x = 15;
if (x > 20)
{
    if (x > 30)
        x = 8;
}
else
    x = 9;
```

Dangling `else` Rule

C++ groups a dangling `else` with the most recent `if`. Thus, for the code above, interpretation 1 is used.

Use Braces and Indentation

It is a good programming practice to use extra braces "{ }"
- to control how your nested `if` statements should be executed.
- to clarify your intended meaning, together with proper indentation.

2.2 Loops or Iterations

2.2.1 `while` Loop (Statement)

- Syntax:

```
while (<bool-exp>) { <stmts> }
```

- `<stmts>` will be repeated as long as the value of `<bool-exp>` is `true`.
- As usual, `<stmts>` can be a single statement, or a sequence of statements (including another `while` statement), or even no statement!
- What does `while (x > 0) ;` do? (It loops forever doing nothing, as long as `x > 0` stays true.)

- In general, a `while` statement only makes sense if the value of `<bool-exp>` may be changed by `<stmts>` inside the `while` loop.

File: while-factorial.cpp — computes $x! = x(x-1)(x-2)\ldots 1$ for a non-negative integer x using a `while` loop.

```
#include <iostream>     /* File: while-factorial.cpp */
using namespace std;

/* To compute x! = x(x-1)(x-2)...1, where x is a non -ve integer */
int main()
{
    int factorial = 1;
    int number;

    cout << "Enter a non-negative integer: ";
    cin >> number;

    while (number > 0)
    {
        factorial *= number; // Same as: factorial = factorial*number
        --number;            // Same as: number = number-1
    }

    cout << factorial << endl;
    return 0;
}
```

Trace of the loop (assume the user enters 4 for the variable `number`):

Iteration	`factorial`	`number`	`(number > 0)`
0	1	4	true
1	4	3	true
2	12	2	true
3	24	1	true
4	24	0	false

File: while-max.cpp — finds the maximum of a list of positive integers; input stops when a non-digit character is entered, using `cin >> x` itself as the loop condition.

```
#include <iostream>     /* File: while-max.cpp */
using namespace std;

// To find the maximum of a list of +ve integers. Stop by inputting a
// character that is not a digit. Assume there is at least one number.
int main()
{
    cout << "Enter a number: ";
    int x; cin >> x;    // Input integers

    int max = x;        // Result initialized with the first number

    cout << "Enter the next number: ";
    while (cin >> x)    // If there is input, cin returns TRUE else \
   FALSE
    {
        if (x > max)
            max = x;
        cout << "Enter the next number: ";
    }

    cout << endl << "The maximum number = " << max << endl;
    return 0;
}
```

File: halving-float.cpp — repeatedly halves a `float` number until it is not greater than 0.1, counting the number of halvings.

```
#include <iostream>     /* File: halving-float.cpp */
using namespace std;

int main()
{
    int count = 0;      // Count how many times that x can be halved
    float x;            // Number to halve

    cout << "Enter a positive number: ";
    cin >> x;

    while (x > 0.1)
    {
        cout << "Halving " << count++ << " time(s); "
             << "x = " << x << endl;
        x /= 2;
    }

    return 0;
}
```

Sample run:

```
Enter a positive number: 7
Halving 0 time(s); x = 7
Halving 1 time(s); x = 3.5
Halving 2 time(s); x = 1.75
Halving 3 time(s); x = 0.875
Halving 4 time(s); x = 0.4375
Halving 5 time(s); x = 0.21875
Halving 6 time(s); x = 0.109375
```

File: halving-int.cpp — the same halving program but with `int x`; integer division truncates, so the loop stops much earlier.

```
#include <iostream>     /* File: halving-int.cpp */
using namespace std;

int main()
{
    int count = 0;      // Count how many times that x can be halved
    int x;              // Number to halve

    cout << "Enter a positive number: ";
    cin >> x;

    while (x > 0.1)
    {
        cout << "Halving " << count++ << " time(s); "
             << "x = " << x << endl;
        x /= 2;
    }

    return 0;
}
```

Sample run:

```
Enter a positive number: 7
Halving 0 time(s); x = 7
Halving 1 time(s); x = 3
Halving 2 time(s); x = 1
```

2.2.2 A Good Programming Practice on Loops

After you have written the codes for a loop, try verifying the following cases:

- The first iteration.
- The second iteration.
- The last iteration.

Infinite Loops

Do you know exactly how many iterations will be performed? How can the loop terminate? Otherwise, you have an infinite loop! And the program runs forever!

2.2.3 `for` Loop (Statement)

- Syntax:

```
for (<for-initialization> ; <bool-exp> ; <post-processing>)
{ <stmts> }
```

- `for` statement is a generalization of the `while` statement. The idea is to control the number of iterations, usually by a counter variable.
- `<for-initialization>` sets up the initial values of some variables, usually a counter, before executing `<stmts>`.
- `<stmts>` are iterated as long as `<bool-exp>` is `true`.
- At the end of each iteration, `<post-processing>` will be executed. The idea is to change some values, again usually the counter, so that `<bool-exp>` may become `false`.

File: for-factorial.cpp — computes $x!$ using a `for` loop with a counter `j`.

```
#include <iostream>     /* File: for-factorial.cpp */
using namespace std;

/* To compute x! = x(x-1)(x-2)...1, where x is a non -ve integer */
int main()
{
    int factorial = 1;
    int number;

    cout << "Enter a non-negative integer: ";
    cin >> number;

    for (int j = 1; j <= number; ++j) // Set up a counter to iterate
        factorial *= j;

    cout << number << "! = " << factorial << endl;
    return 0;
}
```

File: for-power.cpp — computes x^n for integers x and n with $n \geq 0$, reporting an error via `cerr` if $n < 0$.

```
#include <iostream>     /* File: for-power.cpp */
using namespace std;

/* To compute $x^n$, where x and n are integers, and n >= 0 */
int main()
{
    int x;
    int n;              // Power or exponent
    int result = 1;     // Need to initialize it to 1. Why?

    cout << "Enter a number followed by its +ve power: ";
    cin >> x >> n;

    if (n < 0)
```

```
        cerr << "Error: n < 0!" << endl;
    else
    {
        for (int j = 1; j <= n; j++)
            result *= x;

        cout << x << " to the power of " << n << " = " << result << \
   endl;
    }

    return 0;
}
```

Remarks on the `for` statement:

- Notice that the variable `j` in the above 2 examples is only defined inside the `for` loop. When the loop is done, `j` disappears, and you cannot use that `j` anymore.
- `while` statement is a special case of `for` statement. How can you simulate `while` using `for`?
- Sometimes, if the for-body is short, you may even further compact the code as follows:

```
for (int j = 1; j <= number; factorial *= j++)
    ;
```

Stray Semicolon After `for`

Don't mis-type a ";" after the first line of the `for` loop. E.g., what is the result of the following code?

```
for (int j = 1; j <= n; j++);
    result *= x;
```

The empty statement becomes the loop body; `result *= x;` runs only once after the loop.

3 C++ Basics II

3.1 More Basic Data Types in C++

3.1.1 C++ Basic Types

- Common basic types, with typical sizes on a 32-bit machine:

Type	Size (#bytes)	Value Range
`bool`	1	{`true`, `false`}
`char`	1	$[-128, 127]$
`short`	2	$[-32768, 32767]$
`int`	4	$[-2^{31}, 2^{31}-1]$
`long`	4	$[-2^{31}, 2^{31}-1]$
`float`	4	$\pm$[1.17549E-38, 3.40282E+38]
`double`	8	$\pm$[2.22507E-308, 1.79769E+308]

- Not all numbers of a type can be represented by a computer.
- It depends on how many bytes you use to represent it: with more bytes, more numbers can be represented.
- Type names: `short` (`int`), `int`, `long` (`int`), `long long` (`int`).
- Their sizes depend on the CPU and the compiler.
- ANSI C++ requires:

  size of `short` $\leq$ size of `int` $\leq$ size of `long` $\leq$ size of `long long`

  e.g., a 2-byte `short int` can represent the numbers in $[-32768, 32767]$.

- Each integral data type has 2 versions:
  - **signed** version: represents both positive and negative integers, e.g. `signed short`, `signed int`, `signed long`.
  - **unsigned** version: represents only positive integers, e.g. `unsigned short`, `unsigned int`, `unsigned long`.
- Signed versions are the default.
- Obviously `unsigned int` can represent 2 times more positive integers than `signed int`.

Think About It

How are negative numbers represented?

- Floating-point numbers are used to represent real numbers and very large integers (which cannot be held in `long long`).
- Type names:
  - `float` for single-precision numbers.
  - `double` for double-precision numbers.
- Precision: for decimal numbers, if you are given more decimal places, you may represent a number to higher precision.
  - For 1 decimal place: $1.1, 1.2, 1.3, \ldots$ etc.; can't get 1.03.
  - For 2 decimal places: $1.01, 1.02, 1.03, \ldots$ etc.; can't get 1.024.
- In scientific notation, a number has 2 components, e.g. `5.16E-02`:
  - mantissa: 5.16
  - exponent: -2
- More mantissa bits $\Rightarrow$ higher precision. More exponent bits $\Rightarrow$ larger real number.

3.1.2 Integer Arithmetic and Floating-Point Arithmetic

- Arithmetic expressions involving only integers use **integer arithmetic**.
- Arithmetic expressions involving only floating-point numbers use **floating-point arithmetic**.
- For $+$, $-$, $\times$ operations, results should be what you expect.
- However, integer division and floating-point division may give different results, e.g.,

```
10/2 = 5   and   10.0/2.0 = 5.0
9/2 = 4    and   9.0/2.0 = 4.5
4/8 = 0    and   4.0/8.0 = 0.5
```

Integer Division Truncates

When both operands of `/` are integers, the result is an integer: any fractional part is discarded. Thus `9/2` gives `4` (not `4.5`) and `4/8` gives `0` (not `0.5`). To get a floating-point answer, make at least one operand floating-point, e.g. `9.0/2.0`.

3.1.3 Boolean Data Type

- Type name: `bool`.
- Used to represent the truth value, `true` or `false`, of logical (boolean) expressions like:

```
a > b     x + y == 0     true && false
```

- Since C++ evolves from C, C++ follows C's convention:
  - zero may be interpreted as `false`.
  - non-zero values may be interpreted as `true`.
- However, since internally everything is represented by 0's and 1's,
  - `false` is represented as 0.
  - `true` is represented as 1.

- Even if you put other values to a `bool` variable, its internal value always is changed back to either 1 or 0.

Booleans Are Stored as 0 or 1

A `bool` variable only ever holds 0 (`false`) or 1 (`true`) internally. Assigning any non-zero value to a `bool` stores 1.

File: boolalpha.cpp — demonstrates the default numeric output format of `bool` values, and the `boolalpha` / `noboolalpha` output manipulators that switch between printing English words and printing 1 or 0.

```
#include <iostream>     /* File: boolalpha.cpp */
using namespace std;

int main()
{
    bool x = true;
    bool y = false;

    // Default output format of booleans
    cout << x << " && " << y << " = " << (x && y) << endl << endl;

    cout << boolalpha;   // To print booleans in English
    cout << x << " && " << y << " = " << (x && y) << endl << endl;

    cout << noboolalpha; // To print booleans in 1 or 0
    cout << x << " && " << y << " = " << (x && y) << endl;

    return 0;
}
```

File: bool-blood-donation.cpp — demonstrates using `bool` variables to store the results of logical expressions, and using them directly in an `if` condition (blood-type matching: exact match, or donor of type O which matches all).

```
#include <iostream>     /* File: bool-blood-donation.cpp */
using namespace std;

int main()
{
    char donor_blood_type, recipient_blood_type;
    bool exact_match, match_all;

    cout << "Enter donor's bloodtype: A, B, C (for AB), and O. ";
    cin >> donor_blood_type;
    cout << "Enter recipient's bloodtype: A, B, C (for AB), and O. ";
    cin >> recipient_blood_type;

    exact_match = (donor_blood_type == recipient_blood_type);
    match_all = (donor_blood_type == 'O');

    if (exact_match || match_all)
        cout << "Great! A donor is found!" << endl;
    else
        cout << "Keep searching for the right donor." << endl;

    return 0;
}
```

3.2 Type Checking and Type Conversion

3.2.1 Type Checking and Coercion

- Analogy — blood types:

Receiver	Donor
A	A, O
B	B, O
AB	A, B, AB, O
O	O

- For most languages, data types have to be matched during an operation ⇒ **type checking**.
- However, sometimes, a type is made compatible with a different type ⇒ **coercion**.
- **Coercion** is the automatic conversion of the data type of operands during an operation.
- Example: `3 + 2.5` ⇒ `int + double`. The C++ compiler will automatically change it to

  `3.0 + 2.5` ⇒ `double + double`

  Thus, the integer `3` is coerced to the `double` `3.0`.
- Example: convert a small character to a capital letter:

```
char small_y, big_y;
cin >> small_y;               // Character in small case
big_y = small_y + 'A' - 'a'; // Character in big case
```

- Here `big_y`, `small_y`, `'A'`, and `'a'` are "coerced" by "promoting" them to `int` before addition. The result is converted back (or coerced) to `char`.

3.2.2 Automatic Type Conversion and Casting

- Examples:

```
float x = 3.2;  // Initialize x with 3.2 by assignment
double y = 5.7; // Initialize y with 5.7 by assignment
short k = x;    // k = ?
int n;
n = y;          // n = ?
```

- Since `float`/`double` can hold numbers bigger than `short`/`int`, the assignments of `k` and `n` in the above program will cause the compiler to issue a **warning** — not an error:

```
a.cpp:9: warning: converting to 'short int' from 'float'
a.cpp:11: warning: converting to 'int' from 'double'
```

- A **narrowing conversion** changes a value to a data type that might not be able to hold some of the possible values.
- A **widening conversion** changes a value to a data type that can accommodate any possible value of the original data.

Truncation rather than Rounding

C++ uses truncation rather than rounding in converting a `float`/`double` to `short`/`int`/`long`. The fractional part is simply dropped.

- Motivating example:

```
int k = 5;
int n = 2;
float x = n/k;  // What is the value of x?
```

  In the above example, one can get `x = 0.4` by manually converting `n` and/or `k` from `int` to `float`/`double`.
- Syntax — `static_cast` for manual type casting:

  `static_cast<data-type>(value)`

- No more warning messages on narrowing conversion.

```
int k = 5, n = 2;
float x = static_cast<double>(n)/k;
float y = n/static_cast<double>(k);
float z = static_cast<double>(n)/static_cast<double>(k);
```

Cast Only One Operand

To force floating-point division it is enough to cast one of the two operands to `double` (or `float`); the other operand is then coerced automatically by the usual arithmetic conversions.

3.3 Constants

3.3.1 Literal and Symbolic Constants

- Constants represent fixed values, or permanent values that cannot be modified (in a program).
- Examples of literal constants:
  - `char` constants: `'a'`, `'5'`, `'\n'`
  - string constants: `"hello world"`, `"don't worry, be happy"`
  - `int` constants: `123`, `456`, `-89`
  - `double` constants: `123.456`, `-2.90E+11`
- A **symbolic constant** is a named constant with an identifier name.
- The rule for identifier names for constants is the same as that for variables. However, by convention, constant identifiers are written in capital letters.
- A symbolic constant must be defined and/or declared before it can be used (just like variables or functions).
- Once defined, symbolic constants cannot be changed!
- Syntax — constant definition:

```
const <data-type> <identifier> = <value>;
```

- Example:

```
const char BACKSPACE = '\b';
const float US2HK = 7.80;
const float HK2RMB = 0.86;
const float US2RMB = US2HK * HK2RMB;
```

3.3.2 Why Symbolic Constants?

Compared with literal constants, symbolic constants are preferred because they are:

- **More readable.** A literal constant does not carry a meaning. e.g. the number `320` cannot tell you that it is the enrollment quota of COMP2011 in 2015.

```
const int COMP2011_QUOTA = 320;
```

- **More maintainable.** In case we want to increase the quota to 400, we only need to make the change in one place: the initial value in the definition of the constant `COMP2011_QUOTA`.

```
const int COMP2011_QUOTA = 400;
```

- **Type-checked** during compilation.

Constants and Memory

Remark: unlike variable definitions, memory is not allocated for constant definitions with only few exceptions.

File: symbolic-constant.cpp — demonstrates defining symbolic constants with `const` and using them in expressions (computing required numbers of instructors, TAs, and classrooms with `ceil()` from `<cmath>`).

```
#include <iostream>     /* File: symbolic-constant.cpp */
#include <cmath>        // For calling the ceil() function
using namespace std;

int main()
{
    const int COMP2011_QUOTA = 320;
    const float STUDENT_2_PROF_RATIO = 100.0;
    const float STUDENT_2_TA_RATIO = 40.0;
    const float STUDENT_2_ROOM_RATIO = 100.0;

    cout << "COMP2011 requires "
         << ceil(COMP2011_QUOTA / STUDENT_2_PROF_RATIO)
         << " instructors, "
         << ceil(COMP2011_QUOTA / STUDENT_2_TA_RATIO)
         << " TAs, and "
         << ceil(COMP2011_QUOTA / STUDENT_2_ROOM_RATIO)
         << " classrooms" << endl;

    return 0;
}
```

3.4 Further Readings and Examples

3.4.1 Find Out the Size of a Data Type Using `sizeof`

File: value.cpp — demonstrates the `sizeof` operator, which reports the number of bytes occupied by each basic data type.

```
#include <iostream>     /* File: value.cpp */
using namespace std;

int main()
{
    cout << "sizeof(bool) = " << sizeof(bool) << endl;
    cout << "sizeof(char) = " << sizeof(char) << endl;
    cout << "sizeof(short) = " << sizeof(short) << endl;
    cout << "sizeof(int) = " << sizeof(int) << endl;
    cout << "sizeof(long) = " << sizeof(long) << endl;
    cout << "sizeof(long long) = " << sizeof(long long) << endl;
    cout << "sizeof(float) = " << sizeof(float) << endl;
    cout << "sizeof(double) = " << sizeof(double) << endl;
    cout << "sizeof(long double) = " << sizeof(long double) << endl;

    return 0;
}
```

- Sizes of the basic types on 2 computers:

	32-bit machine	64-bit machine
`sizeof(bool)`	1	1
`sizeof(char)`	1	1
`sizeof(short)`	2	2
`sizeof(int)`	4	4
`sizeof(long)`	4	8
`sizeof(long long)`	8	8
`sizeof(float)`	4	4
`sizeof(double)`	8	8
`sizeof(long double)`	12	16

- Note that the figures may be different on your computer.
- A 32(64)-bit machine uses CPUs of which the data bus width and memory address width are 32 (64) bits.

3.4.2 Representation of Floating-Point Data

- Many programming languages use the **IEEE 754** floating-point standard.
- A floating-point value is stored in 3 fields: 1 **sign bit**, an **exponent** field, and a **mantissa** field:

	sign bit	exponent (#bits)	mantissa (#bits)
`float`	1	8	23
`double`	1	11	52

- Binary representation of the mantissa, e.g.

$$1.011_2 = 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

- Binary representation of the exponent: signed integer.
- All floating-point data types in C++ are signed.
- ANSI C++ requires: size of `float` $\leq$ size of `double`.

Think About It

Can every real number be represented by `float` in C++?

3.4.3 Underflow and Overflow

- **Overflow**: occurs when a data type is used to represent a number larger than what it can hold, e.g.,
  - if you use a `short int` to store HK's population;
  - when a `short int` has its max value of 32767, and you want to add 1 to it.
- **Underflow**: occurs when a data type is used to represent a number smaller than what it can hold, e.g.,
  - use an `unsigned int` to store a negative number.

Out-of-Range Values Wrap Silently

Storing a value outside the range of an integral type does not raise a compile-time error. e.g. adding 1 to a `short int` holding its maximum value 32767 overflows the type, and assigning a negative number to an `unsigned int` underflows it.

- **Underflow**: when the negative exponent becomes too large to fit in the exponent field of the floating-point number.
- **Overflow**: when the positive exponent becomes too large to fit in the exponent field of the floating-point number.
- To prevent these from happening, use `double` if memory space allows.
- In fact, all floating literals (e.g., `1.23`) are treated as `double` unless explicitly specified by a suffix (e.g., `1.23f`).

Floating Literals Are `double` by Default

A literal like `1.23` has type `double`, not `float`. Append the suffix `f` (e.g., `1.23f`) when you really mean a `float`.

3.4.4 Modulo Arithmetic

- `mod` is used to get the remainder in an integer division:

$$\text{mod}(17, 5) = 17 \bmod 5 = \texttt{17\%5} = 2$$

- Strictly speaking, m mod n is defined only if n is positive.
- Most programming languages support negative divisors, and different languages may give you different results!
- In C++, the modulo arithmetic is supported by the **remainder operator %**, which allows negative divisors.

Think About It

What are the results of `(-17)%5`, `17%(-5)`, or `(-17)%(-5)`?

3.4.5 Priority Rules for the Usual Arithmetic Conversions for Binary Operations

1. If either operand is of type `long double`, convert the other operand also to `long double`.
2. If either operand is of type `double`, convert the other operand also to `double`.
3. If either operand is of type `float`, convert the other operand also to `float`.
4. Otherwise, the integral promotions shall be performed on both operands.

- Similar rules are used for integral promotion of the operands.
- Compute using integer arithmetic.

Think About It

What is the result of `3/4`?

4 Control Flow II

4.1 Nested Loops

- One may put a loop inside another loop, e.g. a `while` loop inside another `while` loop; these are called nested loops.

File: nested-while-avg.cpp — a `while` loop nested inside another `while` loop to compute the average score for several students.

- The outer loop handles one student per iteration; it repeats as long as the user answers `'y'` or `'Y'`.
- At the start of each outer iteration, the accumulator `sum_of_scores` is reset to zero and the assignment counter `j` is reset to 1.
- The inner loop reads `NUM_ASSIGNMENTS` scores and accumulates them.
- Remark: one should check input errors when reading `score`.

```
#include <iostream> /* File: nested-while-avg.cpp */
using namespace std;

int main()
{
    int NUM_ASSIGNMENTS = 5; // Uppercase variable doesn't change
    int j; // Assignment counter
```

```
    int score, sum_of_scores;
    char reply = 'y'; // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student? (y/n) " << endl;
    while ((cin >> reply)  &&  (reply == 'y' || reply == 'Y'))
    {
        sum_of_scores = 0;  // Reset the accumulator to zero
        j = 1;  // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << \
  " : ";
            cin >> score; // Remark: one should check input errors \
  here
            sum_of_scores += score;
            j++;
        }

        cout << "The average score = " << \
   sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student? (y/n) " ;
    }

    return 0;
}
```

File: multiplication-table.cpp — a `for` loop nested inside another `for` loop to print out the products $j * k$ where $j, k = 1, \ldots, 10$.

- `<iomanip>` is a library that helps control input/output formats.
- `setw(4)` sets the length of the output field to 4.
- The inner counter `k` is reset to 1 for each `j`, because it is declared in the header of the inner `for` loop and is therefore re-initialized on every outer iteration.

```
#include <iostream>     /* File: multiplication-table.cpp */
#include <iomanip>      // a library that helps control input/output \
  formats
using namespace std;

int main()
{
    // To print out products of j*k where j, k = 1,...,10
    for (int j = 1; j <= 10; ++j)
    {
        for (int k = 1; k <= 10; ++k) // Reset k=1 for each j. Why?
            cout << setw(4) << j*k;   // Set the length of output \
  field to 4
        cout << endl;
    }

    return 0;
}
```

4.1.1 `break` and `continue`

`break` and `continue`

- A `break` causes the innermost enclosing loop to exit immediately.
- A `continue` causes the next iteration of the enclosing loop to begin immediately. That is, in a `while` loop, control passes to test the boolean expression again imme-

diately.

File: break-avg.cpp — same averaging program as before, but a negative score stops the inner loop immediately with `break`.

- Question: what is the output with the input $4, 5, -6, 7, 8$?
- When `score` is -6, `break` exits the inner loop at once; `sum_of_scores` is $4 + 5 = 9$, so the average printed is $9/5 = 1$ (integer division), and the program asks for another student.

```
#include <iostream> /* File: break-avg.cpp */
using namespace std;

int main()
{
    int NUM_ASSIGNMENTS = 5; // Uppercase variable doesn't change
    int j; // Assignment counter
    int score, sum_of_scores;
    char reply = 'y'; // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student? (y/n) " << endl;
    while ((cin >> reply)  &&  (reply == 'y' || reply == 'Y'))
    {
        sum_of_scores = 0;  // Reset the accumulator to zero
        j = 1;  // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << \
  " : ";
            cin >> score; // Remark: one should check input errors \
  here

            if (score < 0)
                break;

            sum_of_scores += score;
            j++;
        }
        cout << "The average score = " << \
   sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student? (y/n) " ;
    }
    return 0;
} // Question: What is the output with the input: 4, 5, -6, 7, 8?
```

File: continue-avg.cpp — same averaging program, but a negative score is ignored with `continue`.

- Question: what is the output with the input $4, 5, -6, 7, 8$?
- When `score` is -6, `continue` skips both `sum_of_scores += score;` and `j++;`, so `j` is not incremented and the next input is read again for the same assignment number.

```
#include <iostream> /* File: continue-avg.cpp */
using namespace std;

int main()
{
    int NUM_ASSIGNMENTS = 5; // Uppercase variable doesn't change
    int j; // Assignment counter
    int score, sum_of_scores;
    char reply = 'y'; // 'y' for yes, 'n' for no; initialized to yes

    cout << "Enter scores for the first student? (y/n) " << endl;
    while ((cin >> reply)  &&  (reply == 'y' || reply == 'Y'))
```

```
    {
        sum_of_scores = 0;  // Reset the accumulator to zero
        j = 1;  // Reset the assignment counter to 1

        while (j <= NUM_ASSIGNMENTS)
        {
            cout << "Enter student's score for assignment #" << j << \
   " : ";
            cin >> score; // Remark: one should check input errors \
   here

            if (score < 0)
                continue;

            sum_of_scores += score;
            j++;
        }
        cout << "The average score = " << \
    sum_of_scores/NUM_ASSIGNMENTS << endl;
        cout << "Enter scores for another student? (y/n) " ;
     }
     return 0;
} // Question: What is the output with the input: 4, 5, -6, 7, 8 ?
```

File: break-example.cpp — `break` exits the loop when `j == 1`.

File: continue-example.cpp — `continue` skips the rest of the body when `j == 1`.

- Output of `break-example.cpp`: `Enter iteration 0`, `Leave iteration 0`, `Enter iteration 1`, then the loop exits.
- Output of `continue-example.cpp`: `Enter iteration 0`, `Leave iteration 0`, then `Enter iteration 1` printed forever.

```
/* File: break-example.cpp */
#include <iostream>
using namespace std;

int main()
{
    int j = 0;

    while (j < 3)
    {
        cout << "Enter iteration " << j << endl;

        if (j == 1)
            break;

        cout << "Leave iteration " << j << endl;
        j++;
    }

    return 0;
}

/* File: continue-example.cpp */
#include <iostream>
using namespace std;

int main()
{
    int j = 0;

    while (j < 3)
    {
        cout << "Enter iteration " << j << endl;
```

```
        if (j == 1)
            continue;

        cout << "Leave iteration " << j << endl;
        j++;
    }

    return 0;
}
```

`continue` may skip the loop update

In `continue-example.cpp`, `continue` skips `j++;` as well. Since `j` stays at 1, the condition `j < 3` remains true and the loop never terminates — an infinite loop. Make sure the loop variable is still updated (or the loop is exited some other way) when using `continue` in a `while` loop.

4.1.2 Where Does `continue;` Continue in a `for` Loop?

File: for-continue.cpp — shows what `continue` does inside a `for` loop.

- In a `for` loop, `continue` jumps to the update expression (`j++`), and the loop condition is then tested again.
- Here `j = 1, 2, 3` are printed; when `j == 3`, `j` is set to 10, then `continue` triggers `j++` making `j` equal to 11, so the loop ends.
- If `continue` is replaced by `break`, the output is the same in this example, because the loop ends right after either way.

```
#include <iostream>      /* File: for-continue.cpp */
using namespace std;

int main()
{
    for (int j = 1; j <= 10; j++)
    {
        cout << "j = " << j << endl;

        if (j == 3)
        {
            j = 10;
            continue;   // What if it is replaced by \
   \color{blue}{break};
        }
    }

    return 0;
}
```

4.2 Let's `switch`: C++ Multiple Choices

4.2.1 The `switch` Statement

- The `switch` statement is a variant of the `if-else-if` statement that allows multiple choices based on the value of an integral expression.

```
switch (integral expression)
{
    case constant-1:
        statement-sequence-1;
        break;
```

```
    case constant-2:
        statement-sequence-2;
        break;
    ...
    case constant-N:
        statement-sequence-N;
        break;
    default:    // optional
        statement-sequence-(N+1);
}
```

File: switch-find-comp2011-instructor.cpp — a `switch` on an `int` section number to determine your instructor; the `default` case reports an invalid section via `cerr`.

```
#include <iostream>     /* File: switch-find-comp2011-instructor.cpp \
   */
using namespace std;

int main()              // To determine your instructor
{
    cout << "Enter the COMP2011 section number to find its \
   instructor: ";
    int section;        // COMP2011 section number: should be 1, 2, \
   3, or 4
    cin >> section;     // Input COMP2011 section number

    switch (section)
    {
        case 1:
            cout << "Sergey Brin" << endl; break;
        case 2:
            cout << "Bill Gates" << endl; break;
        case 3:
            cout << "Steve Jobs" << endl; break;
        case 4:
            cout << "Jeff Bezos" << endl; break;
        default:
            cerr << "Error: Invalid lecture section " << section << \
   endl;
            break;
    }

    return 0;
}
```

File: switch-char-bloodtype.cpp — a `switch` on a `char` blood type to find out who may give you blood; the `default` case catches input errors.

- Case labels may be character constants such as `'A'`.

```
#include <iostream>     /* File: switch-char-bloodtype.cpp */
using namespace std;

int main()              // To find out who may give you blood
{
    cout << "Enter your blood type (put 'C' for blood type AB): ";
    char bloodtype; cin >> bloodtype;

    switch (bloodtype)
    {
        case 'A':
            cout << "Your donor must be of blood type: O or A\n";
            break;
        case 'B':
            cout << "Your donor must be of blood type: O or B\n";
            break;
```

```
        case 'C':
            cout << "Your donor must be of blood type: O, A, B, or \
   AB\n";
            break;
        case 'O':
            cout << "Your donor must be of blood type: O";
            break;
        default:        // To catch errors
            cerr << "Error: " << bloodtype << " is not a valid blood \
   type!\n";
            break;
    }
    return 0;
}
```

File: switch-int-grade.cpp — a `switch` on `mark/10` to determine a (fictitious) letter grade.

- Several cases may share the same action, e.g. `case 10:` and `case 9:` both give `'A'` (if `mark` ≥ 90).
- Several cases may be written on one line, e.g. `case 8:  case 7:  case 6:`.

```
#include <iostream>     /* File: switch-int-grade.cpp */
using namespace std;

int main()              // To determine your grade (fictitious)
{
    char grade;         // Letter grade
    int mark;           // Numerical mark between 0 and 100
    cin >> mark;

    switch (mark/10)
    {
        case 10:        // Several cases may share the same action
        case 9:
            grade = 'A'; break; // If mark >= 90
        case 8: case 7: case 6: // May write several cases on 1 line
            grade = 'B'; break; // If 90 > mark >= 60
        case 5:
        case 4:
        case 3:
        case 2:
            grade = 'C'; break; // If 60 > mark >= 20
        case 1:
            grade = 'D'; break; // If 20 > mark >= 10
        default:
            grade = 'F'; break;
    }

    cout << "Your letter grade is " << grade << endl;
    return 0;
}
```

File: if-elseif-grade.cpp — the same grading logic written with an `if-else-if` statement for comparison.

```
#include <iostream>     /* File: if-elseif-grade.cpp */
using namespace std;

int main()              /* To determine your grade (fictitious) */
{
    char grade;         // Letter grade
    int mark;           // Numerical mark between 0 and 100
    cin >> mark;
```

```
    if (mark >= 90)
        grade = 'A';     // mark >= 90
    else if (mark >= 60)
        grade = 'B';     // 90 > mark >= 60
    else if (mark >= 20)
        grade = 'C';     // 60 > mark >= 20
    else if (mark >= 10)
        grade = 'D';     // 20 > mark >= 10
    else
       grade = 'F';      // 10 > mark

    cout << "Your letter grade is " << grade << endl;
    return 0;
}
```

4.2.2 Remarks on `switch`

Remarks on `switch`

- The expression for `switch` must evaluate to an integral value (integer, `char`, `bool` in C++).
- No two cases may have the same value.
- On the other hand, several cases may share the same action statements.
- When a case constant is matched, the statements associated with the case are executed until either
  - a `break` statement,
  - a `return` statement, or
  - the end of the `switch` statement.
- Difference between a `switch` statement and an `if-else-if` statement:
  - A `switch` statement can only test for equality of the value of one quantity.
  - Each expression of an `if-else-if` statement may test the truth value of different quantities or concepts.

File: switch-no-break.cpp — what happens if you forget to `break`? The grading `switch` without any `break` statements.

- Without `break`, execution falls through into the following cases. For example, with `mark` = 95, `mark/10` is 9, and the program prints all of `A`, `B`, `C`, `D` and `F`.

```
#include <iostream>      /* File: switch-no-break.cpp */
using namespace std;

int main()               // To determine your grade (fictitious)
{
    char grade;          // Letter grade
    int mark;            // Numerical mark between 0 and 100
    cin >> mark;

    /* What happens if you forget to break? What is the output? */
    switch (mark/10)
    {
        case 10: case 9:
            cout << "Your grade is A" << endl;
        case 8: case 7: case 6:
            cout << "Your grade is B" << endl;
        case 5: case 4: case 3: case 2:
            cout << "Your grade is C" << endl;
        case 1:
            cout << "Your grade is D" << endl;
        default:
```

```
            cout << "Your grade is F" << endl;
    }
    return 0;
}
```

Forgetting `break` in a `switch`

When a case is matched, execution continues until a `break`, a `return`, or the end of the `switch`. If you forget `break`, control silently falls through into the next case(s), which is usually not intended. End each case with `break` unless the fall-through is deliberate (e.g. sharing cases).

4.2.3 New Data Types with `enum`

- One way to define a new data type is to use the keyword `enum`.

```
enum new-datatype { identifier1 [=value1], identifier2 [=value2], \
   ... };

enum weekday { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
               SATURDAY, SUNDAY };      // 0,1,2,3,4,5,6
enum primary_color { RED = 1, GREEN, BLUE }; // 1,2,3
enum bloodtype { A, B, AB = 10, O };    // 0,1,10,11
```

4.2.4 User-Defined `enum` Type

User-defined `enum` type

- An enumeration is a type that can hold a finite set of symbolic objects.
- The symbolic (meaningful) names of these objects follow the same rule as identifier names, and they make your program easier to read/understand.
- Internally, these objects are represented as integers.
- By default, the first object is given the value zero, then each subsequent object is assigned a value one greater than the previous object's value.
- The integral values of the enumerated objects may be assigned other integral values by the programmer.
- Thus, the objects of an `enum` type act like named integer constants.

File: enum-shapes.cpp — an `enum` declared inside `main()` whose constants are used as `case` labels of a `switch`.

- Printing the `enum` constants shows their underlying integer values: `TEXT = 0`, `LINE = 1`, `RECT = 2`, `CIRCLE = 3`.
- `myshape` is an `int`, not `shapes`, because `cin` cannot read directly into an `enum`; the `enum` constants are simply used as named integer constants in the `case` labels.

```
#include <iostream>     /* File: enum-shapes.cpp */
using namespace std;

int main()
{
    enum shapes { TEXT, LINE, RECT, CIRCLE };
    cout << "supported shapes: "
         << " TEXT = " << TEXT << " LINE = " << LINE
         << " RECT = " << RECT << " CIRCLE = " << CIRCLE << endl;
    int myshape; // Why the type of myshape is not shape?
    cin >> myshape;
```

```
    switch (myshape)
    {
        case TEXT:
            cout << "Call a function to print text" << endl; break;
        case LINE:
            cout << "Call a function to draw a line" << endl; break;
        case RECT:
            cout << "Call a function to draw a rectangle" << endl; \
   break;
        case CIRCLE:
            cout << "Call a function to draw a circle" << endl; break;
        default:
            cerr << "Error: Unsupported shape" << endl; break;
    }

    return 0;
}
```

4.3 Further Readings and Examples

4.3.1 Quiz: Logical Operations

Quiz: Logical Operations

What is the value of each of the following boolean expressions?

1. `4 == 5` ⇒ `false`
2. `x > 0 && x < 10` (if `int x = 5`) ⇒ `true`
3. `5 * 15 + 4 == 13 && 12 < 19 || !false == 5 < 24` ⇒ `true`
4. `true && false || true` ⇒ `true`
5. `x` (if `int x = 5`) ⇒ 5, which is non-zero, i.e. `true` as a boolean
6. `x++ == 6` (if `int x = 5`) ⇒ `false`; the old value 5 is compared, and `x` becomes 6 afterwards
7. `x = 9` ⇒ this is an assignment, not a comparison; its value is 9, i.e. `true` as a boolean
8. `x == 3 == 4` (assume `x` is an `int`) ⇒ `false`; it is evaluated as `(x == 3) == 4`, and `x == 3` is only 0 or 1

4.3.2 The `do-while` Loop

```
do { <stmts> } while (<bool-exp>);
```

`do-while` vs. `while`

- Like the `while` statement, `<stmts>` will be repeated as long as the value of `<bool-exp>` is true.
- Unlike the `while` statement, the `<bool-exp>` is evaluated after `<stmts>` at the bottom of the `do-while` statement.
- That means `<stmts>` in a `do-while` loop will be executed at least once, whereas `<stmts>` in a `while` loop may not be executed at all.

File: do-factorial.cpp — computes $x! = x(x-1)(x-2)\dots 1$ for a non-negative x with a `do-while` loop.

- `factorial *= number;` is the same as `factorial = factorial*number;`; `--number;` is the same as `number = number-1;`.
- The `if (number > 0)` guard keeps $0! = 1$ correct: `factorial` stays at its initial value 1.

```
#include <iostream>  /* File: do-factorial.cpp */
using namespace std; // Compute x! = x(x-1)(x-2)...1; x is non -ve

int main()
{
    int factorial = 1, number;
    cout << "Enter a non-negative integer: ";
    cin >> number;

    if (number > 0)
    {
        do
        {
            factorial *= number; // Same as: \scriptsize{factorial = \
   factorial*number}
            --number;            // Same as: \scriptsize{number = \
   number-1}
        } while (number > 1);
    }

    cout << factorial << endl;
    return 0;
}
```

4.3.3 Which Loop to Use?

Which loop to use?

- `for` loop:
  - When you know how to specify the required number of iterations.
  - When the counter variable is also needed for computation inside the loop, e.g. to compute sums, products, and to count.
- `while` loop:
  - You want to repeat an action but do not know exactly how many times it will be repeated.
  - The number of iterations is determined by a boolean condition, e.g. `while (cin >> x) { ... }`.
- `do-while` loop:
  - The associated actions have to be executed at least once.
  - Otherwise, `do-while` and `while` are used in similar situations.

4.3.4 Common Loop Errors

What is the error in each of the following cases?

Case 1: `sum` is used before being initialized — the accumulator starts from a garbage value.

```
int sum;
while (cin >> x)
    sum += x;
```

Case 2: `j` is not initialized — the loop condition `j < 10` is tested against a garbage value.

```
int j;
while (j < 10)
{
    cout << "hello again!" << endl;
    j++;
}
```

Case 3: the semicolon right after `while (j < 10)` is the whole loop body (an empty statement) — an infinite loop; the block below it is a separate statement executed at most once.

```
int j = 0;
while (j < 10);
{
    cout << "hello again!" << endl;
    j++;
}
```

Common loop errors

- Always initialize an accumulator (e.g. `sum`) before adding to it in a loop.
- Always initialize the loop-control variable (e.g. `j`) before testing it.
- Do not put a stray semicolon after the loop header: `while (j < 10);` creates an empty loop body, usually an infinite loop.

4.3.5 More on `enum`: Mixing Colors

File: enum-colors.cpp — mixing two primary colors represented by an `enum`; the color variables `x` and `y` are declared immediately after the `enum` definition.

- `static_cast<color>(xint)` converts an `int` to a `color` quantity.
- Check what is really printed out: printing an `enum` value prints its underlying integer (e.g. `YELLOW` prints 3, `CYAN` prints 4, `PURPLE` prints 5), not its name.

```
#include <iostream>     /* File: enum-colors.cpp */
using namespace std;

int main()
{   // Declare color variables immediately after the enum definition
    enum color { RED, GREEN, BLUE, YELLOW, CYAN, PURPLE } x, y;
    int xint, yint;     // Input variables for the color variables

    cin >> xint >> yint;
    x = static_cast<color>(xint); // Convert an int to a color \
   quantity
    y = static_cast<color>(yint); // Convert an int to a color \
   quantity

    if ( (x == RED && y == GREEN) || (y == RED && x == GREEN) )
        cout << YELLOW << endl;

    else if ( (x == RED && y == BLUE) || (y == RED && x == BLUE) )
        cout << PURPLE << endl;

    else if ( (x == GREEN && y == BLUE) || (y == GREEN && x == BLUE) )
        cout << CYAN << endl;

    else
        cerr << "Error: only support mixing RED/GREEN/BLUE!" << endl;

    return 0;
} // Check what is really printed out
```

5 Problems for Basics and Control

This chapter walks through four programming problems that exercise the basic control constructs of C++: selection (`if`, `switch`), iteration (`for`, `while`, `do-while`), and the usual arithmetic conversion rules. Each problem is developed incrementally: we start

from a simple first attempt, discuss what is wrong or missing, and refine it into the final solution.

5.1 Problem 1: Guess the Number

Problem Statement

- The program picks a random number in the range 1 to 100.
- Two players take turns to guess the number.
- After each guess, the program tells the player whether the answer is correct, larger than, or smaller than the guess.
- Whoever guesses correctly first wins the game.

Requirements

- Validate that a guessed number is in the range set by the program; keep asking the player to re-enter until the input is valid.
- Determine if a guess is correct and give suitable feedback.
- Keep running until a guess is correct.

Solution Development

We build the program in four steps.

1. **First attempt — 1 player, 1 round.** The answer is fixed beforehand. A single `if/else if/else` chain compares the guess with the answer and prints feedback.
2. **Second attempt — 1 player, multiple rounds.** Wrap the guessing code in a `do-while` loop so that the player can guess again until correct. A `do-while` is natural here because the body must run at least once.
3. **Third attempt — fixed range with input validation.** Two variables `low` and `high` record the current valid range. An inner `while` loop rejects out-of-range input. After each wrong guess the range is narrowed: `low = guess + 1` when the guess is too small, and `high = guess - 1` when it is too big.
4. **Final version — 2 players, random answer.** The answer is produced by the random number generator (RNG): `srand(time(0))` seeds the RNG with the current time (seconds since 1970/01/01 00:00:00), and `rand() % 100 + 1` maps the result into [1..100]. The variable `player` tracks whose turn it is; `player = (player % 2) + 1` flips $1 \to 2$ and $2 \to 1$.

Incremental Development

Start with the simplest version that works, then add one feature at a time: loop for multiple rounds, input validation with a range, and finally random number generation plus a second player. Testing each step is much easier than debugging the full program at once.

while vs. do-while for Validation

The validation loop in `guess3.cpp` is a `while` loop, because the first guess has already been read before validation starts. If you rewrite it with `do-while`, make sure you do not read the input twice before the first check.

Random Numbers

`rand()` (from `<cstdlib>`) returns an `int` between 0 and $2^{31} - 1$. Always call `srand()` once at the beginning; seeding with `time(0)` (declared in `<time.h>`) gives a different sequence every run. Use `rand() % n + a` to get a value in $[a, a + n - 1]$.

File: guess1.cpp

```
#include <iostream>     /* File: guess1.cpp */
using namespace std;

int main()              // 1st attempt: 1 player, 1 round
{
    int number = 10;    // The answer is fixed beforehand
    int guess;

    cout << "Player 1, please enter your guess:" << endl;
    cin >> guess;

    if (guess == number)
        cout << "Player 1, you win!!!" << endl;

    else if (guess < number)
        cout << "Sorry, the number is bigger than " << guess << endl;

    else
        cout << "Sorry, the number is smaller than " << guess << endl;

    return 0;
}
```

File: guess2.cpp

```
#include <iostream>     /* File: guess2.cpp */
using namespace std;

int main()              // 2nd attempt: 1 player, multiple rounds
{
    int number = 10;    // The answer is fixed beforehand
    int guess;

    do                  // Add a loop to implement multiple rounds
    {
        cout << "Player 1, please enter your guess:" << endl;
        cin >> guess;

        if (guess == number)
            cout << "Player 1, you win!!!" << endl;

        else if (guess < number)
            cout << "Sorry, the number is bigger than "
                 << guess << endl;
        else
            cout << "Sorry, the number is smaller than "
                 << guess << endl;
    } while (guess != number);

    return 0;
}
```

File: guess3.cpp

```
#include <iostream>     /* File: guess3.cpp */
using namespace std;

int main()    // 3rd attempt: 1 player, multiple rounds, fixed range
{
    int number = 10;               // The answer is fixed beforehand
    int guess;
    int low = 1, high = 100;  // Add 2 variables to record the range

    do // Add a loop to implement multiple rounds
    {
        cout << "Player 1, please enter your guess:" << endl;
        cin >> guess;
```

```
        while (guess < low || guess > high) // Input validation loop
        {
            cout << "Invalid input, please enter a number between "
                 << low << " and " << high << endl;
            cin >> guess;
        }   // Can this loop be replaced with do-while?

        if (guess == number)
            cout << "Player 1, you win!!!" << endl;

        else if (guess < number)
        {
            cout << "Sorry, the number is bigger than "
                 << guess << endl;
            low = guess + 1; // Update the lower bound of the range
        }
        else
        {
            cout << "Sorry, the number is smaller than "
                 << guess << endl;
            high = guess - 1; // Update the upper bound of the range
        }
    } while (guess != number);

    return 0;
}
```

File: guess-number.cpp

```
#include <iostream>     /* File: guess_number.cpp */
#include <cstdlib>      // Needed for calling the rand() function
#include <time.h>       // May need for calling the time() function
using namespace std;

int main()   // 2 players, multiple rounds, fixed range, random number
{
    /* Random number generation RNG */
    // time(0) number of seconds since 1970/01/01 00:00:00
    // rand() returns an int 0 to 2^31-1
    srand(time(0));                 // (1) Seed the RNG
    int number = rand() % 100 + 1; // (2) a random no. in [1..100]

    int guess;
    int low = 1, high = 100;
    int player = 1;     // Set Player 1 as the current player

    cout << "The generated number is: " << number << endl;
    do
    {
        cout << "Player " << player
             << ", please enter your guess: " << endl;
        cin >> guess;

        while (guess < low || guess > high) // Input validation loop
        {
            cout << "Invalid input, please enter a number between "
                 << low << " and " << high << endl;
            cin >> guess;
        }

        if (guess == number)
            cout << "Player " << player <<", you win!!!" << endl;
        else if (guess < number)
```

```
        {
            cout << "Sorry, the number is bigger than "
                 << guess << endl;
            low = guess + 1; // Update the lower bound of the range
        }
        else
        {
            cout << "Sorry, the number is smaller than "
                 << guess << endl;
            high = guess - 1; // Update the upper bound of the range
        }

        player = (player % 2) + 1; // This makes 1 $\rightarrow$ 2 \
   and 2 $\rightarrow$ 1
    } while (guess != number);

    return 0;
}
```

5.2 Problem 2: Drawing Right-Angled Triangles (RATs)

Problem Statement

Design a program that prints isosceles right-angled triangles (RATs) and allows the user to set the size. A RAT of size 4 looks like:

```
*
**
***
****
```

Variations to try: a fat RAT, a hollow RAT, an upside-down RAT, and finally a whole row of RATs whose count depends on the screen width.

Solution Points

- A single RAT needs two nested `for` loops: the outer loop runs once per row (= height), and the inner loop prints the stars of one row (row i has width i).
- **Fat RAT:** row i has $2i - 1$ stars.
- **Hollow RAT:** print `'*'` only on the boundary — when $j = 0$, $j = i - 1$, or on the last row (i = size) — and a space otherwise. The conditional operator `(cond) ? '*' : ' '` keeps the inner loop to one statement.
- **Upside-down RAT:** simply run the outer loop downwards from `size` to 1.
- **Row of RATs:** the number of RATs that fit is $\lfloor$`TOTAL_NUM_COLUMNS/size`$\rfloor$. Three nested loops are needed (row, which RAT, column), and each RAT must be padded with `size - i` trailing spaces so that the next RAT in the same row lines up correctly.

File: one-rat.cpp

```
#include <iostream>     /* File: one-rat.cpp */
using namespace std;

int main()
{
    cout << "Size of a RAT: " << endl;
    int size;                          // height == width == size
    cin >> size;

    for (int width = 1; width <= size; width++) // #iters==height
    {
        // Draw one row of a RAT
        for (int j = 0; j < width; j++)         // width of a row
```

```
            cout << '*';
        cout << endl;
    }
    return 0;
}
```

File: various-rats.cpp

```
#include <iostream>     /* File: various-rats.cpp */
using namespace std;

int main()
{
    cout << "Size of a RAT: " << endl;
    int size;                                   // height = width = size
    cin >> size;

    cout << "A fat RAT" << endl;
    for (int i = 1; i <= size; i++)             // #iterations = height
    {
        for (int j = 0; j < i*2 - 1 ; j++) // width of a row
            cout << '*';
        cout << endl;
    }

    cout << "A hollow RAT" << endl;
    for (int i = 1; i <= size; i++)
    {
        for (int j = 0; j < i ; j++)
            cout << ((j == 0 || j == i - 1 || i == size) ? '*' : ' ');
        cout << endl;
    }

    cout << "An upside-down RAT" << endl;
    for (int i = size; i >= 1; i--)
    {
        for (int j = 0; j < i; j++)
            cout << '*';
        cout << endl;
    }

    return 0;
}
```

What's Wrong with These Programs?

Missing Braces Change the Loop Body

In the second loop of `one-bad-rat.cpp`, the braces around the inner loop body make `cout << endl;` execute after every star instead of once per row — the output is one star per line, not a triangle. When a loop has no braces, only the single next statement belongs to it.

File: one-bad-rat.cpp

```
#include <iostream>     /* File: one-bad-rat.cpp */
using namespace std;

int main()
{
    cout << "Size of a RAT: " << endl;
```

```
    int size;
    cin >> size;

    cout << "A simple RAT:" << endl;
    for (int i = 0; i < size; i++)
    {
        for (int j = 0; j <= i; j++)
            cout << '*';
        cout << endl;
    }

    cout << "Is this a RAT?" << endl;
    for (int i = 1; i <= size; i++)
        for (int j = 0; j < i * 2 - 1 ; j++)
        {
            cout << '*';
            cout << endl;
        }

    return 0;
}
```

Row of RATs: Two Ways to Go Wrong

- `row-of-bad-rats1.cpp` prints the stars of each RAT but never the separating spaces, so all RATs in a row merge into one wide blob.
- `row-of-bad-rats2.cpp` adds the padding spaces but puts the `cout << endl;` inside the per-RAT loop, so each RAT is printed on its own set of lines — the RATs stack vertically instead of sitting side by side.
- Fix (`row-of-rats.cpp`): the newline belongs to the outermost (row) loop only, and each RAT prints i stars followed by `size` $- i$ spaces.

File: row-of-bad-rats1.cpp

```
#include <iostream>     /* File: row-of-bad-rats1.cpp */
using namespace std;

int main()
{
    cout << "Size of a triangle: " << endl;
    int size;
    cin >> size;

    // Find out the number of RATs in a row
    const int TOTAL_NUM_COLUMNS = 105; // Assumed screen width
    int num_RATs = TOTAL_NUM_COLUMNS / size;

    for (int i = 1; i <= size; i++)
    {
        for (int n = 0; n < num_RATs; n++)
            for (int j = 0; j < i ; j++)
                cout << '*';

        cout << endl;
    }

    return 0;
}
```

File: row-of-bad-rats2.cpp

```
#include <iostream>     /* File: row-of-bad-rats2.cpp */
using namespace std;

int main()
{
```

```
    cout << "Size of a triangle: " << endl;
    int size;
    cin >> size;

    const int TOTAL_NUM_COLUMNS = 50; // Assumed screen width
    int num_RATs = TOTAL_NUM_COLUMNS / size;

    for (int i = 1; i <= size; i++)
        for (int n = 0; n < num_RATs; n++)
        {
            for (int j = 0; j < i ; j++)
                cout << '*';
            for (int j = 0; j < (size-i); j++)
                cout << ' '; // Print enough spaces after a RAT

            cout << endl;
        }

    return 0;
}
```

File: row-of-rats.cpp

```
#include <iostream>             /* File: row-of-rats.cpp */
using namespace std;

int main()
{
    cout << "Size of a triangle: " << endl;
    int size;
    cin >> size;

    // Find out the number of RATs in a row
    const int TOTAL_NUM_COLUMNS = 105; // Assumed screen width
    int num_RATs = TOTAL_NUM_COLUMNS / size;

    for (int i = 1; i <= size; i++)
    {
        for (int n = 0; n < num_RATs; n++)
        {
            for (int j = 0; j < i ; j++)
                cout << '*';
            for (int j = 0; j < (size-i); j++)
                cout << ' '; // Print enough spaces after each RAT
        }
        cout << endl;
    }

    return 0;
}
```

5.3 Problem 3: Count Animals

Problem Statement

A farm has pigs (4.5 weight units each) and sheep (3 units each). The total weight in a barn must be exactly 36 units. List all possible combinations of pigs and sheep. Expected solutions: $0\times4.5+12\times3=36$, $2\times4.5+9\times3=36$, $4\times4.5+6\times3=36$, $6\times4.5+3\times3=36$, $8\times4.5+0\times3=36$.

Solution Points

- Brute force: try every possible number of pigs (`num_pigs * wt_pig <= total_wt`), compute the remaining weight, and check whether it can be made up exactly by sheep.

- **First attempt is broken:** `remain_wt % wt_sheep` uses the `%` operator on `float` operands — the remainder operator `%` is defined for integers only, so the program does not even compile.
- **Second attempt compiles** (replace `%` by subtraction of `num_sheeps * wt_sheep`) but is still unreliable: the test `remain_wt == 0` compares floating-point numbers for exact equality, which is dangerous because values like 4.5 and 0.1 cannot be represented exactly in binary floating point.

File: two-animals-v1.cpp

```
#include <iostream>     /* File: two_animals-v1.cpp */
using namespace std;

int main()
{
    float wt_pig = 4.5;
    float wt_sheep = 3.0;
    float total_wt = 36;

    for (int num_pigs = 0; num_pigs * wt_pig <= total_wt; num_pigs++)
    {
        float remain_wt = total_wt - num_pigs * wt_pig;
        int num_sheeps = remain_wt / wt_sheep;
        remain_wt = remain_wt % wt_sheep;

        if (remain_wt == 0)
            cout << num_pigs << " * " << wt_pig << " + "
                 << num_sheeps << " * " << wt_sheep << " = "
                 << total_wt << endl;
    }
    return 0;
}
```

File: two-animals-v2.cpp

```
#include <iostream>     /* File: two_animals-v2.cpp */
using namespace std;

int main()
{
    float wt_pig = 4.5;
    float wt_sheep = 3.0;
    float total_wt = 36;

    for (int num_pigs = 0; num_pigs * wt_pig <= total_wt; num_pigs++)
    {
        float remain_wt = total_wt - num_pigs * wt_pig;
        int num_sheeps = remain_wt / wt_sheep;
        remain_wt -= num_sheeps * wt_sheep;

        if (remain_wt == 0)
            cout << num_pigs << " * " << wt_pig << " + "
                 << num_sheeps << " * " << wt_sheep << " = "
                 << total_wt << endl;
    }
    return 0;
}
```

Why Comparing Floating-Point Numbers Fails

Adding `0.1` ten times does not give exactly the same bits as computing `10.0 * 0.1`, so `sum == product` prints `0` (false). With `cout.precision(10)` and `boolalpha` the difference becomes visible.

File: float-comparison-v1.cpp

```
#include <iostream>     /* File: float-comparison-v1.cpp */
using namespace std;

int main()
{
    float x = 0.1;
    float product = 10.0 * x;

    float sum = 0.0;
    for (int i = 0; i < 10; ++i)
        sum += x;

    cout << "sum = " << sum << endl;
    cout << "product = " << product << endl;
    cout << "10.0 * x = " << 10.0 * x << endl;
    cout << (sum == product) << endl;

    return 0;
}
```

File: float-comparison.cpp

```
#include <iostream>     /* File: float-comparison.cpp */
using namespace std;

int main()
{
    float x = 0.1;
    float product = 10.0 * x;

    float sum = 0.0;
    for (int i = 0; i < 10; ++i)
        sum += x;

    // Set output precision to 10 significant figures
    cout.precision(10);
    // Print boolean outputs as true or false instead of 1 and 0
    cout << boolalpha;  // boolean + alpha

    cout << "sum = " << sum << endl;
    cout << "product = " << product << endl;
    cout << "10.0 * x = " << 10.0 * x << endl;
    cout << (sum == product) << endl;
    return 0;
}
```

Never Compare Floats with ==

`%` works only on integers, and binary floating point cannot represent most decimal fractions exactly. Avoid exact-equality tests such as `remain_wt == 0`; reformulate the problem with integers when possible (e.g. count in units of 0.5: pig = 9, sheep = 6, total = 72), or test $|a - b| < \epsilon$ for a small ϵ.

Further Work

- What if the total number of animals must not exceed 10? Add a counter and an extra condition (`two-animals-extra.cpp`). Note that this version avoids `%` by repeatedly subtracting `w2` and counting how many sheep fit.
- What if there are three types of animals? Add one more nested loop (`three-animals.cpp`); the same pattern extends to any number of types.
- A recursive solution is also possible if you already know recursion.

File: two-animals-extra.cpp

```
#include <iostream>                    //two_animals_extra.cpp
```

```
using namespace std;
//Assume there are not more than 10 animals in the farm.
int main()
{
    double w1 = 4.5;
    double w2 = 3;
    double total = 36;
    int not_more_than = 10;
    for (int i = 0; i * w1 < total; i++)
    {
        double remaining = total - i * w1;
        int j = 0;
        for (; remaining > 0; j++)
            remaining -= w2;
        if (remaining == 0 && (i + j) <= not_more_than)
            cout << i << " * " << w1 << " + "
                 << j << " * " << w2 << " = "
                 << total << endl;
    }
}
```

File: three-animals.cpp

```
#include <iostream>             //three_animals.cpp
using namespace std;
//assume the weight of the three types of animals are 4.5, 3, and \
   2.5 units
int main()
{
    double w1 = 4.5, w2 = 3, w3 = 2.5;
    double total = 36;
    for (int k = 0; k * w3 < total; k++) //an extra loop k is added
    {
        double remaining1 = total - k * w3; //total $\rightarrow$ \
   remaining1
        for (int i = 0; i * w1 < remaining1; i++)
        {
            double remaining = remaining1 - i * w1;
            int j = 0;
            for (; remaining > 0; j++)
                remaining -= w2;
            if (remaining == 0)
                cout << i << " * " << w1 << " + "
                     << j << " * " << w2 << " + "
                     << k << " * " << w3 << " = "
                     << total << endl;
        }
    }
}
```

5.4 Problem 4: GPA Calculator

Problem Statement

Letter grades convert to grade points as: A = 4.0, B = 3.0, C = 2.0, D = 1.0, F = 0.0. Design a program that calculates a student's GPA (grade point average).

Requirements

- The student first enters the number of credits of a course; the program stops when the number of credits is ≤ 0.

- The student then enters the letter grade (A, B, C, D or F). Invalid grades are rejected and the grade is re-entered.
- The program accumulates the total credits and computes

$$\text{GPA} = \frac{\sum_{i=1}^{n}(\text{grade}_i \times \text{credit}_i)}{\sum_{i=1}^{n}\text{credit}_i}.$$

Program Design

Major components:

1. A loop for each course taken: (a) ask for the number of credits, (b) ask for the letter grade, (c) convert the letter grade to grade points, (d) accumulate total credits and total grade points.
2. After the loop, calculate and output the GPA.

Variables needed: the number of credits, the letter grade, the converted grade points, the sum of credits, and the sum of grade points.

Integer Division Kills the GPA

If `total_grade_points` is declared as `int`, then `total_grade_points / total_num_credits` is integer division and the GPA loses its fraction (e.g. 2.88889 becomes 2). Recall the usual arithmetic conversion rules: if all operands are `int`, the computation uses integer arithmetic; if one operand is `double`/`float`, the other is converted and the result is floating point. So declare `total_grade_points` as `double`.

File: gpa_variables.cpp

```
char grade;
/* Don't forget to initialize some of these variables */
int num_credits, total_num_credits = 0;
int total_grade_points = 0; // Is it a good idea to use integer type \
   here?

// Think about the output
cout << "You have taken a total of " << total_num_credits
     << " credits ..." << endl << "and your GPA is "
     << total_grade_points / total_num_credits << endl;
```

Converting the Grade: if-else vs. switch

Both an `if-else if` chain and a `switch` statement can convert the letter grade. Note the trick used to reject invalid input: the credits are added to `total_num_credits` before the conversion, and the `else`/`default` branch cancels the addition for an invalid grade.

File: gpa_if1.cpp

```
char grade;
int num_credits, total_num_credits = 0;
double total_grade_points = 0;

cout << "No. of credits of your course (0 to stop): ";
cin >> num_credits;
cout << "Your letter grade (A, B, C, D or F): ";
cin >> grade;
total_num_credits += num_credits; // Update total no. of credits

if (grade == 'A')       // Convert letter grade to grade point
    total_grade_points += num_credits * 4;
else if (grade == 'B')
    total_grade_points += num_credits * 3;
```

```
else if (grade == 'C')
    total_grade_points += num_credits * 2;
else if (grade == 'D')
    total_grade_points += num_credits * 1;
else if (grade == 'F')
    total_grade_points += num_credits * 0;
else
    total_num_credits -= num_credits;
```

File: gpa_switch1.cpp

```
/*
 * Codes for variables definition and initialization, and inputs
 */

switch (grade)          // Convert letter grade to grade point
{
    case 'A':
        total_grade_points += num_credits * 4; break;
    case 'B':
        total_grade_points += num_credits * 3; break;
    case 'C':
        total_grade_points += num_credits * 2; break;
    case 'D':
        total_grade_points += num_credits * 1; break;
    case 'F':
        total_grade_points += num_credits * 0; break;
    default:
        total_num_credits -= num_credits;
}
```

Allowing Lowercase Grades

To accept both `'A'` and `'a'`, the `if` version tests both with `||`; the `switch` version stacks two case labels — `case 'A':` has no `break`, so control falls through into `case 'a':`.

Intentional Fall-Through

Stacking case labels (`case 'A': case 'a': ...`) is the idiomatic way to share one body among several values in a `switch`. It is one of the few places where omitting `break` is correct — everywhere else a missing `break` is a bug.

File: gpa_if2.cpp

```
/*
 * Codes for variables definition and initialization, and inputs
 */

if ((grade == 'A') || (grade == 'a'))
    total_grade_points += num_credits * 4;

else if ((grade == 'B') || (grade == 'b'))
    total_grade_points += num_credits * 3;

else if ((grade == 'C') || (grade == 'c'))
    total_grade_points += num_credits * 2;

else if ((grade == 'D') || (grade == 'd'))
    total_grade_points += num_credits * 1;

else if ((grade == 'F') || (grade == 'f'))
    total_grade_points += num_credits * 0;

else
    total_num_credits -= num_credits;
```

File: gpa_switch2.cpp

```
switch (grade)          // Convert letter grade to grade point
{
    case 'A':
    case 'a':
        total_grade_points += num_credits * 4; break;
    case 'B':
    case 'b':
        total_grade_points += num_credits * 3; break;
    case 'C':
    case 'c':
        total_grade_points += num_credits * 2; break;
    case 'D':
    case 'd':
        total_grade_points += num_credits * 1; break;
    case 'F':
    case 'f':
        total_grade_points += num_credits * 0; break;
    default:
        total_num_credits -= num_credits;
}
```

Complete Program

The final version uses named constants (`const double A = 4.0;` etc.) instead of magic numbers, a `do-while(true)` loop terminated by `break` when the credit input is ≤ 0, and a `switch` with stacked case labels for uppercase/lowercase grades.

Why `while (true)` Is Not an Infinite Loop

The loop condition is always true, but the `break` inside the body (after checking `num_credits <= 0`) provides the exit. This "loop-and-a-half" pattern is common when the termination test naturally belongs in the middle of the loop body.

File: gpa.cpp

```
#include <iostream>     /* File: gpa.cpp */
using namespace std;

const double A = 4.0;   // Definition of Constants
const double B = 3.0;
const double C = 2.0;
const double D = 1.0;
const double F = 0.0;

int main()
{   // Variables definition and initialization
    char grade;
    int num_credits, total_num_credits = 0;
    double total_grade_points = 0;

    do
    {
        cout << "No. of credits of your course (0 to stop): ";
        cin >> num_credits;

        if (num_credits <= 0) // What does this do?
            break;

        cout << "Your letter grade (A, B, C, D or F): ";
        cin >> grade;
        total_num_credits += num_credits; // Update total no. of \
   credits
```

```
        switch (grade)  // Convert letter grade to grade point
        {
            case 'A':   // No break here; execute code in case 'a'
            case 'a':
                total_grade_points += num_credits * A; break;
            case 'B':
            case 'b':
                total_grade_points += num_credits * B; break;
            case 'C':
            case 'c':
                total_grade_points += num_credits * C; break;
            case 'D':
            case 'd':
                total_grade_points += num_credits * D; break;
            case 'F':
            case 'f':
                total_grade_points += num_credits * F; break;
            default :
                cout << "Invalid input, please enter your grade \
   again!\n";
                total_num_credits -= num_credits;
        }
    } while (true);     // Why is this not an infinite loop?

    cout << "You have taken a total of " << total_num_credits
         << " credits ..." << endl << "and your GPA is "
         << total_grade_points / total_num_credits << endl;

    return 0;
}
```

Two More Complete Versions

gpa_no_constant.cpp is the same program written with literal floating-point numbers (4.0, 3.0, ...) instead of named constants — it works, but named constants make the code easier to read and modify. gpa_int.cpp deliberately uses const int grade points and an int accumulator: it compiles, but the final division is integer division, so the printed GPA is truncated — a concrete demonstration of the integer-division trap discussed above.

File: gpa_no_constant.cpp

```
#include <iostream>                          //gpa.cpp
using namespace std;

int main()
{
    char grade;
    double total_grade_points = 0;
    int credits = 0, total_credits = 0;
    do
    {
        cout << "Credits of your course (0 to stop):";
        cin >> credits;
        if (credits <= 0)
            break;
        total_credits += credits;
        cout << "Your letter grade: A B C D F:";
        cin >> grade;
        switch (grade)
        {
```

```
            case 'A':  //no break here, execute code in case 'a'
            case 'a':  total_grade_points += credits * 4.0;
                       break;
            case 'B':
            case 'b':  total_grade_points += credits * 3.0;
                       break;
            case 'C':
            case 'c':  total_grade_points += credits * 2.0;
                       break;
            case 'D':
            case 'd':  total_grade_points += credits * 1.0;
                       break;
            case 'F':
            case 'f':  total_grade_points += credits * 0.0;
                       break;
            default :  cout << "Invalid input" << endl;
                       total_credits -= credits;
        }
    } while (true);   //why this is not an infinite loop?

    cout << "You have totally taken " << total_credits
         << " credits." << endl << "Your GPA is "
         << total_grade_points / total_credits << endl;

    return 0;
}
```

File: gpa_int.cpp

```
#include <iostream>                          //gpa.cpp
using namespace std;

const int A = 4;
const int B = 3;
const int C = 2;
const int D = 1;
const int F = 0;

int main()
{
    char grade;
    int grade_point = 0;
    int credit = 0, total_credit = 0;
    do
    {
        cout << "Credit of your course (0 to stop):";
        cin >> credit;
        if (credit <= 0)
            break;
        total_credit += credit;
        cout << "Your letter grade: A B C D F:";
        cin >> grade;
        switch (grade)
        {
            case 'A':  //no break here, execute code in case 'a'
            case 'a':  grade_point += credit * A;
                       break;
            case 'B':
            case 'b':  grade_point += credit * B;
                       break;
            case 'C':
            case 'c':  grade_point += credit * C;
                       break;
            case 'D':
            case 'd':  grade_point += credit * D;
                       break;
```

```
            case 'F':
            case 'f':  grade_point += credit * F;
                       break;
            default :  cout << "Invalid input" << endl;
                       total_credit -= credit;
        }
    } while (true);   //why this is not an infinite loop?

    cout << "Your have totally taken " << total_credit
         << " credits." << endl << "Your GPA is "
         << grade_point / total_credit << endl;

    return 0;
}
```

6 Function I

In science we write $z = f(u, v, w, x, y, \ldots)$; familiar examples are `sin(x)`, `cos(x)`, `log(x)`, `sqrt(x)`, etc. This chapter generalizes the notion of functions to programming: a function receives input parameters and produces a returned result.

6.1 Function Basics

6.1.1 Basic Function Syntax and Function Call

- A function definition has the form:

```
<return-type> <function-name> ( <formal-parameter-list> )
                    { <function-body> }
```

- The first line is called the function header; the part enclosed in curly braces is the function body.

Function Syntax at a Glance

A definition = header (`<return-type> <function-name>(<formal-parameter-list>)`) plus a body in braces. A call uses the function name followed by an actual parameter list: `<function-name>(<actual-parameter-list>)`.

- Any legal C++ identifier can be used for `<function-name>`.
- As with variables and constants, use meaningful names: the name should describe what the function does.
- The name `main` is reserved: you must define it, and define it exactly once — each program can only have one `main()` function.
- When a program is run, the shell (the command interpreter of the operating system) looks for `main()` and starts execution there.
- A function is invoked by a function call: `<function-name>(<actual-parameter-list>)`.

6.1.2 Formal Parameter List and Actual Parameter List

- The `<formal-parameter-list>` appears in the function definition: it is a list of variable declarations separated by commas: `<type1 variable1>, <type2 variable2>, ..., <typeN variableN>`.
- The `<actual-parameter-list>` appears in a function call: it is a list of comma-separated objects passed to the called function: `<object1>, <object2>, ..., <objectN>`.
- There is a one-to-one correspondence between the actual parameters (a.k.a. arguments) and the formal parameters.

- During the call, the following initializations are performed: `type1 variable1 = object1`, `type2 variable2 = object2`, ..., `typeN variableN = objectN`.

Arguments Initialize Parameters

C++ is strongly typed: the type of each actual parameter must be the same as (or "matched" with) its corresponding formal parameter, and the compiler performs type checking. The only exception is when an automatic type conversion — coercion — can be done, just like normal initialization or assignment of an object to a variable of a different type.

6.1.3 Function Header and Function Body

The function body usually consists of:

- constant declarations,
- variable declarations and definitions,
- other C++ statements,
- the `return` statement.

Braces Are Always Required

It is legal to have an empty function body, but the curly braces must be there even if there is zero or only one statement inside — this is different from the `if`-statement or `while`-statement.

6.1.4 Return Type and the `return` Statement

- Usually a function returns something — in C++ we call it an object. The returned object may be:
  - a signal telling the caller the status of the function (did it run successfully? did it fail?),
  - the result of some computation, e.g. factorial, sum, etc.,
  - a new object created by the function, e.g. a new window.
- `<return-type>` specifies the data type of the single returned object. It can be any C++ built-in type (`char`, `int`, etc.) or user-defined type, except the array type.

Returning Multiple Objects

Since only a single object is returned by a function, how can you return multiple objects back to the caller? (One answer appears later in this chapter: pass extra arguments by reference.)

Syntax: `return <expression>;`

- A `return` statement gives "2" things back to the caller:
  1. program control: it stops running the called function; the caller takes back control and continues its execution;
  2. an object: the object (value) represented by `<expression>` is returned to the caller.
- If a function has a return value, its body must contain at least one `return` statement.

Return Type Mismatch

The value of `<expression>` should have the same type as the declared `<return-type>`, or be convertible to it by coercion; otherwise it is a compilation error.

A first complete function: `max` returns the greater of two integers. Note the two `return` statements.

File: max.cpp — finds the greater value between two input numbers.

```
#include <iostream>      /* File: max.cpp */
using namespace std;     /* To find the greater value between x and y \
   */

int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}   // Question: can you write with only 1 return statement?

int main()
{
    int x, y;
    cout << "Enter 2 numbers: ";
    cin >> x >> y;

    cout << "The bigger number is " << max(x, y) << endl;
    return 0;
}
```

One return Statement Only?

Can you rewrite `max` with only one `return` statement? (Hint: store the answer in a variable, or use the conditional operator `(a > b) ? a : b`.)

The function below computes the Euclidean distance between two points; it uses `sqrt()` from the math library `<cmath>`. `main()` then finds the lengths of the three sides of a triangle by calling the same function three times with different arguments.

File: distance-fcn.cpp — Euclidean distance between 2 points and the side lengths of a triangle.

```
#include <iostream>      /* File: distance-fcn.cpp */
#include <cmath>         // Math library info
using namespace std;

/* To find the Euclidean distance between 2 points */
double  euclidean_distance(double x1, double y1, double x2, double y2)
{
    double x_diff = x1 - x2;
    double y_diff = y1 - y2;
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

int main()   /* To find the length of the sides of a triangle */
{
    double xA, yA, xB, yB, xC, yC;
    cout << "Enter the co-ordinates of point A: "; cin >> xA >> yA;
    cout << "Enter the co-ordinates of point B: "; cin >> xB >> yB;
    cout << "Enter the co-ordinates of point C: "; cin >> xC >> yC;

    cout << " AB = " << euclidean_distance(xA, yA, xB, yB) << endl;
    cout << " BC = " << euclidean_distance(xB, yB, xC, yC) << endl;
    cout << " CA = " << euclidean_distance(xC, yC, xA, yA) << endl;
    return 0;
}
```

6.1.5 `void`: A New Type

- `void` means nothing, emptiness.
- A function that returns nothing to the caller has return type `void`.

- A function that takes no arguments from the caller may either leave the `<formal-parameter-list>` empty, e.g. `int fcn_example() { ...}`, or write `void` in the list, e.g. `void print_hkust(void) { cout << "hkust" << endl; }`.

6.2 Modular Programming

6.2.1 Why Modular Programming?

- Real application software often contains hundreds of thousands or even millions of lines of code.
- A common approach to complex problems is divide-and-conquer: divide the problem into smaller parts and solve each part in turn.
- In programming we divide a large program into modules, each implemented by a function. This is called modular programming or top-down programming.

Task: draw a tree consisting of a triangular top (the leaves), a rectangular trunk, and a trapezium pot. Sample output:

```
    *
   ***
  *****
 *******
*********
   &&&
   &&&
   &&&
^^^^^^^^^^^^^
 ^^^^^^^^^^^
  ^^^^^^^^^
```

Requirements:

- The user defines the drawing symbols of the 3 parts and the height of the tree (an odd integer).
- For simplicity, all other dimensions (width/height of leaves, trunk, pot) are computed from the tree height using pre-determined formulas; the exact formulas are not important.

Design (top-down): `main()` reads the symbols and the height and calls `print_tree()`; `print_tree()` computes the dimensions and calls `print_leaves()`, `print_trunk()` and `print_pot()`, which all draw row by row through `print_one_row()`; a small `max()` enforces minimum sizes. The leaves have $2n + 1$ symbols on the n-th row; the trunk is a rectangle with width = height = 2/3 of the tree's height; the pot's height is 1/3 and width is 2/3 of the tree's.

File: draw-tree.cpp — the complete modular tree-drawing program.

```
#include <iostream> /* File: draw-tree.cpp */
using namespace std;

int max(int a, int b) { return (a > b) ? a : b; }

void print_one_row(int num_leading_spaces, int num_symbols, char \
   symbol)
{
    for (int j = 0; j < num_leading_spaces; ++j)
        cout << ' ';

    for (int j = 0; j < num_symbols; ++j)
        cout << symbol;

    cout << endl;
}

void print_leaves(int tree_height, char leaf_symbol)
```

```
{
    for (int row = 0, num_leading_spaces = tree_height;
         row < tree_height;
         ++row, --num_leading_spaces)
        print_one_row(num_leading_spaces, 2*row+1, leaf_symbol);
}

void print_trunk(int tree_width, int trunk_height, int trunk_width, \
   char trunk_symbol)
{
    int num_leading_spaces = (tree_width - trunk_width)/2;

    for (int row = 0; row < trunk_height; ++row)
        print_one_row(num_leading_spaces, trunk_width, trunk_symbol);
}

void print_pot(int tree_width, int pot_height, int pot_base_width, \
   char pot_symbol)
{
    int num_leading_spaces = (tree_width - pot_base_width)/2;
    int row = pot_height, width = pot_base_width;
    while (row > 0)
    {
        print_one_row(num_leading_spaces, width, pot_symbol);
        --row;
        ++num_leading_spaces;
        width -= 2;
    }
}

/* The leaves have 2n+1 symbols on the n-th row.
 * The trunk is just a rectangle: width = height = 2/3 of tree's \
   height.
 * pot's height = 1/3 of tree's. pot's width = 2/3 of tree's.
 *
 ************ The exact formulas are not important! ************
 */
void print_tree(int tree_height, char leaf_symbol, char \
   trunk_symbol, char pot_symbol)
{
    int tree_width = 2*tree_height + 1;
    int trunk_width = max(1, tree_width/6);
    int trunk_height = max(1, tree_height/3);
    int pot_width = tree_width * 2/3;
    int pot_base_width = (pot_width % 2) ? pot_width : pot_width + 1;
    int pot_height = max(2, tree_height/3);

    print_leaves(tree_height, leaf_symbol);
    print_trunk(tree_width, trunk_height, trunk_width, trunk_symbol);
    print_pot(tree_width, pot_height, pot_base_width, pot_symbol);
}

int main()
{
    char leaf_symbol, trunk_symbol, pot_symbol;
    int tree_height;

    cout << "Enter the character symbols for tree, trunk, and pot: ";
    cin >> leaf_symbol >> trunk_symbol >> pot_symbol;
    cout << "Enter height of the tree (an odd integer, please): ";
    cin >> tree_height;

    cout << endl << endl << endl;
```

```
    print_tree(tree_height, leaf_symbol, trunk_symbol, pot_symbol);
    return 0;
}
```

6.2.2 Why Function?

- When you have several segments of code doing similar things, they are good candidates for a function.
- A function allows write-once-call-many: write it once, call it many times with the same or different arguments.
- Functions make programs easier to understand and easier to modify.
- Functions allow reusable code (e.g. `log`, `sqrt`, `sin`).
- Functions separate the concept (what is done) from the implementation (how it is done).
- The last two points lead to binary libraries: sets of compiled functions that can be shared without revealing their implementation.

Why Functions

Duplicated code is the signal: wrap the common pattern once in a function and call it with different arguments. You gain readability, easier modification, and reusability.

The next two programs demonstrate the idea: `factorial-sum.cpp` repeats almost identical loops three times to compute $x! + y! + z!$; `factorial-sum-fcn.cpp` extracts the pattern into a `factorial()` function and calls it three times.

File: factorial-sum.cpp — computing $x!+y!+z!$ with duplicated loops (no function).

```
#include <iostream>      /* File: factorial-sum.cpp */
using namespace std;

int main()               /* To compute x! + y! + z! */
{
    int x, y, z;
    int fx = 1, fy = 1, fz = 1;

    cout << "Enter x, y, z: "; cin >> x >> y >> z;

    for (int j = 2; j <= x; ++j) { fx *= j; } // Compute x!

    for (int j = 2; j <= y; ++j) { fy *= j; } // Compute y!

    for (int j = 2; j <= z; ++j) { fz *= j; } // Compute z!

    cout << x << "! + " << y << "! + " << z << "! = "
         << fx + fy + fz << endl;

    return 0;
}
```

File: factorial-sum-fcn.cpp — the same computation with a `factorial` function (write-once-call-many).

```
#include <iostream>      /* File: factorial-sum-fcn.cpp */
using namespace std;

int factorial(int x)     /* To compute x! where x >= 0 */
{
    int factorial = 1, j = 2;
    while (j <= x)
    {
        factorial *= j;
        j++;
    }
```

```
    return factorial;
}

int main()              /* To compute x! + y! + z! */
{
    int x, y, z;
    cout << "Enter x, y, z: "; cin >> x >> y >> z;
    cout << x << "! + " << y << "! + " << z << "! = "
         << factorial(x) + factorial(y) + factorial(z) << endl;
    return 0;
}
```

6.3 Parameter Passing Methods

6.3.1 Pass-by-Value

- C++ supports 2 ways to pass arguments to a function:
  1. pass-by-value (PBV), or call-by-value (CBV);
  2. pass-by-reference (PBR), or call-by-reference (CBR).
- If you call a function with an expression, the expression is first evaluated and the result is passed to the function, e.g. `max(3 + 5, 2 + 9)` becomes `max(8, 11)` before `max` is called.
- In pass-by-value, the value of an actual parameter is copied into the corresponding formal parameter.
- If the actual parameter is a literal constant (e.g. `max(2, 3)`), obviously it will not change.
- If the actual parameter is a variable (e.g. `max(x, y)`), only its value is copied; the variable itself has nothing to do with the operation of the function — in particular, its value cannot be modified by the function.
- All function examples presented so far use pass-by-value.
- Memory picture: the formal parameters `a` and `b` live in the callee's memory as independent copies of `x` and `y` from the caller's memory.

Pass-by-Value swap Does Not Swap

The `swap` below compiles and runs, but `x` and `y` keep their original values: the function only exchanges the copies `a` and `b`, which are destroyed when the function returns.

File: pbv-swap.cpp — a pass-by-value `swap` that fails to swap the caller's variables.

```
/* File: pbv-swap.cpp */
#include <iostream>
using namespace std;

void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);
    cout << "(x , y) = " << '(' << x
         << " , " << y << ')' << endl;
    return 0;
}
```

Passing a Big Object

What happens if the argument is a big object (e.g. of several MB)? Pass-by-value copies the whole object, which is expensive — this motivates pass-by-reference below.

6.3.2 Reference Variables

Syntax: `<type>& <variable1> = <variable2>;`

- A reference variable is an alias of another variable.
- A reference variable must always be bound to an object; therefore it must be initialized when it is defined.
- Once defined and bound, you cannot re-bind it to another object.
- Aliases share the same memory space, so you may modify the value in that memory through any of them.

Rules of Reference Variables

(1) A reference is an alias, not a new object. (2) It must be initialized at definition. (3) It can never be re-bound to another object afterwards.

Assignment Is Not Re-Binding

In `z = a;` the reference `z` is not re-bound to `a`; the value of `a` is assigned to the object `z` is bound to. A reference stays bound to its original object for its whole lifetime.

File: ref-declaration.cpp — reference variables sharing memory with the variables they are bound to.

```
#include <iostream>     /* File: ref-declaration.cpp */
using namespace std;

int main()
{
    int a = 1, b = 2;
    int& x = a;          // now x = a = 1
    int &y = b;          // now y = b = 2
    int &w = a, &z = y; // now w = a = x = 1, z = b = y = 2

    a++;        cout << a << '\t' << x << '\t' << w << endl;
    x += 5;     cout << a << '\t' << x << '\t' << w << endl;
    a = w - x; cout << a << '\t' << x << '\t' << w << endl;

    y *= 10; cout << b << '\t' << y << '\t' << z << endl;
    b--;     cout << b << '\t' << y << '\t' << z << endl;
    z = 999; cout << b << '\t' << y << '\t' << z << endl;

    z = a;               // that is not re-binding z to a
    cout << b << '\t' << y << '\t' << z << endl;
    return 0;
}
```

Here `a`, `x`, `w` all refer to the same integer object, and `b`, `y`, `z` refer to another one; modifying through any alias changes the shared value.

6.3.3 Pass-by-Reference

- Pass-by-reference does not copy the actual parameters; the formal parameter becomes a reference variable (alias) bound to the actual parameter.
- Add the symbol `&` after the type name of a formal parameter to request pass-by-reference.

- On entering `swap(x, y)`, the effect is `int& a = x; int& b = y;` — `a` and `b` are initialized (bound) to `x` and `y` respectively.
- Since formal and actual parameters share the same memory, any modification of the formal parameter also changes the actual parameter.

What Pass-by-Reference Does

PBV: callee gets an independent copy. PBR: callee gets an alias of the caller's object, so writes through the parameter are visible to the caller (and no copying cost).

File: pbr-swap.cpp — a working `swap` using reference parameters.

```
/* File: pbr-swap.cpp */
#include <iostream>
using namespace std;

void swap(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);
    cout << "(x , y) = " << '(' << x
         << " , " << y << ')' << endl;
    return 0;
}
```

Executing `swap(x, y)` is equivalent to running `int temp = x; x = y; y = temp;` directly on the caller's variables.

With a working `swap`, sorting 3 numbers needs only 3 conditional swaps.

File: sort3.cpp — sort 3 numbers in ascending order using `swap`.

```
#include <iostream>     /* File: sort3.cpp */
using namespace std;

void swap(int& x, int& y) /* To swap 2 numbers */
{
    int temp = x;
    x = y;
    y = temp;
}

int main()              /* To sort 3 numbers in ascending order */
{
    int x, y, z;
    cout << "Enter 3 numbers, x, y, z: ";
    cin >> x >> y >> z;

    if (x > y) swap(x, y);
    if (x > z) swap(x, z);
    if (y > z) swap(y, z);

    cout << "x , y , z = " << x << " , " << y << " , " << z << endl;
    return 0;
}
```

6.3.4 Remarks on Function Calls

- A function call has higher precedence than other operators. For example, the two statements `if (factorial(x) < 200) return -1;` and `if (max(a, b) > 10) return -1;` may be shortened to `if (factorial(x) < 200 || max(a, b) > 10) return -1;` — the calls are executed first and their returned values are compared and combined by `<` and `||`.
- Before C++11 you cannot define a function inside another function: all C++ functions (except private class member functions) are global — any function can call any other function if they are properly declared. After C++11, local functions can be defined inside another function by the lambda expression.
- For a function with more than one formal parameter, some parameters may be passed by value while others by reference; there is no restriction on their number and order.

File: call-by-val-ref1.cpp — contrasting the two passing methods: `f_ref` modifies the caller's variable while `f_val` does not.

```
#include <iostream> /* File: call-by-val-ref1.cpp */
using namespace std;

int f_ref(int& x) { return ++x; }
int f_val(int x) { return ++x; }

int main()
{
    int a = 5;

    cout << f_ref(a) << endl;
    cout << a << endl;

    cout << f_val(a) << endl;
    cout << a << endl;

    return 0;
}
```

Reference parameters also answer the earlier question of "returning" multiple objects: pass extra objects by reference and let the function write the results into them.

File: sum-and-difference.cpp — two inputs by value and two outputs by reference.

```
#include <iostream>     /* File: sum-and-difference.cpp */
using namespace std;

// To find the sum and difference of 2 given numbers
void sum_and_difference(int x, int y, int& sum, int& difference)
{
    sum = x + y;
    difference = x - y;
}

int main()
{
    int x, y, sum, difference;
    cout << "Enter 2 numbers: ";
    cin >> x >> y;

    sum_and_difference(x, y, sum, difference);
    cout << "The sum of " << x << " and " << y << " is " << sum << \
   endl;
    cout << "The difference between " << x << " and " << y << " is "
         << difference << endl;
    return 0;
}
```

6.3.5 Lifetime of Local Variables

- All local variables defined inside a function, including the formal parameters, are destroyed when the function returns.
- They are created afresh on every call; those created on the current call are different from those in previous calls.
- If a formal parameter is a reference variable, only the reference itself is destroyed on return; the actual parameter bound to it still exists afterwards.
- Pass-by-reference is more efficient for large objects (no copying), but there is a risk of accidentally modifying the object.

Lifetime of Local Variables

Locals (and value parameters) are born at each call and die at return. A reference parameter only dies itself — the caller's object it was bound to lives on, which is exactly why PBR can "bring results back".

6.3.6 `const` References as Function Arguments

- You can (and should!) express your intention to keep a reference argument unchanged by making it `const`.
- Advantage 1: if you accidentally try to modify the argument inside the function, the compiler catches the error.
- Advantage 2: you may pass both `const` and non-`const` arguments (including literal constants) to a function that takes a `const` reference parameter; conversely, only non-`const` arguments may be passed to a non-`const` reference parameter:

Passing object ...	to `const T&` parameter	to `T&` parameter
literal constant	OK	error
`const` object	OK	error
non-`const` object	OK	OK

Use const Reference for Large Objects

Passing a large object by `const` reference gives the efficiency of pass-by-reference (no copying) together with the safety of pass-by-value (the function cannot modify it): the compiler enforces your intention.

File: cbcr.cpp — modifying through a plain reference is fine; through a `const` reference it is a compilation error.

```
// Fine
void call_by_ref(int& x) { x += 10; }

// Error!
void call_by_const_ref(const int& x) { x += 10; }
```

What a Non-const Reference Parameter Accepts

A `T&` parameter cannot bind to a literal constant or a `const` object — the compiler rejects it. If a function only reads its reference argument, declare the parameter `const T&` so that all kinds of arguments are accepted.

File: const-ref-arg.cpp — which calls give compilation errors? (`call_by_ref(1234)` and `call_by_ref(y)`.)

```
#include <iostream>      /* File: const-ref-arg.cpp */
using namespace std;

void call_by_ref(int& a) { cout << a << endl; }
void call_by_const_ref(const int& a) { cout << a << endl; }
```

```
int main()
{
    int x = 50;
    const int y = 100;
    // Which of the following give(s) compilation error?
    /* passing const literals */
    call_by_ref(1234);
    call_by_const_ref(1234);

    /* passing const objects */
    call_by_ref(y);
    call_by_const_ref(y);

    /* passing non-const objects */
    call_by_ref(x);
    call_by_const_ref(x);
    return 0;
}
```

6.3.7 Return-by-Value and Return-by-Reference

- Just as you may pass parameters by value or by reference, you may also return objects by value (RBV) or by reference (RBR).
- All function examples seen so far return by value, e.g. `int max(int a, int b) { return (a > b) ? a : b; }` and the `factorial` function.
- During RBV the returned object is copied to the caller; if the returned object is big, the return is slow due to copying.
- RBR may solve the problem — more about this later.

6.4 Further Examples

- Constants `ROCK = 0`, `PAPER = 1`, `SCISSORS = 2` represent the three choices.
- `srand(seed)` initializes the random number generator (declared in `<cstdlib>`); `rand() % 3` produces an integer converted to ROCK/PAPER/SCISSORS.
- `print_choice()` returns an `int` status: 0 for success and -1 to signal an error (an invalid player or choice); `main()` checks the returned status and quits with `return -1;` on error.
- `print_game_result()` is a `void` function that prints `DRAW!`, `COMPUTER WINS!`, or `USER WINS!`.

Check the Return Status

Using the returned object as a status signal (0 = OK, -1 = error) lets the caller detect invalid situations and react, e.g. stop the program early with `return -1;` from `main()`.

File: rps-game.cpp — the rock/paper/scissors game (a more concise solution with error messages is in `rps-game2.cpp`).

```
#include <iostream>  /* File: rps-game.cpp */
#include <cstdlib>   // Info about random number generator rand()
using namespace std; // A more concise solution with error message \
   is in rps-game2.cpp

const int ROCK = 0, PAPER = 1, SCISSORS = 2; // 0/1/2 to represent \
   ROCK/PAPER/SCISSORS

/* To print out the choice picked by the computer or the user */
```

```
int print_choice(char player, int choice) // 'C' for computer and \
   'U' for user
{
    if (player == 'C')
        cout << "Computer";
    else if (player == 'U')
        cout << "User";
    else
        return -1; // Better also print an error message

    cout << " picks ";

    if (choice == ROCK)
        cout << "rock" << endl;
    else if (choice == PAPER)
        cout << "paper" << endl;
    else if (choice == SCISSORS)
        cout << "scissors" << endl;
    else
        return -1; // Better also print an error message

    return 0;
}


/* To print game result: "DRAW!", "COMPUTER WINS!", or "PLAYER \
   WINS!" */
void print_game_result(int computer_choice, int user_choice)
{
    if (computer_choice == user_choice)
        cout << "\tDRAW!" << endl;

    else if (computer_choice == ROCK && user_choice == SCISSORS
        || computer_choice == SCISSORS && user_choice == PAPER
        || computer_choice == PAPER && user_choice == ROCK)
        cout << "\tCOMPUTER WINS!" << endl;

    else
        cout << "\tUSER WINS!" << endl;
}


int main()
{
    int seed;   // To seed the random number generator
    cout << "Enter an integer: "; cin >> seed;
    srand(seed);  // Initialize random number generator

    int computer_choice = rand()%3; // rand() produces an integer \
   which is
    int user_choice = rand()%3; // then converted to \
   ROCK/PAPER/SCISSORS

    if (print_choice('C', computer_choice) != 0) return -1; // -1 \
   signals an error
    if (print_choice('U', user_choice) != 0) return -1; // -1 \
   signals an error
    print_game_result(computer_choice, user_choice);
    return 0;
}
```

File: rps-game2.cpp — the alternative solution mentioned on the slides: it prints error messages via `cerr` and combines the two status checks with `||`.

```
#include <iostream>  /* File: game.cpp */
#include <cstdlib>   // Info about random number generator rand()
using namespace std;

const int ROCK = 0, PAPER = 1, SCISSORS = 2; // 0/1/2 to represent \
   ROCK/PAPER/SCISSORS

/* To print out the choice picked by the computer or the user */
int print_choice(char player, int choice) // 'C' for computer and \
   'U' for user
{
    if (player == 'C')
        cout << "Computer";
    else if (player == 'U')
        cout << "User";
    else
    {
        cerr << "Error: Invalid user --- " << player << endl; return \
   -1;
    }
    cout << " picks ";

    if (choice == ROCK)
        cout << "rock" << endl;
    else if (choice == PAPER)
        cout << "paper" << endl;
    else if (choice == SCISSORS)
        cout << "scissors" << endl;
    else
    {
        cerr << "Error: Invalid game choice --- " << choice << endl; \
   return -1;
    }

    return 0;
}


/* To print game result: "DRAW!", "COMPUTER WINS!", or "PLAYER \
   WINS!" */
void print_game_result(int computer_choice, int user_choice)
{
    if(computer_choice == user_choice)
        cout << "\tDRAW!" << endl;
    else if(computer_choice == ROCK && user_choice == SCISSORS
        || computer_choice == SCISSORS && user_choice == PAPER
        || computer_choice == PAPER && user_choice == ROCK)
        cout << "\tCOMPUTER WINS!" << endl;
    else
        cout << "\tUSER WINS!" << endl;
}


int main()
{
    int seed;   // To seed the random number generator
    cout << "Enter an integer: "; cin >> seed;
    srand(seed);  // initialize random number generator

    int computer_choice = rand()%3; // rand() produces an integer \
   which is
    int user_choice = rand()%3; // then converted to \
```

```
   ROCK/PAPER/SCISSORS

    if (print_choice('C', computer_choice) != 0 || print_choice('U', \
   user_choice) != 0)
        return -1; // -1 signals an error

    print_game_result(computer_choice, user_choice);
    return 0;
}
```

7 Array

An **array** is a collection of homogeneous objects: objects of the same type, e.g., a collection of `int`, `char`, `double`, ..., or user-defined types. The only exception is that the array elements cannot be reference variables.

Motivation

- A function to sort 3 integers can be written as `void sort_3_int(int& x, int& y, int& z);`, and one for 6 integers as `void sort_6_int(int& u, ... , int& z);`. But how about sorting 10,000 integers? Are you going to create 10,000 different variable names?
- Array is designed to solve this problem: you only need one identifier name to address all the 10,000 integers, and there is a way to refer to each of them.
- It can solve problems like: read a list of student names and sort them in alphabetical order.
- In an Excel file, each column/row is basically an array so that you can do common operations (like average, max, min, count) on it.

7.1 1-Dimensional Array

7.1.1 Definition of a 1D Array

- Syntax: `<data-type> <array-name>[<size>];`
- `<size>` should be a positive constant. It can be a constant expression too.

```
int number[10000];       // An array of 10,000 uninitialized integers
const int NUM_STUDENTS = 335;
char gender[NUM_STUDENTS];       // An array of 335 char
float score[NUM_STUDENTS + 1];  // An extra element to hold the mean
int n = 3;
double x[n];    // Compilation error on VC++: size is NOT a constant
int value[-4];  // Compilation error: array size cannot be -ve
```

Array Size Must Be a Positive Constant

The size in an array definition must be a positive constant (or constant expression). Using a non-`const` variable like `n` above, or a negative number, is a compilation error.

7.1.2 Subscripting: Access to Each Array Element

- A 1D array is an ordered list of elements; successive elements are stored in contiguous memory. If `x` is an `int` array with `sizeof(int) = 4` starting at address 1000, then `x[0]`, `x[1]`, `x[2]`, ..., `x[N-1]` sit at addresses $1000, 1004, 1008, \ldots, 1000 + 4(N-1)$.
- To access an element, use the subscript operator `[ ]` with an array index.
- For an array of size N, the indices run from $0, 1, 2, \ldots, N-1$.

- Each array element is treated like a regular variable: you may assign a value to it, assign its value to another variable, and pass it by value or by reference to a function.

Array Indices Start from Zero

In C++ the first element is `x[0]` and the last element of an array of size N is `x[N-1]`. Elements occupy contiguous memory, so the subscript operator is just an offset from the array's starting address.

7.1.3 Array and Control

- Array works particularly well with loops: e.g., use a `for`-loop to access and manipulate each array element in turn. This is not a coincidence, but part of the C++ language design.

```
int y;           // A regular int variable
int x[3];        // An array of 3 int numbers
x[0] = 34;       // Array indices start from zero in C++
x[1] = 289;
x[2] = 75;       // Index of the last element is 2 NOT 3!
y = x[2];        // Now both y and x[2] are 75
max(x[2], x[0]);    // Pass array elements by value
swap(x[1], x[0]);   // Pass array elements by reference
for (int j = 0; j < 3; j++)
    x[j] *= 3;      // Triple each element of an array
```

File: array-mean.cpp — reads 5 scores with a `for`-loop and accumulates them to compute the mean.

```
#include <iostream>     /* array-mean.cpp */
using namespace std;

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];
    float sum_score = 0.0;     // Don't forget initializing the sum

    for (int j = 0; j < NUM_STUDENTS; ++j)
    {
        cin >> score[j];
        sum_score += score[j]; // Accumulate the scores
    }

    cout << "mean score = " << sum_score/NUM_STUDENTS << endl;
    return 0;
}
```

File: array-max.cpp — finds the maximum score; initializing the max with the first element is a good practice (assuming at least 1 student).

```
#include <iostream>     /* array-max.cpp */
using namespace std;

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];
    // Read in the first student's score. Assume #students >= 1
    cin >> score[0];
    float max_score = score[0]; // A good way to initialize max score

    for (int j = 1; j < NUM_STUDENTS; ++j)
```

```
    {
        cin >> score[j];
        if (max_score < score[j])
            max_score = score[j];
    }

    cout << "max score = "  << max_score << endl;
    return 0;
}
```

7.1.4 Wrong Subscript: Common Reason for Segmentation Fault

- The C++ compiler does not automatically check that an array index is out of bound: for an array of size N, it won't check if it is subscripted with an index between 0 and $N-1$, neither at compile-time nor at run-time.
- There is no compilation error for the following code: `int x[10]; x[-2] = 5; x[100] = 9;`
- When the code runs, `x[-2] = 5;` puts the value 5 into the memory space 2×4 bytes (size of 2 `int`) before the array `x`. Similarly, `x[100] = 9;` puts 9 into the memory 90×4 bytes beyond the array.
- This is a common cause of the run-time error called **segmentation fault** — your program trespasses into memory locations that may not belong to it.

Out-of-Bound Subscripts Cause Segmentation Faults

C++ performs no bounds checking at compile-time or run-time. Indexing before `x[0]` or beyond `x[N-1]` silently reads or writes memory that does not belong to the array — a common source of segmentation faults.

7.1.5 Array Initialization and Common Mis-uses

- Just like any local variable, when an array is defined, its elements are not initialized automatically.
- Syntax for defining and initializing a 1D array simultaneously:
  `<data-type> <array-name>[<size>] = {<value0>, <value1>, ...};`
- If there are fewer values than the array size, the unspecified values will be zeros.
- It is a compilation error if there are more values than the array size.
- If you leave out the array size, the compiler counts the number of initializing values and uses that as the array size.
- Once defined, you cannot assign values to an array using the initialization syntax.

```
int a[5] = {1, 2, 3, 4, 5};
/* Same as
int a[5];
a[0] = 1; a[1] = 2; a[2] = 3; a[3] = 4; a[4] = 5;
*/
int b[5] = {1, 2};       // => 1, 2, 0, 0, 0
int c[5] = {};           // => 0, 0, 0, 0, 0
int d[] = {1, 2, 3};     // Compiler determines the size=3 \
   automatically
int e[3];
// Compilation error:
// can't assign values to an array using the { } syntax
e = {5, 6, 7};
// Compilation error: can't declare an array of references
double x = 1.5, y = 2.5, z = 3.5;
double& s[] = {x, y, z};
```

Brace Initialization Only Works at Definition

The { } initialization syntax can only be used in the definition statement. Writing `e = {5, 6, 7};` after `e` is defined is a compilation error. An array of references (e.g., `double& s[]`) is also illegal.

- While each array element can be treated as a simple variable, the whole array, as represented by the array identifier, cannot.

```
int x[] = {1, 2, 3, 4, 5 };
int y[] = {6, 7, 8, 9, 0 };
int z[5];
/* Incorrect way */
// Cannot assign to array elements using the initialization syntax
x = {5, 4, 3, 2, 1};
x = 8;      // x is not an integer! Its elements are.
x += 2;     // x is not an integer! Its elements are.
x = y;      // No assignment between 2 arrays
z = x + y;  // Cannot +, -, *, / on the array, but only its elements
/* Correct way; what does each for-statement do? */
for (int j = 0; j < 5; ++j) x[j] = 5 - j;
for (int j = 0; j < 5; ++j) x[j] = 8;
for (int j = 0; j < 5; ++j) x[j] += 2;
for (int j = 0; j < 5; ++j) x[j] = y[j];
for (int j = 0; j < 5; ++j) z[j] = x[j] + y[j];
```

7.1.6 Pass a 1D Array to a Function

- Since the array identifier alone does not tell us about its size, a function that operates on an array needs at least 2 input arguments: the array identifier, and the array size (of type `int`).

```
/* function header */
float mean_score(float score[], int size) { ... }
float max_score(float score[], int size) { ... }
/* inside the main() */
float score[NUM_STUDENTS];
mean_score(score, NUM_STUDENTS);
max_score(score, NUM_STUDENTS);
```

File: array-mean-max-fcn.cpp — the mean/max computations rewritten as functions taking an array plus its size; `main()` also shows how to check whether an input operation succeeds.

```
#include <iostream>     /* array-mean-max-fcn.cpp */
using namespace std;

float mean_score(float score[], int size)
{
    float sum_score = 0.0; // Don't forget initializing the sum to 0
    for (int j = 0; j < size; j++)
        sum_score += score[j]; // Accumulate the scores
    return sum_score/size;
}

float max_score(float score[], int size)
{
    // Initialize the max score to that of the first student
    float max_score = score[0];
    for (int j = 1; j < size; j++)
        if (max_score < score[j])
```

```
            max_score = score[j];
    return max_score;
}

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];

    for (int j = 0; j < NUM_STUDENTS; j++)
        if (!(cin >> score[j]))
            return -1;

    cout << "mean score = " << mean_score(score, NUM_STUDENTS) << \
   endl;
    cout << "max score = "  << max_score(score, NUM_STUDENTS) << endl;
    return 0;
}
```

- Notice how we may check if the input operation is successful: `if (!(cin >> score[j]))` — it returns true/false if it succeeds/fails, respectively.
- While a regular variable may be passed to a function by value or by reference, an array variable is always passed by value.
- However, although the array variable is passed by value, its elements are effectively passed by reference: any change to an array element inside the function will persist even after the function returns.
- Just like a regular variable, you pass an array to a function simply by its variable name, e.g., `max_score(score, NUM_STUDENTS);`

Array Elements Are Effectively Passed by Reference

The array variable itself is passed by value, but the function works on the caller's actual elements. Modifications to array elements inside a function are visible after the function returns.

File: array-add-rotate.cpp — functions that print an array, add two arrays element-wise into a third, and circularly rotate an array up by one position; all modifications persist in the caller.

```
#include <iostream>     /* array-add-rotate.cpp */
using namespace std;

void array_print(int x[], int size)
{
    for (int j = 0; j < size; j++)
        cout << x[j] << '\t';
    cout << endl;
}

void array_add(int x[], int y[], int z[], int size)
{
    for (int j = 0; j < size; j++)
        z[j] = x[j] + y[j];
}

void circular_rotation(int x[], int size)
{
    int item_0 = x[0];    // Save the first element before rotation
    for (int j = 1; j < size; j++)
        x[j-1] = x[j];    // Rotate up
    x[size - 1] = item_0; // Fix the last element
}

int main()
```

```
{
    int a[] = {1, 2, 3, 4};
    int b[] = {11, 12, 13, 14};
    int c[4];

    array_add(a, b, c, 4);
    array_print(c, 4);
    cout << endl;

    for (int k = 0; k < 4; k++)
    {
        circular_rotation(a, 4);
        array_print(a, 4);
    }

    return 0;
}
```

File: random-element.cpp — contrasts return-by-value with return-by-reference: `element_rbr` returns a reference to a random array element, so it can appear on the left-hand side of an assignment.

```
#include <iostream>     /* random-element.cpp */
#include <cstdlib>
using namespace std;    /* To return a random array element */

int  element_rbv(int x[], int size) { return x[rand() % size]; }
int& element_rbr(int x[], int size) { return x[rand() % size]; }

void print_array(int x[], int size)
{
    for (int j = 0; j < size; j++) cout << x[j] << ' ';
    cout << endl;
}

int main()
{
    srand(time(0));     // Seed the random num generator
    int data[6] = {10, 20, 30, 40, 50, 60};

    cout << element_rbv(data, 6) << endl; // Same as: cout << data[?]
    cout << element_rbr(data, 6) << endl; // Same as: cout << data[?]
    print_array(data, 6);
    element_rbr(data, 6) += 8;            // Same as: data[?] += 8
    print_array(data, 6);
}
```

7.1.7 Constant Array

- Just like simple constants, an array of constants can be made using the keyword `const`: `const int x[] = { 1, 2, 3, 4};` defines 4 integer constants — `x[0]`, `x[1]`, `x[2]`, and `x[3]` are all of the type `const int`.
- Like simple constants, a constant array must be initialized when it is defined, and once defined, its elements cannot be modified.
- One main use of constant array is in the definition of the formal parameters of a function: to disallow modification of the elements of an array passed to a function, declare that array constant using `const`. Inside the function, the array is read-only; however, the original array in the caller is still writable.

Use const Array Parameters for Read-Only Access

If a function only needs to read an array (e.g., computing a mean or max), declare the

parameter `const float score[]`. The compiler then guarantees the function cannot modify the caller's data.

File: const-array-mean-max-fcn.cpp — the same mean/max functions, but with `const` array parameters so the scores cannot be modified inside the functions.

```
#include <iostream>     /* const-array-mean-max-fcn.cpp */
using namespace std;

float mean_score(const float score[], int size)
{
    float sum_score = 0.0;  // Don't forget initializing the sum to 0
    for (int j = 0; j < size; j++)
        sum_score += score[j]; // Accumulate the scores
    return sum_score/size;
}

float max_score(const float score[], int size)
{
    // Initialize the max score to that of the first student
    float max_score = score[0];
    for (int j = 1; j < size; j++)
        if (max_score < score[j])
            max_score = score[j];
    return max_score;
}

int main()
{
    const int NUM_STUDENTS = 5;
    float score[NUM_STUDENTS];

    for (int j = 0; j < NUM_STUDENTS; j++)
        if (!(cin >> score[j]))
            return -1;

    cout << "mean score = " << mean_score(score, NUM_STUDENTS) << \
   endl;
    cout << "max score = "  << max_score(score, NUM_STUDENTS) << endl;
    return 0;
}
```

7.2 Multi-dimensional Array

In general, an array can be multi-dimensional.

7.2.1 C++ 2-Dimensional Array

- Syntax: `<data-type> <array-name>[<size1>][<size2>];`
- Example: `int a[2][3] = {1,2,3,4,5,6};` (with `sizeof(int) = 4`, starting at address 1000). Conceptually this is a table of 2 rows and 3 columns: `a[0][0]`...`a[0][2]` form row 0 and `a[1][0]`...`a[1][2]` form row 1.
- Physically the 6 cells are stored linearly at consecutive addresses $1000, 1004, \ldots, 1020$, in the order `a[0][0]`, `a[0][1]`, `a[0][2]`, `a[1][0]`, `a[1][1]`, `a[1][2]`.

A 2D array can be initialized in 2 ways: row by row, or like a 1D array since the array cells are actually stored linearly in the memory.

```
/* Initialize row by row */
int point[5][2] = {     // An int array with 5 rows and 2 columns
    {1, 1},
```

```
    {2, 4},
    {3, 9},
    {4, 16},
    {5, 25}
};
/*
 * Initialize using the fact that the cells of a 2D
 * array actually are stored linearly in the memory
 */
int point[5][2] = { 1,1, 2,4, 3,9, 4,16, 5,25 };
```

File: distance-fcn.cpp — the previous (pre-array) example: a function that computes the Euclidean distance between two points, used to find the lengths of the sides of a triangle from 6 separate coordinate variables.

```
#include <iostream>     /* File: distance-fcn.cpp */
#include <cmath>        // Math library info
using namespace std;

/* To find the Euclidean distance between 2 points */
double  euclidean_distance(double x1, double y1, double x2, double y2)
{
    double x_diff = x1 - x2;
    double y_diff = y1 - y2;
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

int main()   /* To find the length of the sides of a triangle */
{
    double xA, yA, xB, yB, xC, yC;
    cout << "Enter the co-ordinates of point A: "; cin >> xA >> yA;
    cout << "Enter the co-ordinates of point B: "; cin >> xB >> yB;
    cout << "Enter the co-ordinates of point C: "; cin >> xC >> yC;

    cout << " AB = " << euclidean_distance(xA, yA, xB, yB) << endl;
    cout << " BC = " << euclidean_distance(xB, yB, xC, yC) << endl;
    cout << " CA = " << euclidean_distance(xC, yC, xA, yA) << endl;
    return 0;
}
```

File: 2d-array-fcn.cpp — stores the points in a 2D array and computes the distances between all pairs of points into another 2D array; note that the second dimension must be given in the function parameters.

```
#include <iostream>     /* File: 2d-array-fcn.cpp */
#include <cmath>
using namespace std;

float  euclidean_distance(float x1, float y1, float x2, float y2)
{
    float x_diff = x1 - x2, y_diff = y1 - y2;
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

void  compute_all_distances(
    const float point[][2], float dist[][3], int num_points)
{
    for (int i = 0; i < num_points; i++)
        for (int j = 0; j < num_points; j++)
            dist[i][j]
                = euclidean_distance(point[i][0], point[i][1],
                                     point[j][0], point[j][1]);
}
```

```
void print_2d_array(const float a[][3], int num_rows, int num_columns)
{
    for (int i = 0; i < num_rows; i++)
    {
        for (int j = 0; j < num_columns; j++)
            cout << a[i][j] << '\t';
        cout << endl;
    }
}

int main()
{
    float dist[3][3];   // Distances between any pairs of points
    float point[3][2]   // (x, y) coordinates of 3 points
        = { {1.0, 1.0} , {2.0, 2.0} , {4.0, 3.0} };

    compute_all_distances(point, dist, 3);
    print_2d_array(dist, 3, 3);
    return 0;
}
```

File: 2d-array-fcn2.cpp — a better design: each row of a 2D array `a[N][M]` can itself be treated as a 1D array of M elements (addressed as `a[i]`), so the distance function can simply take two 1D arrays.

```
/* File: 2d-array-fcn2.cpp
 * Each row of a 2D array, e.g., a[N][M], can be treated as a 1D \
   array.
 * The i-th row can be addressed as a[N],
 * which represents a 1D array of M elements.
 */

float euclidean_distance(const float point_A[], const float point_B[])
{
    float x_diff = point_A[0] - point_B[0];
    float y_diff = point_A[1] - point_B[1];
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

void compute_all_distances(
    const float point[][2], float dist[][3], int num_points)
{
    for (int i = 0; i < num_points; i++)
        for (int j = 0; j < num_points; j++)
            dist[i][j] = euclidean_distance(point[i], point[j]);
}
```

7.2.2 C++ N-Dimensional Array

- Syntax: `<data-type> <array-name>[<size1>][<size2>]...[<sizeN>];`
- Example: `int a[2][2][2] = {1,2,3,4,5,6,7,8};` Conceptually this is a cube (with axes x, y, z), but physically the 8 cells lie at consecutive addresses $1000, 1004, \ldots, 1028$ in the order `a[0][0][0]`, `a[0][0][1]`, `a[0][1][0]`, `a[0][1][1]`, `a[1][0][0]`, `a[1][0][1]`, `a[1][1][0]`, `a[1][1][1]`.
- Although conceptually a 2D array is like a matrix, and a 3D array is like a cube, the elements of a multi-dimensional array are stored linearly in memory (just like a 1D array).
- In C++, the elements of a multi-dimensional array are stored in **row-major order**: row by row. There are programming languages (e.g., FORTRAN) that store multi-dimensional array elements in **column-major order**: column by column.

- In row-major order, the last dimension index runs fastest, while the first dimension index runs slowest.
- If a multi-dimensional array is used in a C++ function, all dimensions other than the first must be specified in its declaration in the function header.

Row-Major Order and Function Parameters

C++ stores multi-dimensional arrays linearly in row-major order: the last index varies fastest. Consequently, when a multi-dimensional array is a function parameter, every dimension except the first must be given a constant size, e.g., `const float point[][2]`.

7.3 C String: Special 1D Character Array

7.3.1 C String

- In general, one cannot deal with the whole array at once, but has to deal with each array element, one at a time, because a sequence of, e.g., integers, does not represent a new object.
- A `char` array is different: a sequence of `char`s may be interpreted as a word or sentence or paragraph or even a novel!
- C++ follows C's trick of representing a character string by a 1D character array with the end-marker `'\0'`: just add the **null character** `'\0'` (ASCII code = 0) after the last character of the string you need. e.g., `'h' 'k' 'u' 's' 't' '\0'`.
- For a string of length N, add `'\0'` as the $(N+1)$-th element of its `char` array.
- Now if everyone writes functions of `char` arrays that represent strings with the above understanding, then one doesn't need to pass the size of such `char` arrays to their functions!
- C++ allows another notation using the double quotes, e.g., `"hkust"` = `'h' 'k' 'u' 's' 't' '\0'`.

The Null Terminator Replaces the Size Argument

Because a C string ends with the null character `'\0'` (ASCII 0), functions can find the end of the string by themselves — no separate size argument is needed, unlike for general arrays.

File: c-string.cpp — turns a plain `char` array into a C string by writing the null character, and shows the double-quote initialization notation.

```
#include <iostream>     /* File: c-string.cpp */
using namespace std;

int main()
{
    char s1[6] = {'h', 'k','u','s','t', 'z'};

    // At this point, s1 is still a simple char array
    for (int j = 0; j < 5; j++)
        cout << s1[j];
    cout << endl;

    s1[5] = '\0';        // Now, s1 is a C string
    cout << s1 << endl;

    // Another notation for initializing literal constant strings
    char s2[20] = {'h', 'k','u','s','t', '\0'};
    cout << "s2 = " << s2 << endl;
    char s3[20] = "hkust"; cout << "s3 = " << s3 << endl;
    return 0;
}
```

7.3.2 C String Functions

File: c-string-fcn.cpp — implements `str_len` (string length) and `str_concatenate` (concatenation) by scanning for the null character; no array size is passed. (Read Appendix B of the textbook for more C string library functions.)

```
#include <iostream>     /* File: c-string-fcn.cpp */
using namespace std;
const char NULL_CHAR = '\0';

int str_len(const char s[])
{
    int j;
    for (j = 0; s[j] != NULL_CHAR; j++)
        ;
    return j;
}

int str_concatenate(const char s1[], const char s2[], char s[])
{
    int j;
    for (j = 0; s1[j] != NULL_CHAR; j++)
        s[j] = s1[j];   // Copy s1 to s

    for (int k = 0 ; s2[k] != NULL_CHAR; k++, j++)
        s[j] = s2[k];   // Copy s2 after s1

    s[j] = NULL_CHAR;   // Make s a C String
    return j;
}

int main()
{
    char a[20] = "Albert";
    char b[20] = "Einstein";
    char c[20];
    int length;

    cout << "length of string a = " << str_len(a) << endl;
    cout << "length of string b = " << str_len(b) << endl;

    length = str_concatenate(a, b, c);
    cout << "After concatenation: "
         << c << " of length " << length << endl;

    return 0;
} // Read Appendix B of the textbook for more C string library \
   functions.
```

File: str-array.cpp — a 2D `char` array stores 5 C-strings (each of maximum length 15 characters), and each row `s[j]` is printed as one C string.

```
#include <iostream>     /* File: str-array.cpp */
using namespace std;

void print_strings(const char s[][16], int num_of_strings)
{
    for (int j = 0; j < num_of_strings; j++)
        cout << s[j] << " ";
    cout << endl;
}

int main()
{
    // 5 C-strings, each having a max. length of 15 char
```

```
    const char word[5][16] = {
        "hong kong",
        "university",
        "of",
        "science",
        "technology"
    };

    print_strings(word, 5);
    return 0;
}
```

7.3.3 Reading C Strings with cin

- `cin` will skip all white spaces before reading data of the required type until it sees the next white space. White spaces are any sequence of `' '`, `'\t'` and `'\n'`.
- For `char x; cin >> x;`, if the input is `"  hkust  "`, `cin` will skip all the leading white spaces, and gives `'h'` to `x`.
- The same is true for reading a C string. For `char x[20]; cin >> x;`, if the input is `"  hkust  "`, `cin` will skip all the leading white spaces, and gives `"hkust"` to `x`.
- Thus, `cin >>` is not good at reading multiple words or even a paragraph including possibly the newline. Instead, use:
  `cin.getline(char s[], int max-num-char, char terminator);`
- `cin.getline()` will stop when either (`max-num-char` − 1) characters are read, or the terminating character `terminator` is seen. The terminating character is removed from the input stream but is not read into the string.
- The C-string terminating null character is automatically inserted at the end of the read string.

cin ¿¿ Stops at White Space

`cin >>` skips leading white space and stops at the next white space, so it cannot read phrases containing spaces or newlines. Use `cin.getline()` with a maximum character count and a terminating character to read a whole line.

File: read-str.cpp — demonstrates `cin.getline()` with the default newline terminator and with a custom terminator `'W'`; `cin.clear()` resets the failbit when the maximum number of characters has been read.

```
#include <iostream>     /* File: read-str.cpp */
using namespace std;

int main()
{
    const int MAX_LINE_LEN = 255;
    char s[MAX_LINE_LEN+1];

    // Read until the newline character (default)
    cin.getline(s, MAX_LINE_LEN+1, '\n');
    cout << s << endl;

    // Read until the character 'W'
    cin.clear(); // Clear the failbit if max #chars are read
    cin.getline(s, sizeof(s), 'W');
    cout << s << endl;

    return 0;
}
```

File: palindrome.cpp — checks whether each input line is a palindrome using two indices reading the array from both ends; the input loop ends either at a newline or when the maximum number of characters is read (then `cin.gcount()` equals `MAX_LINE_LEN` and

the failbit is cleared with `cin.clear()`).

```
#include <iostream>     /* File: palindrome.cpp */
using namespace std;

bool palindrome(const char x[]) {
    int j = 0;              // An index reading the array from top \
   (left)
    int k = strlen(x) - 1; // An index reading the array from bottom \
   (right)

    for ( ; j < k; ++j, --k)
        if (x[j] != x[k])
            return false;
    return true;
}

int main() {
    const int MAX_LINE_LEN = 255; char whole_line[MAX_LINE_LEN+1];

    while (cin.getline(whole_line, MAX_LINE_LEN+1, '\n')// Input \
   ends in newline
           || cin.gcount() == MAX_LINE_LEN) // Input ends with max \
   #chars
    {
        cout << whole_line << endl
             << boolalpha << palindrome(whole_line) << endl;
        cin.clear();    // Clear the failbit if max #chars are read
    }
    return 0;
}
```

8 Function II: Recursion

8.1 Think Recursively!

8.1.1 Motivating Examples

- **Tower of Hanoi game.**
  - It consists of 3 pegs, and a stack of discs of different sizes.
  - It starts with all discs stacked up on one peg with smaller discs sitting on top of bigger discs.
  - The goal is to move the entire stack of discs to another peg, making use of the remaining peg.
  - Rules: only one disc may be moved at a time; no disc may be placed on top of a smaller disc.
- **Factorial function.** The high-school definition is $n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$. Its recursive definition is:

$$0! = 1, \qquad n! = n \times (n-1)! \text{ if } n > 0.$$

- **Polyline.** A polyline with n line segments is equivalent to adding another line segment to a polyline with $n-1$ line segments.
- **Fibonacci numbers.** $0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$
  - Fibonacci (1202) investigated how fast rabbits could breed: a newly-born pair of rabbits (one male, one female) are put in a field; rabbits mate at the age of one month so that at the end of its 2nd month a female can produce another pair; suppose the rabbits never die, and the female always produces one new pair every month from the 2nd month on. How many pairs will there be in one year?

– What is special with the numbers: except for the first 2 numbers, each number is the sum of the last 2 numbers in the sequence.

8.1.2 Thinking Recursively

- In some problems, it may be natural to define the problem in terms of the problem itself!
- Recursion is useful for problems that can be represented by a simpler version of the same problem:
  1. To solve the Tower of Hanoi game that uses n discs, one first solves the Tower of Hanoi game that uses $n-1$ discs.
  2. To find the value of $n!$, first find the value of $(n-1)!$ and then multiply the result with n.
  3. To draw a polyline of n segments, first draw a polyline of $n-1$ segments, and then add the last line segment.
  4. To find the nth Fibonacci number, first find the $(n-1)$th and $(n-2)$th Fibonacci numbers, and then add them up.

8.2 Programming Recursion in C++

8.2.1 Implement Recursion by Recursive Function

- In programming, recursion means that a function calls itself!
- Although it looks strange in the beginning, solving a programming task by recursion renders the program
  – easier to write,
  – easier to read (understand),
  – shorter (in codes).
- To implement a recursive solution:
  1. Decompose the problem into sub-problems — which are smaller examples of the same problem — plus some additional work that "glues" the solutions of the sub-problems together.
  2. The smallest sub-problem has a non-recursive solution.

Base Case and Recursive Case

Every recursive function needs at least one base case (the smallest sub-problem with a non-recursive solution, which is also the ending case) and a recursive case that reduces the problem toward the base case.

8.2.2 Example: Factorial Function

The recursive factorial follows directly from the recursive definition $0! = 1$, $n! = n \times (n-1)!$.

File: factorial.cpp — recursive factorial with error checking, a base case and a recursive case.

```
int factorial(int n)     /* factorial.cpp */
{
    if (n < 0)           // Error checking
        return -1;
    else if (n == 0)     // Base case; ending case too!
        return 1;
    else                 // Recursive case
        return n * factorial(n-1);
}
```

File: factorial2.cpp — an equivalent version without the `else` branches.

```
int factorial(int n)    /* factorial2.cpp */
{
    if (n < 0)          // Error checking
        return -1;
    if (n == 0)         // Base case; ending case too!
        return 1;
    return n * factorial(n-1); // Recursive case
}
```

File: factorial-main.cpp — driver that reads integers and prints their factorials (reporting an error when the function returns a negative value).

```
#include <iostream> /* File: factorial-main.cpp */
using namespace std;

int factorial(int);

int main()
{
    int x;
    int result;

    while (cin >> x)
    {
        result = factorial(x);

        if (result < 0)
            cout << "Input must be >= 0" << endl;
        else
            cout << "factorial(" << x << ") = " << result << endl;
    }

    return 0;
}
```

How the recursive call `factorial(3)` works:

- `factorial(3)`: $3 < 0$ false, $3 == 0$ false $\Rightarrow$ `3 * factorial(2)`
- `factorial(2)`: $2 < 0$ false, $2 == 0$ false $\Rightarrow$ `2 * factorial(1)`
- `factorial(1)`: $1 < 0$ false, $1 == 0$ false $\Rightarrow$ `1 * factorial(0)`
- `factorial(0)`: $0 < 0$ false, $0 == 0$ true $\Rightarrow$ `return 1`
- The calls then return in reverse order: `return 1*1 = 1`, then `return 2*1 = 2`, then `return 3*2 = 6`.

The same task can be solved iteratively with a loop.

File: non-recursive-factorial.cpp — iterative factorial using a `for` loop.

```
int factorial(int n)    /* non-recursive-factorial.cpp */
{
    int result = 1;     // Default value for n = 0 or 1
    if (n < 0)          // Error checking
        return -1;

    for (int j = 2; j <= n; j++) // When n >= 2
            result *= j;
    return result;
}
```

8.2.3 Infinite Recursion

- With loops, we have to be careful that the ending condition will be met eventually; otherwise we get an infinite loop (e.g. `for (int j = 1; j != 10; j += 2)` never meets its ending condition).

- Just like infinite loops, we have to be careful that a recursion will eventually end up with a non-recursive base case; otherwise we get infinite recursion!

File: infinite-recursion.cpp — forgetting the base case (which is the ending case too) causes infinite recursion.

```
int factorial(int n)
{
    // Forget the base case!
    return n * factorial(n-1);
}
```

Forgetting the Ending Case

Two typical mistakes both lead to infinite recursion:
- Forgetting the base case entirely, as in `infinite-recursion.cpp`.
- Forgetting to check `n < 0`: with only the `n == 0` base case, the recursion never terminates for negative n.

8.2.4 Example: The Fibonacci Function

The recursive Fibonacci function has two base cases, mirroring the definition of the sequence.

File: fibonacci.cpp — recursive Fibonacci with two base cases.

```
int fibonacci(int n)     /* File: fibonacci.cpp */
{
    if (n == 0)          // Base case #1
        return 0;

    if (n == 1)          // Base case #2
        return 1;

    return fibonacci(n-1) + fibonacci(n-2);
}
```

File: fibonacci-main.cpp — driver that reads n and prints `fibonacci(n)`.

```
#include <iostream>      /* File: fibonacci-main.cpp */
using namespace std;

int fibonacci(int);

int main()
{
    int n;

    while (cin >> n)
    cout << "fibonacci(" << n << ") = "
         << fibonacci(n) << endl;

    return 0;
}
```

Inefficiency of Recursive Fibonacci

The recursive Fibonacci function performs repeated computations. For example, to compute $f(4)$ it computes $f(3)$ and $f(2)$; computing $f(3)$ again computes $f(2)$ and $f(1)$, and each $f(2)$ recomputes $f(1)$ and $f(0)$. The same sub-problems are solved over and over again.

File: non-recursive-fibonacci.cpp — iterative version that keeps track of the last two Fibonacci numbers and avoids repeated computations.

```
int fibonacci(int n)    /* non-recursive-fibonacci.cpp */
{
    int fn;             // keep track of f(n)
    int fn_1 = 1;       // keep track of f(n-1)
    int fn_2 = 0;       // keep track of f(n-2)

    if (n == 0) return 0; // Base case #1
    if (n == 1) return 1; // Base case #2

    for (int j = 2; j <= n; j++)
    {
        fn = fn_1 + fn_2; // f(n) = f(n-1) + f(n-2)

        // Prepare for the calculation of the next fibonacci number
        fn_2 = fn_1;    // f(n-2) = f(n-1)
        fn_1 = fn;      // f(n-1) = f(n)
    }

    return fn;
}
```

8.2.5 Example: Counting Zeros in an Integer

- Example: for the integer 120809, there are 2 zeros.
- Basic idea:
  - Break down the number into quotient and remainder.
  - Count the number of zeros in the quotient and the remainder.

File: num-zeros.cpp — counts the zero digits of an integer by recursion on `n/10` and `n%10`.

```
int num_zeros(int n)    /* File: num-zeros.cpp */
{
    if (n == 0)              // Base case #1
        return 1;

    else if (n < 10 && n > 0) // Base case #2
        return 0;

    else
        return num_zeros(n/10) + num_zeros(n%10);
}
```

File: num-zeros-main.cpp — driver that reads integers and prints the number of zeros in each.

```
#include <iostream>     /* File: num-zeros-main.cpp */
using namespace std;

int num_zeros(int);

int main()
{
    int n;

    while (cin >> n)
        cout << "num_zeros(" << n << ") = " << num_zeros(n) << endl;

    return 0;
}
```

8.2.6 Example: Recursive Solution of Tower of Hanoi

To move a stack of n discs from peg A to peg C using peg B:

1. recursively move the top $n-1$ discs from peg A to peg B (using peg C),
2. move the largest disc from peg A to peg C,
3. recursively move the $n-1$ discs from peg B to peg C (using peg A).

The base case is $n = 0$: nothing to move.

File: toh.cpp — prints the sequence of moves that solves the Tower of Hanoi game.

```
#include <iostream>     /* File: toh.cpp */
using namespace std;

void tower_of_hanoi(int num_discs, char pegA, char pegB, char pegC)
{
    if (num_discs == 0) // Base case
        return;

    tower_of_hanoi(num_discs-1, pegA, pegC, pegB);

    cout << "move disc " << num_discs
         << " from peg " << pegA << " to peg " << pegC << endl;

    tower_of_hanoi(num_discs-1, pegB, pegA, pegC);
}
```

File: toh-main.cpp — driver that reads the number of discs and calls `tower_of_hanoi` with pegs `'A'`, `'B'`, `'C'`.

```
#include <iostream>     /* File: toh-main.cpp */
using namespace std;

void tower_of_hanoi(int, char, char, char);

int main()
{
    int n;

    while (cin >> n)
        tower_of_hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

8.2.7 Example: Binary Search

- Binary search looks for a value in an array sorted in ascending order.
- Compare the target with the middle element: if they are equal, the search is done; if the middle element is larger, search the lower half; otherwise search the upper half.
- Example: searching for the value 9 in $3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33$: the 1st call examines the middle of the whole range; since 9 is smaller, the 2nd call searches only the lower half (3 to 13), and so on.

File: bsearch.cpp — recursive binary search on an index range `[first, last]`.

```
const int NOT_FOUND = -1;       /* File: bsearch.cpp */
int bsearch(const int data[ ], // sorted in ascending order
            int first,         // lower bound index
            int last,          // upper bound index
            int value)         // value to search
{
    if (last < first)          // Base case #1
        return NOT_FOUND;

    int mid = (first + last)/2;

    if (data[mid] == value)
        return mid;             // Base case #2
```

```
    else if (data[mid] > value) // Search the lower half
        return bsearch(data, first, mid-1, value);

    else                        // Search the upper half
        return bsearch(data, mid+1, last, value);
}
```

File: bsearch-main.cpp — driver that searches user inputs in a fixed sorted array.

```
#include <iostream>     /* File: bsearch-main.cpp */
using namespace std;

int bsearch(const int[], int, int, int);

int data[] = { 3, 5, 7, 9, 11, 13, 15, 17 };

int main()
{
    int x;
    int num_data = sizeof(data)/sizeof(int);

    while (cin >> x)
    cout << bsearch(data, 0, num_data-1, x) << endl;

    return 0;
}
```

The same task can be solved iteratively with a loop.

File: non-recursive-bsearch.cpp — iterative binary search with a `while` loop; the search range is maintained by updating `first` and `last`.

```
const int NOT_FOUND = -1;       /* File: non-recursive-bsearch.cpp */
int bsearch(const int data[ ], // sorted in ascending order
            int size,          // number of data in the array
            int value)         // value to search
{
    int first = 0;
    int last = size - 1;

    while (first <= last)
    {
        int mid = (first + last)/2;
        if (data[mid] == value)
            return mid;        // Value found!
        else if (data[mid] > value)
            last = mid - 1;    // Set up for searching the lower half
        else
            first = mid + 1;   // Set up for searching the upper half
    }

    return NOT_FOUND;
}
```

File: non-recursive-bsearch-main.cpp — driver for the non-recursive version (note that it passes the array size instead of the index range).

```
#include <iostream>     /* File: bsearch-main.cpp */
using namespace std;

int bsearch(const int[], int, int);

int data[] = { 3, 5, 7, 9, 11, 13, 15, 17 };

int main()
{
    int x;
    int num_data = sizeof(data)/sizeof(int);
```

```
    while (cin >> x)
    cout << bsearch(data, num_data, x) << endl;

    return 0;
}
```

8.2.8 Advantages and Disadvantages of Recursion

- Many natural phenomena are recursion: a smaller part of oneself is embedded in itself! Examples: a tree, infinite mirror images, fractals.
- As a result, it is usually easier for a programmer to write a solution using recursion ⇒ greater productivity.
- The greater programming productivity is achieved at the expense of more computing resources. To run recursion, it usually requires
  - more memory,
  - more computational time.
- The reason is that whenever a function is called, the computer has to memorize its current state, and passes control from the caller to the callee.

Activation Record

On every function call, the computer

- sets up a new data structure (you may think of it as a scratch paper for rough work) called an activation record which contains information such as
  - where the caller stops,
  - what actual parameters are passed to the callee,
  - new local variables created by the callee function,
  - the return value of the function at the end;
- removes the activation record of the callee when it finishes;
- passes control back to the caller.

8.2.9 Backtracking

- Backtracking is a problem solving technique that examines partial solutions, abandoning unsuitable ones and returning to consider other candidates.
- It follows the "Choose, Explore, Un-choose" loop.
- It can be used to solve crossword puzzles, escape from mazes, or find solutions that are constrained by rules.
- Backtracking can be implemented with a recursive algorithm.

The recursive backtracking pattern:

1. Function `Backtrack(current state)`:
2. Base case: if the current state is a solution / satisfies the goal, add a copy of the current state to the results and return.
3. Iterate through all possible choices; for each choice c in the available choices:
   (a) Constraint check (pruning): if choice c is valid for the current state,
   (b) **Choose**: apply choice c to the current state,
   (c) **Recurse**: call `Backtrack` on the updated state,
   (d) **Un-choose**: undo choice c from the current state.

Example: Generating All Bitstrings of Length N.

- Goal: given a fixed length N, construct every unique sequence composed exclusively of the characters `'0'` and `'1'`.
- Example: if $N = 3$, the bitstrings are "000", "001", "010", "011", "100", "101", "110", "111".
- Basic idea:

– At each position, choose the two digits one by one.
– Place the chosen digit and recursively fill the remaining positions.
– When we iterate through the two choices of characters, there is no need to "undo" as we will overwrite the character at the current index in the next iteration.

File: generate-bitstrings.cpp — generates and prints all bitstrings of length N with recursive backtracking.

```
#include <iostream>     /* File: generate-bitstrings.cpp */
using namespace std;

const char CHOICES[] = {'0', '1'};
const char NULL_CHAR = '\0';

void gen_strings(char s[],    // current bitstring
                 int n,           // target length
                 int index)       // current position
{
    if (index == n) {             // Base Case
        s[index] = NULL_CHAR;     // Make it a valid C String
        cout << s << endl;
        return;
    }

    for (int i=0; i<2; ++i)       // Choices: '0' and '1'
    {
        // Choose: Place the character at the current index
        s[index] = CHOICES[i];

        // Recurse: Move to the next index
        gen_strings(s, n, index + 1);

        // Un-choose (Backtrack)
        // In this specific problem, we don't need to "undo"
        // because it will be overwritten in the next iteration.
        // If undo is needed, we can reset it to the null char.
    }
}

int main()
{
    const int N = 3;
    char s[N + 1] = {}; // +1 for the null character

    gen_strings(s, N, 0);

    return 0;
}
```

No Un-choose Needed Here

In this specific problem the "un-choose" step can be omitted: the character at the current index will simply be overwritten in the next iteration. If an undo were needed, one could reset the position to the null character.

Visualizing the Bitstring Program. The recursive calls of generating the bitstrings of length 3 form a tree:

- The root call `gen_strings(s, 3, 0)` fills index 0; it makes 2 recursive calls with `index = 1`, one with `s[0] = '0'` and one with `s[0] = '1'`.
- Each call at index 1 again branches into 2 calls at index 2, and each of those branches into 2 calls at `index == n`.
- The calls with `index == n` are the base cases: they terminate the string with the null character and print it.

- The $2^3 = 8$ leaves of the tree print exactly the 8 bitstrings "000" to "111"; every internal node corresponds to one "Choose, Recurse" step of the backtracking pattern.

8.3 Further Examples

Example: Factoring.

- Goal: find how many times a factor m appears in the integer n.
- Example: if $n = 48$ and $m = 4$, since $48 = 4 \times 4 \times 3$, the answer is 2.
- Basic idea:
  - Divide n by m until the remainder is non-zero.
  - Increment the count by 1 for every successful division.

File: factor.cpp — counts how many times m divides n by recursion on `n/m`.

```
int num_factors(int n, int m) /* File: factor.cpp */
{
    if (n % m != 0)      // Base case
        return 0;
    else
        return 1 + num_factors(n/m, m);
}
```

9 Scope

9.1 What is the Scope of an Identifier?

- **Scope** is the region of codes in which an identifier declaration is active.
- The scope of an identifier is determined by the location of its declaration.
- In general, an identifier is active from the location of its declaration to the end of its scope.
- In C++, there is a big difference between identifiers declared outside or inside a function.
- Programmers commonly talk about the following 2 kinds of scope (though they are not official in C++'s standard):
  - **global scope**: when an identifier is declared outside any function.
  - **local scope**: when an identifier is declared inside a function.
- Technically, there are at least 3 kinds of scope: **file scope**, **function scope**, and **block scope**.

Three Kinds of Scope

File scope (global), function scope (local), and block scope (local). An identifier is active from its declaration to the end of its scope.

9.2 Example: File/Function/Block Scope

The following program shows all three kinds of scope in one file: the global array `a` has file scope, function parameters and body variables have function scope, and the loop variables `j` and `k` have block scope.

File: scope.cpp — demonstrates file, function and block scope of identifiers in a single program.

```
#include <iostream>      /* File: scope.cpp */
using namespace std;

void my_print(const int b[], int size) // b and size are local \
   variables with a FUNCTION SCOPE
```

```
{
    for (int j = 0; j < size; j++) // j is a local variable with a \
   BLOCK SCOPE
    {
        int k = 10;     // k is a local variable with a BLOCK SCOPE
        cout << "array[" << j <<"] = " << b[j] << '\t' << k*b[j] << \
   endl;
    }
    cout << endl;
}

int a[] = {1,2,3,4,5}; // a is a global variable with a FILE SCOPE

void bad_swap(int& x, int& y) // x, y are local variables with a \
   FUNCTION SCOPE
{
    int temp = x;       // temp is a local variable with a FUNCTION \
   SCOPE
    x = y;
    y = temp;

    a[3] = 100;
}

int main()
{
    // num_array_elements is a local variable with a FUNCTION SCOPE
    int num_array_elements = sizeof(a)/sizeof(int);

    bad_swap(a[1], a[2]); my_print(a, num_array_elements);
    bad_swap(a[3], a[4]); my_print(a, num_array_elements);
    return 0;
}
```

9.3 File Scope

- **File scope** is the technical term for global scope.
- Variables with file scope are **global variables**; they can be accessed by any functions in the same file, or in other files with proper external declarations.
- Unlike local variables, global variables are **initialized to 0** when they are defined without an explicit initializer.
- All function identifiers have file scope; thus, **all functions are global in C++**.

Avoid Global Variables

Undisciplined use of global variables may lead to confusion and makes a program hard to debug. Try to avoid using global variables — use only local variables, and pass them between functions.

9.4 Function Scope

- Function scope is one kind of **local scope**.
- All variables/constants declared in the formal parameter list, or inside the function body, have function scope.
- They are also called **local variables/constants** because they can only be accessed within the function — and not by any other functions.
- They are **short-lived**. They come and go: they are created when the function is called, and are destructed when the function returns.

9.5 Block Scope

- Block scope is also a kind of **local scope**.
- A block of codes is created when you enclose codes within a pair of braces { }. For example, codes inside the body of `for`, `while`, `do-while`, `if`, `else`, `switch`, etc.
- Variables/constants with block scope are also local because they can only be used within the block.
- Similarly to function scope, variables or constants having block scope are short-lived: they are created when the block is entered, and are destructed when the block is finished.
- (There are also namespace scope and class scope, but we won't talk about them.)

9.6 Example: Problems with a Global Variable

The following program shows how a global variable can be modified from several functions, which easily leads to confusion: `main()`, `increment_pbv()` and `increment_pbr()` all touch the global `number`, while pass-by-value and pass-by-reference behave differently for the parameters.

File: global-var-confusion.cpp — demonstrates the confusion caused by a global variable used together with pass-by-value and pass-by-reference.

```
#include <iostream>     /* File: global-var-confusion.cpp */
using namespace std;

int number; // Definition of the global variable, number, with FILE \
   scope. It is initialized to 0.

void increment_pbv(int x)
{
    x++;                // x is a local variable with a FUNCTION scope
    cout << "x = " << x << endl;

    number++; // global variable, number, used in the function, void \
   increment_pbv(int)
}

void increment_pbr(int& y)
{
    y++;              // y is a local reference variable with a \
   FUNCTION scope
    cout << "y = " << y << endl;

    number++; // global variable, number, used in the function, void \
   increment_pbr(int&)
}

int main()
{
    increment_pbv(number); // global variable, number, used in the \
   function, int main()
    cout << "number = " << number << endl;

    increment_pbr(number);  // global variable, number, used in the \
   function, int main()
    cout << "number = " << number << endl;
    return 0;
}
```

9.7 Identifiers of the Same Name

The notion of scope has the following implications:

- An identifier can only be declared **once** in the same scope.
- Only the **name** matters: you cannot declare 2 variables/constants of the same name in the same scope even if they have different types.

```
int x = 1;
char x = 'b';   // error!
```

- However, the same identifier name may be "re-used" for variables or constants in **different scopes**.
- The different scopes may not overlap with each other, or, one scope may be inside another scope.

Compiler Scope Rule

When an identifier is declared more than once but under different scopes, the compiler associates an occurrence of the identifier with its declaration in the **innermost enclosing scope**.

Same Name in Same Scope

Two variables/constants of the same name in the same scope are a compile error, even with different types. Re-using a name is only allowed across different scopes.

9.8 Example: Scope Resolution

The comments in the following skeleton show which declaration of `j` or `k` applies at each statement, according to the compiler scope rule.

```
int main()
{
    int j;      // Apply to S1,S5,S6
    int k;      // Apply to S1,S2,S3,S4,S6
    S1;
    for (...)
    {
        int j;  // Apply to S2,S4
        S2;
        while (...)
        {
            int j;  // Apply to S3
            S3;
        }
        S4;
    }
    while (...)
    {
        int k;  // Apply to S5
        S5;
    }
    S6;
}
```

Which j Applies to S7?

In the code below, `S7` appears before the declaration of the innermost `j` inside the `while` block. Which `j` does `S7` refer to?

```
int main()
{
    int j;      // Apply to S1,S5,S6
```

```
    int k;      // Apply to S1,S2,S3,S4,S6
    S1;
    for (...)
    {
        int j;  // Apply to S2,S4
        S2;
        while (...)
        {
            S7;     // <--- Which j?
            int j;  // Apply to S3
            S3;
        }
        S4;
    }
    while (...)
    {
        int k;  // Apply to S5
        S5;
    }
    S6;
}
```

Declaration Position Matters

An identifier is active only from the location of its declaration to the end of its scope. A use that appears textually before a declaration in the same block binds to the declaration in the enclosing scope instead.

10 Struct

10.1 C++ Structure

10.1.1 What is a Structure?

- A structure is, in general, a collection of heterogeneous objects — different kinds of objects. (c.f. an array, which is a collection of homogeneous objects.)
- Everyday examples: an address book; the human body.
- It is equivalent to record in Pascal.
- C++ allows you to define a new user-defined data type using the keyword `struct`.

10.1.2 struct Definition

Syntax of a struct definition:

```
struct <structure identifier>
{
    <data-type> <identifier for the 1st member>;
    <data-type> <identifier for the 2nd member>;
    ...
};
```

Syntax to define/declare a struct variable:

```
<structure identifier> <variable>;
```

- Each object in a struct is called its member.
- The data types of various members of a struct can be the same or different.
- The member types can be basic data types, user-defined data types, or a pointer to the new struct currently being defined!

- The struct definition just defines a new user-defined data type, not an object. It is usually defined globally.

struct Definition Creates a Type

A struct definition only introduces a new user-defined data type; it does not create any object. Variables of that type are defined separately, and the definition is usually placed globally (e.g. in a header file).

Example: File: point.h — defines a struct `Point` with two `double` members.

```
/* File: point.h */

struct Point
{
    double x;
    double y;
};
```

10.1.3 Access struct Members by the . Operator

Syntax:

```
<struct-variable>.<member-variable>
```

Examples:

```
Point a, b;     // a, b contain garbage

// Initialize a Point struct by memberwise assignments
a.x = 24.5;
a.y = 123.0;

// Input/output of a Point struct one member at a time
cin >> a.x >> a.y;
cout << '(' << b.x << ", " << b.y << ')';
```

- A struct variable that is not initialized contains garbage.
- A struct can be initialized later by memberwise assignments.
- Input/output of a struct is done one member at a time.

10.1.4 struct-struct Assignment: Memberwise Copy

- struct-struct assignment is done by memberwise copy: each member of the struct on the RHS is copied to the corresponding member of the same kind of struct on the LHS.
- Even a member array can be copied this way!

```
struct Example
{
    int x;
    float y[5];
};

// Memberwise copy between 2 structs
Example a, b;
b = a;

// Similar to but different from memberwise assignments
b.x = a.x;
b.y = a.y;  // Error: arrays can't be assigned to each other!!!
```

Array Members Cannot Be Assigned Directly

`b.y = a.y;` is an error because arrays cannot be assigned to each other. However, the whole-struct assignment `b = a;` is legal and copies even the array member `y` through memberwise copy.

10.1.5 Initialization of a struct Variable

- Just like an array variable, a struct variable can be initialized when it is defined using the initializer list with braces:

```
Point a = { 24.5, 123.0 };
```

- If it is not initialized during its definition, later its members can only be modified using separate memberwise assignments or struct-struct assignment (memberwise copy).

```
b.x = 24.5;      // Separate memberwise assignments
b.y = 123.0;     // if no similar object to copy from

// struct-struct assignment to copy a's members to b's
b = a;
```

- For relatively big structures, write a function to do that.

```
void init_point(Point& p, float x, float y)
{
    p.x = x; p.y = y;   // Memberwise initialization
}
```

Write an init Function for Big Structures

When a structure has many members, write an initialization function that takes the struct by reference and sets its members, instead of repeating memberwise assignments everywhere.

Example (Euclidean Distance): File: point-test.cpp — reads 3 points from the user and prints the length of the sides of the triangle they form.

```
#include <iostream>      /* File: point-test.cpp */
using namespace std;
#include "point.h"
#include "point-distance.cpp"

int main()       /* To find the length of the sides of a triangle */
{
    Point a, b, c;
    cout << "Enter the co-ordinates of point A: "; cin >> a.x >> a.y;
    cout << "Enter the co-ordinates of point B: "; cin >> b.x >> b.y;
    cout << "Enter the co-ordinates of point C: "; cin >> c.x >> c.y;

    cout << endl << "Results: " << endl;
    print_distance(a, b);
    print_distance(b, c);
    print_distance(c, a);
    return 0;
}
```

File: point-distance.cpp — functions to compute the 2D Euclidean distance between two points and to print points and distances; structs are passed by const reference.

```
/* File: point-distance.cpp */
#include <cmath>         // Header file for C math lib

/* To find the 2D Euclidean distance between 2 points */
```

```
double euclidean_distance(const Point& p1, const Point& p2)
{
    double x_diff = p1.x - p2.x, y_diff = p1.y - p2.y;
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

void print_point(const Point& p)
{
    cout << '(' << p.x << ", " << p.y << ')';
}

void print_distance(const Point& p1, const Point& p2)
{
    cout << "Distance between "; print_point(p1);
    cout << " and "; print_point(p2);
    cout << " is " << euclidean_distance(p1, p2) << endl;
}
```

10.1.6 Case Study: The Student Record struct

File: student-record.h — defines an enum `Dept`, a struct `Date`, a struct `Student_Record` with a nested `Date` member, and a global constant array of department names.

```
enum Dept { CSE, ECE, MATH }; /* File: student-record.h */

struct Date
{
    unsigned int year;
    unsigned int month;
    unsigned int day;
};

struct Student_Record
{
    char name[32];
    unsigned int id;
    char gender;
    Dept dept;
    Date entry;
};

// Global constants for department names
const char dept_name[][30]
  = {"Computer Science", "Electrical Engineering", "Mathematics"};
```

Accessing members of a `Student_Record`:

```
#include <cstring>  // Load the lib header file for strcpy

Student_Record x;   // x contains garbage
strcpy(x.name, "Adam");
x.id = 12345;
x.gender = 'M';
x.dept = CSE;

// Notice how members of nested structures can be assigned
// values through successive use of the dot operator
x.entry.year = 2006;
x.entry.month = 9;
x.entry.day = 1;
```

- A `char` array member such as `name` is assigned with `strcpy`, not with `=`.
- Members of nested structures are accessed through successive use of the dot operator, e.g. `x.entry.year`.

Initialize using the braces while it is defined (note the nested braces for the `Date` member):

```
Student_Record a = { "Adam", 12345, 'M', CSE, { 2006, 9, 1 } };
```

Initialize using functions:

```
void init_date(Date& x,
               unsigned int year,
               unsigned int month,
               unsigned int day)
{
    x.year = year;
    x.month = month;
    x.day = day;
}

void init_student_record(Student_Record& a,
                         const char name[], unsigned int id,
                         char gender, Dept dept, const Date& date)
{
    strcpy(a.name, name);
    a.id = id;
    a.gender = gender;
    a.dept = dept;
    a.entry = date;     // struct-struct assignment
}
```

Example: File: student-record.cpp — defines `print_date` and `print_student_record` and prints one initialized student record.

```
#include <iostream>     /* File: student-record.cpp */
#include "student-record.h"
using namespace std;

void print_date(const Date& date) {
    cout << date.year << '/' << date.month << '/' << date.day << endl;
}

void print_student_record(const Student_Record& x) {
    cout.width(12); cout << "name: " << x.name << endl;
    cout.width(12); cout << "id: " << x.id << endl;
    cout.width(12); cout << "gender: " << x.gender << endl;
    cout.width(12); cout << "dept: " << dept_name[x.dept] << endl;
    cout.width(12); cout << "entry date: "; print_date(x.entry);
}

int main()
{
    Student_Record a = { "Adam", 12345, 'M', CSE, { 2006, 9, 1 } };
    print_student_record(a); return 0;
}
```

10.2 Array of Structures

10.2.1 Array of Structures

- You may create an array of basic data types as well as user-defined data types, such as structures.
- Example: `student_record sr[3];` creates an array of 3 student records (e.g. "Adam", 12000, 'M', CSE; "Bob", 11000, 'M', MATH; "Cathy", 10000, 'F', ECE — ignoring the `Date` member of `Student_Record`).

Example: File: student-record-array.cpp — creates and initializes an array of 3 `Student_Record` objects and prints them one by one; the number of records is computed by `sizeof(sr)/sizeof(Student_Record)`.

```
#include <iostream>     /* File: student-record-array.cpp */
using namespace std;
#include "student-record-functions.cpp"

int main()
{
    Student_Record sr[] = {
        { "Adam",  12000, 'M', CSE,  { 2006, 1, 10 } },
        { "Bob",   11000, 'M', MATH, { 2005, 9, 1  } },
        { "Cathy", 10000, 'F', ECE,  { 2006, 8, 20 } }
    };

    for (int j = 0; j < sizeof(sr)/sizeof(Student_Record); ++j)
        print_student_record(sr[j]);

    return 0;
}
```

File: student-record-functions.cpp — shared helper functions to print and initialize `Date` and `Student_Record` objects.

```
/* File: student-record-functions.cpp */
#include <cstring>
#include "student-record.h"

void print_date(const Date& date)
{
    cout << date.year << '/' << date.month << '/' << date.day << endl;
}

void print_student_record(const Student_Record& x)
{
    cout.width(12); cout << "name: " << x.name << endl;
    cout.width(12); cout << "id: " << x.id << endl;
    cout.width(12); cout << "gender: " << x.gender << endl;
    cout.width(12); cout << "dept: " << dept_name[x.dept] << endl;
    cout.width(12); cout << "entry date: "; print_date(x.entry);
}




void init_date(Date& x, unsigned int year,
               unsigned int month, unsigned int day)
{
    x.year = year;
    x.month = month;
    x.day = day;
}

void init_student_record(Student_Record& a, const char name[],
                         unsigned int id, char gender,
                         Dept dept, const Date& date)
{
    strcpy(a.name, name);
    a.id = id;
    a.gender = gender;
    a.dept = dept;
    a.entry = date;     // struct-struct assignment
```

```
}
```

Example: File: sort-student-record.cpp — sorts 3 student records by id; swapping two records is easy thanks to struct-struct assignment.

```
#include <iostream>     /* File: sort-student-record.cpp */
using namespace std;
#include "student-record-functions.cpp"

void swap_SR(Student_Record& x, Student_Record& y) {
    Student_Record temp = x; x = y; y = temp;
}
void sort_3SR_by_id(Student_Record sr[]) {
    // What does the following print?
    cout << "#records = " << sizeof(sr)/sizeof(Student_Record) << \
   endl;
    if (sr[0].id > sr[1].id) swap_SR(sr[0], sr[1]);
    if (sr[0].id > sr[2].id) swap_SR(sr[0], sr[2]);
    if (sr[1].id > sr[2].id) swap_SR(sr[1], sr[2]);
}

int main() {
    Student_Record sr[] = {
        { "Adam",  12000, 'M', CSE,  { 2006, 1, 10 } },
        { "Bob",   11000, 'M', MATH, { 2005, 9, 16 } },
        { "Cathy", 10000, 'F', ECE,  { 2006, 8, 27 } } };
    sort_3SR_by_id(sr);
    for (int j = 0; j < sizeof(sr)/sizeof(Student_Record); j++)
        print_student_record(sr[j]);
    return 0;
}
```

What Does `sizeof(sr)` Print Inside the Function?

Inside `sort_3SR_by_id`, what does the statement printing `sizeof(sr)/sizeof(Student_Record)` output — is it 3?

sizeof on an Array Parameter

The array parameter `sr` of `sort_3SR_by_id` is really a pointer, so `sizeof(sr)` inside the function gives the size of a pointer, not of the whole array — the printed "#records" is not 3. In `main`, `sizeof(sr)` works correctly because `sr` is a genuine array there.

10.2.2 Return by Reference Again

File: student-record-rbr.cpp — `smaller_id` returns a reference to the record with the smaller id, so the returned object can be modified directly: `smaller_id(a, b).dept = CSE;` changes the department of the actual student.

```
#include <iostream>     /* File: student-record-rbr.cpp */
using namespace std;
#include "student-record-functions.cpp"

Student_Record& smaller_id(Student_Record& x, Student_Record& y)
{
    return (x.id < y.id) ? x : y;
}

int main() /* To allow student will smaller ID to transfer to CSE */
{
```

```
    Student_Record a = { "Amy", 12000, 'F', MATH, { 2006, 1, 10 } };
    Student_Record b = { "Bob", 11000, 'M', MATH, { 2005, 9, 21 } };

    cout << "<<< Before changing department >>>" << endl;
    print_student_record(a); print_student_record(b);
    smaller_id(a, b).dept = CSE;

    cout << "\n\n<<< After changing department >>>" << endl;
    print_student_record(a); print_student_record(b);
    return 0;
}
```

Question: What happens if `smaller_id()` returns by value?

File: a.cpp — the return-by-value counterpart of the above program (its header comment names it `student-record-rbv.cpp`); now `smaller_id(a, b).dept = CSE;` only modifies a temporary copy, so the records of `a` and `b` stay unchanged.

```
#include <iostream>     /* File: student-record-rbv.cpp */
using namespace std;
#include "student-record-functions.cpp"

Student_Record smaller_id(Student_Record& x, Student_Record& y)
{
    return (x.id < y.id) ? x : y;
}

int main() /* To allow student will smaller ID to transfer to CSE */
{
    Student_Record a = { "Amy", 12000, 'F', MATH, { 2006, 1, 10 } };
    Student_Record b = { "Bob", 11000, 'M', MATH, { 2005, 9, 21 } };

    cout << "<<< Before changing department >>>" << endl;
    print_student_record(a); print_student_record(b);
    smaller_id(a, b).dept = CSE;

    cout << "\n\n<<< After changing department >>>" << endl;
    print_student_record(a); print_student_record(b);
    return 0;
}
```

Return by Reference Allows Modifying the Original

When a function returns a struct by reference, the call result is an lvalue referring to the original object, so assignments like `smaller_id(a, b).dept = CSE;` modify the caller's record. Returning by value yields a copy, and any modification is lost.

11 Function III: Miscellaneous

11.1 Part I: Function Declaration and Function Definition

11.1.1 Some Function Terminology

- A declaration such as `int max(int, int);` is a **function prototype**.
- The **signature** of a function is its name together with the list of formal-parameter types (without identifier names). The return type is not part of the signature.

```
int max(int, int);   // <-- a function prototype (the whole \
   declaration)
 ^
 |-- return type
     +-- function name: max
```

```
  max(int, int)   // <-- the signature: name + types of formal \
parameters
```

11.1.2 Function Prototype

A function prototype consists of:

1. the function name;
2. the return data type;
3. the number of formal parameters;
4. the data type of the formal parameters.

Example: function prototypes (the comments show the corresponding definitions).

```
// int factorial(int n) { ... }
int factorial(int);

// float euclidean_distance(float x1,float y1,float x2,float y2) { \
   ... }
float euclidean_distance(float, float, float, float);

/* void print_tree(int tree_height, char tree_symbol,
char trunk_symbol, char pot_symbol) { ... } */
void print_tree(int, char, char, char);
```

- The identifier names of the formal parameters are **not** part of the signature — the names are immaterial.
- A function prototype describes the **interface** of the function: what parameters it takes in and what value it returns. Technically, a function prototype is also called the **application programming interface (API)**.

```
/* All the following 3 function definitions are equivalent */
int max(int x, int y) { return (x > y) ? x : y; }
int max(int a, int b) { return (a > b) ? a : b; }
int max(int f, int g) { return (f > g) ? f : g; }
```

Prototype vs Signature

The prototype is the whole declaration: return type, name, plus the number and types of formal parameters. The signature is only the name plus the parameter types — parameter names and the return type are immaterial.

11.1.3 Function Declaration vs. Definition

- A function is **declared** by writing down its interface — its function prototype.
- A function is **defined** by writing down its function header plus its function body.
- A function definition asks the compiler to generate machine codes according to the C++ codes in its function body; a function declaration just informs the compiler about the function's interface without generating any machine codes.
- A function may be **declared many times**, but a function can be **defined only once**.
- When a function is defined, it is also declared. But simply declaring a function does not define it.

Declare Many Times but Define Once

Declarations may be repeated as often as needed; a definition (which also serves as a declaration) must appear exactly once in the whole program.

In C++, **all functions must be declared before they can be used**, so that the compiler can

- make sure the exact number of arguments are passed;
- do type checking on the arguments passed to the function.

That is, if function A wants to call function B, function B must be

- declared/defined before, or
- declared inside function A before calling function B.

However, a function need not be defined before it can be used, although it must be defined eventually somewhere in the whole program in order that the program can be compiled to an executable.

11.1.4 Different Ways of Declaring Functions

Consider a program where `main()` calls `mean_max()`, which in turn calls `mean()` and `max()`:

```
main --> mean_max --> mean
                  \-> max
```

The next three examples show three different ways of making the needed declarations.

The simplest approach: define each function before any function that calls it. Since a definition is also a declaration, no separate prototypes are needed.

File: fcn-prototype1.cpp — functions are declared simply by defining them in the right order.

```
#include <iostream>     /* File: fcn-prototype1.cpp */
using namespace std;

int max(int x, int y) { return (x > y) ? x : y; }
int mean(int x, int y) { return (x + y)/2; }

void mean_max(int x, int y, int& mean_num, int& max_num)
{
    mean_num = mean(x, y);
    max_num = max(x, y);
}

int main()
{
    int average, bigger;

    mean_max(6, 4, average, bigger);
    cout << "mean = " << average << endl << "max = " << bigger << \
   endl;
    return 0;
}
```

Alternatively, put global prototypes wherever they are needed so that each caller sees the callee's interface before its own code.

File: fcn-prototype2.cpp — `mean_max` is declared globally before `main()`; `max` and `mean` are declared globally before the definition of `mean_max`.

```
#include <iostream>     /* File: fcn-prototype2.cpp */
using namespace std;

void mean_max(int, int, int&, int&); // main only needs to know \
   mean_max

int main()
{
    int average, bigger;
    mean_max(6, 4, average, bigger);
    cout << "mean = " << average << endl << "max = " << bigger << \
   endl;
```

```
    return 0;
}

int max(int, int);      // mean_max needs to know max and mean
int mean(int, int);

void mean_max(int x, int y, int& mean_num, int& max_num)
{
    mean_num = mean(x, y);
    max_num = max(x, y);
}

int max(int x, int y) { return (x > y) ? x : y; }
int mean(int x, int y) { return (x + y)/2; }
```

- A function can only "see" other functions that are declared **before its own definition**, even if it does not use them.

  A function may also be declared locally, inside the body of the function that calls it.

File: fcn-prototype3.cpp — `mean_max` is declared locally inside `main()`; `max` and `mean` are declared locally inside `mean_max`.

```
#include <iostream>     /* File: fcn-prototype3.cpp */
using namespace std;

int main()
{
    void mean_max(int, int, int&, int&);
    int average, bigger;

    mean_max(6, 4, average, bigger);
    cout << "mean = " << average << endl << "max = " << bigger << \
   endl;
    return 0;
}

void mean_max(int x, int y, int& mean_num, int& max_num)
{
    int max(int, int);
    int mean(int, int);

    mean_num = mean(x, y);
    max_num = max(x, y);
}

int max(int x, int y) { return (x > y) ? x : y; }
int mean(int x, int y) { return (x + y)/2; }
```

- With local declarations, only those functions that are used by a function are known to the latter.

When two functions call each other (mutual recursion), one of them must be declared first with a **forward declaration**, since each definition needs the other function to be known.

File: odd-even.cpp — `odd()` calls `even()` and vice versa, so `even` is forward-declared before `odd` is defined.

```
#include <iostream>     /* File: odd-even.cpp */
using namespace std;

bool even(int);

bool odd(int x) { return (x == 0) ? false : even(x-1); }

bool even(int x) { return (x == 0) ? true : odd(x-1); }

int main()
```

```
{
    int x;
    cin >> x;            // Assume x > 0

    cout << boolalpha << odd(x) << endl;
    cout << boolalpha << even(x) << endl;

    return 0;
}
```

11.2 Part II: Function Overloading

11.2.1 Signature of a Function

- Recall that in C++, **all functions are global**. That means, in general, all functions can "see" each other.
- Just as we use one's signature to identify a person, we identify a function by its **signature**.
- A function's signature includes its **name** and the **list of formal parameters without their identifier names**.
- No two C++ functions can have the same signature, **even if they have different return types**.
- But two C++ functions can have the same name with different signatures ⇒ **function overloading**.

```
// The following 2 function definitions of
// pick_one cannot appear in the same program
int pick_one(int x, float y) { return x; }
float pick_one(int x, float y) { return y; }
```

Differing Only in Return Type Is an Error

The return type is not part of the signature. Two prototypes that differ only in their return type have the same signature and cannot appear in the same program.

11.2.2 Function Overloading

C++ allows several functions to have the **same name** but **different types of input parameters**.

```
int max(int x, int y) { return (x > y) ? x : y; }
int max(int x, int y, int z) { return max(max(x,y), z); }
double max(double a, double b) { return (a > b) ? a : b; }

void swap(int& a, int& b) { int temp = a; a = b; b = temp; }
void swap(float& a, float& b) { float temp = a; a = b; b = temp; }
void swap(double& a, double& b) { double temp = a; a = b; b = temp; }

int absolute(int a) { return (a < 0) ? -a : a; }
int absolute(int& a) { return (a = (a < 0) ? -a : a); }
```

Question: with both versions of `absolute()` above, how can you call the pass-by-reference version `int absolute(int&)`?

File: which-absolute.cpp — asks which overloaded `absolute` is called for a literal argument and for a variable argument.

```
#include <iostream>     /* File: which absolute.cpp */
using namespace std;

int absolute(int a) { cout << "version 1: "; return (a < 0) ? -a : \
   a; }
```

```
int absolute(int& a) { cout << "version 2: "; return (a = (a < 0) ? \
   -a : a); }

int main()
{
    int x = -456;

    cout << absolute(-234) << endl; // Which absolute function is \
   called?
    cout << absolute(x) << endl;    // Which absolute function is \
   called?

    return 0;
}
```

- `absolute(-234)` calls version 1, `int absolute(int)`: a literal is an rvalue and cannot bind to the non-const reference `int&`.
- `absolute(x)` is an exact match for both versions. In fact, modern g++ rejects this call as **ambiguous** (a compile error) — in practice you cannot call version 2 while version 1 is visible; overloads on `T` and `T&` are a dangerous design.

Overloading on Value and Reference Is Ambiguous

Overloading `f(int)` against `f(int&)` compiles by itself, but a call with an `int` variable matches both versions equally well and is ambiguous to the compiler. Only a call with an rvalue (e.g. a literal) unambiguously selects the by-value version.

Neither renaming the formal parameters nor changing only the return type creates a new signature:

```
/* Identifier names of formal parameters are immaterial */
int max(int x, int y) { return (x > y) ? x : y; }
int max(int a, int b) { return (a > b) ? a : b; }

/* Return type is not part of the signature */
void swap(int& a, int& b) { int temp = a; a = b; b = temp; }
int swap(int& a, int& b) { int temp = a; a = b; b = temp; return a; }
```

Both pairs are redefinitions of the same signature and hence compile errors.

11.2.3 Overloaded Function Resolution

- When an overloaded function is called, C++ will determine exactly which function among those with the same name should be called — **function resolution**.
- Function resolution is done by comparing the types of the **actual parameters** passed in a function call and the **formal parameters** in the function definitions, and finding the **best match**.

Best Match Order

The compiler looks for a match in the following order:

1. **exact match**;
2. match after some **type promotion**: `char`/`bool`/`short` → `int`; `float` → `double`;
3. match after some **standard type conversion**: between integral types; between floating types; between integral and floating types;
4. match after some **user-defined type conversion** (later).

Given the two overloaded functions:

```
int test(int a, double b);
int test(double a, int b);
```

- `test(3, 4.6)`: the compiler picks the **first** version (exact match).

- `test('a', 4.6)`: the compiler again picks the **first** version, by promoting `'a'` to an `int`.
- `test(3.2, 4.6)`: it can either match the first version by a narrowing conversion of the first parameter to `int`, or match the second version by a narrowing conversion of the second parameter to `int`. Since neither one is more preferable than the other ⇒ **compilation error**!

Ambiguous Call Is a Compile Error

If two or more overloads match a call equally well, the compiler does not guess — the program fails to compile. Design overloads so that every call has a single best match.

11.3 Default Function Arguments

11.3.1 Default Function Argument

- Sometimes, we would like a function to have certain **default behaviour**, but still allow the user to change it.
- C++ allows the user to call a function with **fewer arguments** if all he wants is its default behaviour, and with **more arguments** if he wants some particular behaviour of the function.
- A function may have **more than one** default argument. But all default arguments must be specified **at the end** of the formal parameter list.

```
/* The following 2 prototypes are equivalent */
void func(int x, float& y, char gender = 'M', bool alive = true);
void func(int, float&, char = 'M', bool = true);
```

- The default argument(s) may be specified in a function **declaration** or in the function **definition**, but **not both**.
- Usually we put them on the function declaration. Why? Because the compiler fills in the missing arguments at the call site, where typically only the declaration is visible.
- A function with default arguments looks like several overloaded functions, but it is **not**.

Default Arguments Go Last

All default arguments must be specified at the end of the formal parameter list, and they may appear in the declaration or the definition — but not both.

Put Defaults in the Declaration

Since the caller only sees the declaration (prototype), placing the default values there lets every call site use them. Putting them in the definition only helps callers that appear after that definition.

The true `getline` function header is:

```
cin.getline(char s[ ], int max-num-char, char terminator='\n');
```

Thus, you may call it with only 2 arguments, and the default terminator is the newline character.

```
const int MAX_LINE_LEN = 1000;
char s[MAX_LINE_LEN+1];

cin.getline(s, sizeof(s));          // terminator = newline
cin.getline(s, sizeof(s), '.'));    // terminator = full stop
```

11.3.2 Examples: Functions with Default Arguments

File: increment-default-arg.cpp — `step` defaults to 1, so `increment(10)` returns 11 and `increment(10, 5)` returns 15.

```
#include <iostream>     /* File: increment-default-arg.cpp */
using namespace std;

int increment(int x, int step = 1)
{
    return (x + step);
}

int main()
{
    cout << increment(10) << endl;
    cout << increment(10, 5) << endl;

    return 0;
}
```

File: sort-default-arg.cpp — the `order` parameter defaults to `ASCENDING`; the first call sorts `a` and `b` in ascending order (prints 8 then 24), and the second call explicitly requests descending order (prints 24 then 8).

```
#include <iostream>     /* File: sort-default-arg.cpp */
using namespace std;

enum sort_order {ASCENDING, DESCENDING};

void swap(int& a, int& b) { int temp = a; a = b; b = temp; }

void sort(int& x, int& y, sort_order order = ASCENDING)
{
    if (order == ASCENDING) { if (x > y) swap(x, y); }
    else { if (x < y) swap(x, y); }
}

int main()
{
    int a = 24, b = 8;
    sort(a, b); cout << a << '\t' << b << endl;
    sort(a, b, DESCENDING); cout << a << '\t' << b << endl;
    return 0;
}
```

12 Pointer

12.1 lvalue (Address) and rvalue (Content)

12.1.1 Variables

- A variable is a symbolic name assigned to some memory storage.
- The size of this storage depends on the type of the variable, e.g. `char` is 1-byte long and `int` is 4-byte long.
- The difference between a variable and a literal constant is that a variable is addressable. e.g. in `x = 100;`, `x` is a variable and `100` is a literal constant.

12.1.2 lvalue and rvalue

A variable has dual roles. Depending on where it appears in the program, it can represent an

- **lvalue**: location of the memory storage (read-write).
- **rvalue**: value in the storage (read-only).

They are so called because a variable represents an lvalue (rvalue) if it is written to the left (right) of an assignment statement, e.g. `x = x + 1;`.

Which of the following C++ statements are valid? Why?

File: test-lr-value.cpp — the quiz program on lvalue/rvalue: `++++++x` is valid but `x++++++` is not.

```
#include <iostream>      /* File: test-lr-value.cpp */
using namespace std;

int main()
{
    int x;

    cout << ++++++x << endl;
    cout << x++++++ << endl;

    return 0;
}
```

- `4 = 1;` — invalid: a literal constant is not an lvalue.
- `(x + 10) = 6;` — invalid: the result of `x + 10` is an rvalue.
- `cout << ++++++x << endl;` — valid (ANSI C++ Ref. Section 5.3.2).
- `cout << x++++++ << endl;` — invalid (ANSI C++ Ref. Section 5.2.6).

12.1.3 lvalue & rvalue: Return-by-Reference vs. Return-by-Value

`++x`: the pre-increment operator

1. requires `x` to be passed-by-reference
2. modify `x` by incrementing it by 1
3. returns `x` (with its new value) by reference
4. the returned `x` is an **lvalue**

`x++`: the post-increment operator

1. requires `x` to be passed-by-reference
2. saves the current value of `x` in some temporary local variable
3. modify `x` by incrementing it by 1
4. returns the old value of `x` in the local variable by value
5. the returned value is an **rvalue**

Pre-increment Returns lvalue While Post-increment Returns rvalue

`++x` returns `x` itself by reference (an lvalue), so `++++++x` is legal. `x++` returns the old value by value (an rvalue), so `x++++++` is illegal.

Analogy: a website has an lvalue and an rvalue too! The lvalue of the CSE website is its address (URL); the rvalue is its content (the webpage you see).

12.1.4 Get the Address by the Reference Operator &

Syntax: get the address of a variable.

```
& <variable>
```

File: var-addr.cpp — prints the values and the addresses of variables of types `int` and `short`.

```
#include <iostream> /* File: var-addr.cpp */
using namespace std;

int main()
{
```

```
    int x = 10, y = 20;
    short a = 9, b = 99;

    cout << "x = " << x << '\t' << "address of x = " << &x << endl;
    cout << "y = " << y << '\t' << "address of y = " << &y << endl;
    cout << "a = " << a << '\t' << "address of a = " << &a << endl;
    cout << "b = " << b << '\t' << "address of b = " << &b << endl;
    return 0;
}
```

Example: File: fcn-var-addr.cpp — shows the difference between PBV and PBR by printing addresses of formal parameters inside a function.

```
#include <iostream>     /* File: fcn-var-addr.cpp */
using namespace std;

void f(int x2, int& y2)
{
    short a = 9, b = 99;
    cout << endl << "Inside f(int, int&)" << endl;
    cout << "x2 = " << x2 << '\t' << "address of x2 = " << &x2 << \
   endl;
    cout << "y2 = " << y2 << '\t' << "address of y2 = " << &y2 << \
   endl;
    cout << "a  = " << a  << '\t' << "address of a  = " << &a  << \
   endl;
    cout << "b  = " << b  << '\t' << "address of b  = " << &b  << \
   endl;
}

int main()
{
    int x = 10, y = 20;
    cout << endl << "Inside main()" << endl;
    cout << "x = " << x << '\t' << "address of x = " << &x << endl;
    cout << "y = " << y << '\t' << "address of y = " << &y << endl;
    f(x, y);
    return 0;
}
```

Question: Can you see the difference between PBV and PBR? (The PBV parameter `x2` has a different address from `x`; the PBR parameter `y2` has the same address as `y`.)

12.2 What is a Pointer?

12.2.1 Pointer Variable

Syntax: pointer variable definition.

```
<type>* <variable>;
```

- A pointer variable stores the address of another variable.
- If variable `y` stores the address of variable `x`, we say "`y` points to `x`."
- Notice that a pointer variable is just a variable which has its own address in memory.

File: pointer-var.cpp — defines an `int` pointer `y` that points to `x`, and prints both its value (an address) and its own address.

```
#include <iostream>     /* File: pointer-var.cpp */
using namespace std;

int main() {
    int x = 10; int* y = &x; // y now contains the address of x
```

```
    cout << "x = " << x << '\t' << "address of x = " << &x << endl;
    cout << "y = " << y << '\t' << "address of y = " << &y << endl;
    return 0;
}
```

12.2.2 Get the Content by the Dereference Operator *

Syntax: get the content through a pointer variable.

```
*<pointer variable>
```

File: pointer-deref.cpp — uses `*y` to read the content of the address stored in `y` and assigns it to `z`.

```
#include <iostream>     /* File: pointer-deref.cpp */
using namespace std;

int main()
{
    int x = 10, z = 20;
    int* y = &x; // y now contains the address of x
    cout << "x = " << x << '\t' << "address of x = " << &x << endl;
    cout << "z = " << z << '\t' << "address of z = " << &z << endl;
    cout << "y = " << y << '\t' << "address of y = " << &y << endl;

    z = *y; // Get content from the address stored in y, put it into z
    cout << endl;
    cout << "z = " << z << '\t' << "address of z = " << &z << endl;
    cout << "y = " << y << '\t' << "*y = " << *y << endl;
    return 0;
}
```

Example: File: pointer.cpp — demonstrates modifying pointed-to values through dereferencing, and the difference between assigning `*p1 = *p2` (copy contents) and `p1 = p2` (copy addresses).

```
#include <iostream>     /* File: pointer.cpp */
using namespace std;

int main()
{
    int x1 = 10, x2 = 20;
    int *p1 = &x1;      // p1 now points to x1
    int *p2 = &x2;      // p2 now points to x2

    *p1 = 5;            // now x1 = 5
    *p2 += 1000;        // now x2 = 1020
    *p1 = *p2;          // now *p1 = *p2 = x1 = x2 = 1020, but p1 != \
   p2
    p1 = p2;            // now p1 and p2 both point to x2

    cout << "x1 = " << x1 << '\t' << "&x1 = " << &x1 << endl;
    cout << "x2 = " << x2 << '\t' << "&x2 = " << &x2 << endl;
    cout << "p1 = " << p1 << '\t' << "*p1 = " << *p1 << endl;
    cout << "p2 = " << p2 << '\t' << "*p2 = " << *p2 << endl;

    return 0;
}
```

Example: File: pointer-sizeof.cpp — compares `sizeof` a pointer, `sizeof` the object it points to, and `sizeof` the address of the pointer itself, for `char`, `short`, `int` and `double`.

```
#include <iostream> /* File: pointer-sizeof.cpp */
using namespace std;

int main()
{
    char   c = 'A';   char* pc = &c;
    short  s = 5;    short* ps = &s;
    int    i = 10;     int* pi = &i;
    double d = 5.6; double* pd = &d;

    cout << sizeof(pc) << '\t' << sizeof(*pc) << '\t' << sizeof(&pc)
         << endl;
    cout << sizeof(ps) << '\t' << sizeof(*ps) << '\t' << sizeof(&ps)
         << endl;
    cout << sizeof(pi) << '\t' << sizeof(*pi) << '\t' << sizeof(&pi)
         << endl;
    cout << sizeof(pd) << '\t' << sizeof(*pd) << '\t' << sizeof(&pd)
         << endl;

    return 0;
}
```

- `sizeof(pc)`: size of the pointer itself (same for all pointer types on a given machine).
- `sizeof(*pc)`: size of the pointed-to object (depends on the type).
- `sizeof(&pc)`: the address of the pointer is again a pointer, so it has pointer size.

12.2.3 What can a Pointer Point to?

A pointer can point to

- objects of basic types: `char`, `short`, `int`, `long`, `float`, `double`, etc.
- objects of user-defined types: `struct`, `class` (discussed later)
- another pointer!
- even to a function ⇒ function pointer!

Example: File: pointer-pointer.cpp — a chain of pointers: `xppp` points to `xpp`, which points to `xp`, which points to `x`; each level of dereferencing goes one step deeper.

```
#include <iostream>     /* File: pointer-pointer.cpp */
using namespace std;

int main()
{
    int x = 16;
    int* xp = &x;       // xp --> x
    int** xpp = &xp;    // xpp --> xp --> x
    int*** xppp = &xpp; // xppp --> xpp --> xp --> x

    cout << "x address    = " << &x   << "  x    = " << x << endl;
    cout << "xp address   = " << &xp  << "  xp   = " << xp
         << "  *xp   = "      << *xp  << endl;

    cout << "xpp address  = " << &xpp << "  xpp  = " << xpp
         << "  *xpp  = "      << *xpp << "  **xpp  = " << **xpp << \
   endl;

    cout << "xppp address = " << &xppp   << "  xppp = " << xppp
         << "  *xppp = "      << *xppp   << "  **xppp = " << **xppp
         << "  ***xppp = "    << ***xppp << endl;
    return 0;
}
```

12.2.4 Variable, Reference Variable, Pointer Variable

File: confusion.cpp — compares an `int` variable, a reference variable (alias) and a pointer variable, printing values and addresses in hexadecimal.

```
#include <iostream>     /* File: confusion.cpp */
using namespace std;

int x = 5;              // An int variable
int& xref = x;          // A reference variable: xref is an alias of x
int* xptr = &x;         // A pointer variable: xptr points to x

void xprint()
{
    cout << hex << endl; // Print numbers in hexadecimal format
    cout << "x =     " << x     << "\t\tx address     = " << &x    \
   << endl;
    cout << "xref =  " << xref  << "\t\txref address  = " << &xref \
   << endl;
    cout << "xptr =  " << xptr  << "\txptr address  = "   << &xptr \
   << endl;
    cout << "*xptr = " << *xptr << endl;
}

int main()
{
    x += 1; xprint();
    xref += 1; xprint();
    xptr = &xref; xprint(); // Now xptr points to xref

    return 0;
}
```

- `xref` is an alias of `x`: `&xref` is the same as `&x`.
- `xptr` is a separate variable with its own address; its value is the address of `x`.

12.2.5 `const` Pointer and Pointer to `const` Objects

Syntax: const pointer definition.

```
<type>* const <pointer variable> = &<another variable>;
```

- A const pointer must be initialized when it is defined; just like any C++ constant.
- A const pointer, once initialized, cannot be changed to point to something else.
- However, you are free to change the content in the address it points to.

```
int x = 10, y = 20;
int* const xcp = &x;
xcp = &y;       // Compile Error: a const pointer!
*xcp = 5;       // Compile Okay: what it points to is not const
```

Syntax: definition of pointer to a const object.

```
const <type>* <pointer variable>;
```

```
int x = 10, y = 20;
const int* pc = &x;
pc = &y;        // Compile Okay: pc is free to point to x, y, z, or \
   any int
*pc = 5;        // Compile Error: its content is const when accessed \
   thru pc!
y = 8;          // Compile Okay: y is not a const object
```

- It is not necessary to initialize a pointer to const object when it is defined, though you may.
- You are free to change the pointer itself to point to different objects during program execution.
- However, the content of the object pointed to by such pointer cannot be changed through the pointer. But the content of the object can still be changed by the object directly!
- Analogy: the anchor texts on a webpage that allow you to surf other websites are pointers to const webpages since, in your perspective, you cannot change the content of those webpages.

const Pointer vs Pointer to const

`int* const p` is a const pointer: `p` cannot be re-pointed, but `*p` can be modified. `const int* p` is a pointer to const: `p` can be re-pointed, but `*p` cannot be modified through `p`.

Quiz: (const) Pointer to (const) Objects

Can you tell the differences among the following?

```
int* p;
const int* p;
int* const p;
const int* const p;
```

12.2.6 PBR = PBV + Pointer

- The programming language C only has one way to pass arguments to a function, which is PBV.
- To simulate the effect of PBR, one may pass the address of an object to a function.
- Inside the function, the object is represented by a pointer.
- Then one may change the object's value by dereferencing the object's pointer inside the function.

File: pbv-pointer.cpp — a `swap` function written with pointers: addresses of `a` and `b` are passed, and the values are swapped by dereferencing.

```
#include <iostream>     /* File: pbv-pointer.cpp */
using namespace std;

void swap(int* x, int* y)
{
    cout << "x = " << x << "\t*x = " << *x << endl;
    cout << "y = " << y << "\t*y = " << *y << endl << endl;

    int temp = *x; *x = *y; *y = temp;

    cout << "x = " << x << "\t*x = " << *x << endl;
    cout << "y = " << y << "\t*y = " << *y << endl << endl;
}

int main()
{
    int a = 10, b = 20;
    cout << "a = " << a << "\t\t\t&a = " << &a << endl;
    cout << "b = " << b << "\t\t\t&b = " << &b << endl << endl;

    swap(&a, &b);
    cout << "a = " << a << "\t\t\tb = " << b << endl;
    return 0;
}
```

12.2.7 Common Uses of Pointer

- Indirect addressing (c.f. anchor text).
- Dynamic object creation/deletion.
- Advanced uses that will not be covered in this course:
  - writing generic functions that can work on any data type (e.g., a sorting function that sorts any data type)
  - implementation of object-oriented technologies such as inheritance
  - polymorphism (virtual function)

12.3 Pointer to Structure

12.3.1 Pointer to struct and the -> Operator

File: point.h — the `Point` struct with two `double` members.

```
/* File: point.h */

struct Point
{
    double x;
    double y;
};
```

You may also define a pointer variable for a struct object. Two ways to access struct members through a pointer:

1. Dereference the pointer and use the `.` operator.

```
Point a;          // a contains garbage
Point* ap = &a;   // Now ap points to a
// Dereference ap, then use the . operator
(*ap).x = 3.5;
(*ap).y = 9.7;
```

2. Directly use the `->` operator.

```
Point a;          // a contains garbage
Point* ap = &a;   // Now ap points to a
// No dereferencing when using the -> operator
ap->x = 3.5;
ap->y = 9.7;
```

Example: File: point-test.cpp — reads 3 points and prints the lengths of the 3 sides of a triangle by calling `print_distance` with the addresses of the points.

```
#include <iostream>     /* File: point-test.cpp */
#include "point.h"
using namespace std;

// To compute and print the Euclidean distance between 2 points
void print_distance(const Point*, const Point*);

int main()   /* To find the length of the sides of a triangle */
{
    Point a, b, c;
    cout << "Enter the co-ordinates of point A: "; cin >> a.x >> a.y;
    cout << "Enter the co-ordinates of point B: "; cin >> b.x >> b.y;
    cout << "Enter the co-ordinates of point C: "; cin >> c.x >> c.y;

    print_distance(&a, &b);
    print_distance(&b, &c);
    print_distance(&c, &a);
```

```
    return 0;
}
/* g++ -o point-test point-test.cpp point-distance.cpp */
```

File: point-distance.cpp — implements `euclidean_distance`, `print_point` and `print_distance`; members are accessed with `->` through `const Point*` parameters.

```
#include <iostream>     /* File: point-distance.cpp */
#include <cmath>
#include "point.h"
using namespace std;

double euclidean_distance(const Point* p1, const Point* p2)
{
    double x_diff = p1->x - p2->x, y_diff = p1->y - p2->y;
    return sqrt(x_diff*x_diff + y_diff*y_diff);
}

void print_point(const Point* p)
{
    cout << '(' << p->x << ", " << p->y << ')';
}

void print_distance(const Point* p1, const Point* p2)
{
    cout << "Distance between "; print_point(p1);
    cout << " and "; print_point(p2);
    cout << " is " << euclidean_distance(p1, p2) << endl;
}
```

12.3.2 Student Records: Manipulation and Sorting via Pointers

File: student-record.h — defines `enum Dept`, `struct Date`, `struct Student_Record` and the global department-name table.

```
/* File: student-record.h */
enum Dept { CSE, ECE, MATH };

struct Date
{
    unsigned int year;
    unsigned int month;
    unsigned int day;
};

struct Student_Record
{
    char name[32];
    unsigned int id;
    char gender;
    Dept dept;
    Date entry;
};
// Global constants for department names
const char dept_name[ ][30]
  = {"Computer Science", "Electrical Engineering", "Mathematics"};
```

File: sort-student-record.cpp — creates an array of 3 student records, modifies the 3rd one via `set_student_record`, sorts them by id and prints them.

```
#include "student-record.h" /* File: sort-student-record.cpp */
#include "student-record-extern.h"
```

```
int main()
{
    Student_Record sr[] = {
        { "Adam", 12000, 'M', CSE, { 2006 , 1 , 10 } },
        { "Bob", 11000, 'M', MATH, { 2005 , 9 , 1 } },
        { "Cathy", 10000, 'F', ECE, { 2006 , 8 , 20 } } };

    Date d;  // Modify the 3rd record
    set_date(&d, 1980, 12, 25);
    set_student_record(&sr[2], "Jane", 18000, 'F', CSE, &d);

    sort_3SR_by_id(sr);
    for (int j = 0; j < sizeof(sr)/sizeof(Student_Record); j++)
        print_student_record(&sr[j]);
    return 0;
}
/* g++ -o sort-sr sort-student-record.cpp student-record-functions.cpp
   student-record-swap.cpp */
```

File: student-record-swap.cpp — `swap_SR` swaps two records through pointers; `sort_3SR_by_id` sorts 3 records by id with 3 conditional swaps.

```
#include "student-record.h" /* File: student-record-swap.cpp */

void swap_SR(Student_Record* x, Student_Record* y)
{
    Student_Record temp = *x;
    *x = *y;
    *y = temp;
}

void sort_3SR_by_id(Student_Record sr[])
{
    if (sr[0].id > sr[1].id) swap_SR(&sr[0], &sr[1]);
    if (sr[0].id > sr[2].id) swap_SR(&sr[0], &sr[2]);
    if (sr[1].id > sr[2].id) swap_SR(&sr[1], &sr[2]);
}
```

File: student-record-functions.cpp — print/set helper functions; note `a->entry = *date;` is a struct-struct assignment.

```
#include <iostream> /* File: student-record-functions.cpp */
#include "student-record.h"
using namespace std;

void print_date(const Date* date)
{
    cout << date->year << '/'
         << date->month << '/'
         << date->day << endl;
}

void print_student_record(const Student_Record* x)
{
    cout << endl;
    cout.width(12); cout << "name: "   << x->name << endl;
    cout.width(12); cout << "id: "     << x->id << endl;
    cout.width(12); cout << "gender: " << x->gender << endl;
    cout.width(12); cout << "dept: "   << dept_name[x->dept] << endl;
    cout.width(12); cout << "entry date: "; print_date(&x->entry);
}

void set_date(Date* x, unsigned int year,
              unsigned int month, unsigned int day)
```

```
{
    x->year = year;
    x->month = month;
    x->day = day;
}

void set_student_record(Student_Record* a, const char name[],
                        unsigned int id, char gender, Dept dept,
                        const Date* date)
{
    strcpy(a->name, name);
    a->id = id;
    a->gender = gender;
    a->dept = dept;
    a->entry = *date;   // struct-struct assignment
}
```

File: student-record-extern.h — declarations of all the student-record functions shared between files.

```
/* File: student-record-extern.h */

void print_date(const Date*);
void print_student_record(const Student_Record*);

void set_date(Date* x, unsigned int, unsigned int, unsigned int);
void set_student_record(Student_Record*, const char[],
                        unsigned int, char, Dept, const Date*);

void swap_SR(Student_Record*, Student_Record*);
void sort_3SR_by_id(Student_Record sr[]);
```

12.4 Dynamic Memory/Objects Allocation and Deallocation

12.4.1 Static Objects and Dynamic Objects

```
float a = 2.3;              // Global float variable
int main()
{
    int x = 5;              // Local int variable
    char s[16] = "hkust";   // Local char array
    return 0;
}
```

- Up to now, all (local and global) variables you use require static memory allocation: their memory are allocated by the compiler during compilation.
- When these variables — static objects — go out of their scope, their memory are released automatically back to the computer's memory store (RAM).
- Question: What if you want to create an object, or an array whose size is unknown until a user specifies at runtime?
- C++ allows you to create an object, or an array of objects — dynamic objects — on-the-fly at runtime.
- The memory of dynamic objects
  - has to be allocated at runtime explicitly by you, ⇒ using the operator `new`.
  - will persist even after the object goes out of scope.
  - has to be deallocated at runtime explicitly by you, ⇒ using the operator `delete`.
- Static objects are managed using a data structure called **stack**.
- Dynamic objects are managed using a data structure called **heap**.

A C++ program's memory consists of: program codes, (global) static data, the stack, and the heap.

12.4.2 Dynamic Memory Allocation: Operator `new`

Syntax: dynamic memory allocation using `new`.

```
<type>* <pointer-variable> = new <type>;
```

```
int* ip = new int;
*ip = 5;
Date d19970701 = { 1997, 7 , 1 };
Student_Record* srp = new Student_Record;
set_student_record(srp, "Chris", 100, 'M', CSE, d19970701);
```

For the line `int* ip = new int;`:

- The computer finds from the heap an amount of memory equal to `sizeof(int)` and gives it to your program.
- The `new` operator, which is actually a function, will return a value which is the address of the starting location of that piece of memory.
- That piece of memory is unnamed, and you need to use an `int` pointer variable (here, `ip`) to point to it — holding its address (that is returned by the `new` operator).
- There is no other way to access the unnamed memory allocated by the operator `new` except through the pointers.

For the line `Student_Record* srp = new Student_Record;`:

- The computer gives you an amount of unnamed memory equal to `sizeof(Student_Record)` from the heap.
- You need to hold its address using a `Student_Record` pointer variable (here, `srp`).
- Notice that the variables, `ip` and `srp`, are static objects. Only the unnamed memories returned by the `new` operator are dynamic objects.
- Both local static objects and dynamic objects come and go. However, the stack will allocate and deallocate local static objects automatically for you. But you have to manage the allocation and deallocation of dynamic objects yourselves.

12.4.3 Dynamic Memory Deallocation: Operator `delete`

Syntax: dynamic memory deallocation using `delete`.

```
delete <pointer-variable>;
```

```
delete ip;      // ip is now a dangling pointer
ip = nullptr;   // ip is now a null pointer
delete srp;     // srp is now a dangling pointer
srp = nullptr;  // srp is now a null pointer
```

12.4.4 Common Bug I: Dangling Pointer

- Operator `delete` releases memory pointed to by a pointer variable (here, `ip` or `srp`) back to the heap for recycle.
- However, after the `delete` operation, the pointer variable still holds the address of the previously allocated unnamed memory. Now the pointer becomes a **dangling pointer**.
- A dangling pointer is a pointer that points to a location whose memory is deallocated.
- Analogy: you keep the old address of a friend who has moved to a new house.
- Runtime error usually occurs when you try to dereference a dangling pointer either because
  - the memory is no long accessible as it is taken back.
  - the memory has already been recycled and is re-allocated to some other functions or even other programs!
- Modifying the object a dangling pointer points to leads to unpredictable results that usually end up in a program crash.

- To play safe, reset a dangling pointer to a null pointer by setting its value to `nullptr`.
- `nullptr` is a new keyword in C++11 and is used to indicate a pointer that has not been set to point to something useful. In the past, a null pointer is represented by `NULL` or `0`.

Good Practices with Pointers

1. Always initialize a pointer to `nullptr` when defining a pointer variable.
2. Always check whether a pointer is a `nullptr` before using it.

Example: File: dangling-pointer.cpp — returns the address of a local variable; after the function returns, the returned pointer dangles.

```
int* create_and_init(int value) /* File: dangling-pointer.cpp */
{
    int x = value;       // x is a local variable
    int* p = &x;         // p is also a local variable
    return p;
}

int main()
{
    int* ip = create_and_init(10);
    cout << *ip << endl;
    return 0;
}
```

- Local pointer variable, `p` is pointing to another local variable, `x`. Both are automatically allocated when the function `create_and_init()` is called, and are automatically deallocated when `create_and_init()` returns.
- Question: What does the pointer variable, `ip` point to after the call to `create_and_init()` returns?

Never Return the Address of a Local Variable

Local variables are deallocated when the function returns, so a pointer to them becomes a dangling pointer. Dereferencing it leads to unpredictable results or a crash.

12.4.5 Common Bug II: Memory Leak

- Memory leak occurs when dynamically allocated memory that is no longer needed is not released.
- Since the memory allocated by operator `new` is unnamed, always keep track of it using a pointer variable.
- If you lose track of it, it will become inaccessible and there will be memory leak.
- When you leak a lot of memory, then the computer does not have enough memory to run your program ⇒ runtime error.
- Analogy: your home is full of old stuffs that you don't need anymore. Thus, your room is running out of space. You should properly recycle them.

```
int* ip = new int;  // First unnnamed int
*ip = 5;
ip = new int;       // Last unnamed int is lost
*ip = 8;
```

Example:

```
void swap(Date& x, Date& y)
{
    Date* temp = new Date; *temp = x; x = y; y = *temp;
}
int main()
```

```
{
    Date a = { 2006 , 1 , 10 }; Date b = { 2005 , 9 , 1 };
    swap(a, b); return 0;
}
```

- The variable, `Date* temp` is a local variable in the function `swap()`.
- Everytime when `swap()` is called, `temp` is automatically allocated on a stack.
- `new Date` returns an unnamed memory of size equal to `sizeof(Date)` from the heap.
- When `swap()` returns,
  - the memory for local variables like `temp` will be deallocated automatically.
  - However, the memory allocated by operator `new` remains until operator `delete` is used to deallocate it.
  - the whole program finishes, the operating system will take back all memory dynamically allocated by the program that has not been deleted.
- Question: What happens to the unnamed memory returned by `new Date` when `swap()` returns back to `main()`? (It leaks — nobody holds its address anymore.)

12.5 A Dynamic Data Structure: Linked List

12.5.1 What is a Linked List?

- A list is a linear sequence of objects.
- You may implement a list by an array, e.g. `int x[5];`
  - Advantage: array is an efficient data structure that works well with loops and recursion.
  - Disadvantage: size of the array is determined in advance.
- A **linked list** links objects together by pointers so that each object is pointing to the next object in the sequence (list).
  - Advantage: It is dynamic; it grows and shrinks to any size as you want at runtime.
  - Disadvantage: requires additional memory for the linking pointers; takes more time to manipulate its items.
- Each object in a linked list is usually called a "node". The typical C++ definition for a node in a linked list is a struct (or later class):

```
struct ll_node          // C++11 allows default values for members
{
    <type> data;        // contains useful information
    ll_node* next;      // the link to the next node
};
```

- The first and the last node of a linked list always need special attention.
- For the last node, its `next` pointer is set to `nullptr` to tell that it is the end of the linked list.
- We need a pointer variable, usually called `head` to point to the first node.
- Once you get the `head` of the linked list, you get the whole list!

The head Pointer Represents the Whole List

A linked list is accessed through a single pointer `head` to its first node; the last node has `next == nullptr`. Once you get the head of the linked list, you get the whole list.

12.5.2 Basic Operations on a Linked List

```
/* To create a node */
ll_node* p = new ll_node;

/* To access/modify the data in a node */
```

```
cout << p->data;
cout << (*p).data;

cin >> p->data;
p->next = nullptr;

/* To set up the head of a linked list */
ll_node* head = nullptr;    // An empty linked list
head = p;                   // head points to the node that p points \
   to

/* To delete a node */
delete p;                   // Dangling pointer
p = nullptr;                // Reset the pointer for safety reason
```

Common operations:

- Create a new linked list.
- Search data in the list.
- Delete a node in the list.
- Insert a new node in the list.

For all these operations, again special attention is usually needed when the operation involves the first or the last node.

12.5.3 LL-String: Creation, Length, Print and Search

Let's use a linked list (instead of an array) of characters to represent a string.

File: ll_cnode.h — defines the `ll_cnode` struct (a char node) and declares all linked-list operations.

```
#include <iostream> /* File: ll_cnode.h */
using namespace std;

struct ll_cnode // C++11 allows default values for members
{
    char data;          // Contains useful information
    ll_cnode* next = nullptr; // Link to the next node
};

const char NULL_CHAR = '\0';
ll_cnode* ll_create(char);
ll_cnode* ll_create(const char []);
int ll_length(const ll_cnode*);
void ll_print(const ll_cnode*);
ll_cnode* ll_search(ll_cnode*, char c);
void ll_insert(ll_cnode*&, char, unsigned);
void ll_delete(ll_cnode*&, char);
void ll_delete_all(ll_cnode*&);
```

File: ll_main.cpp — creates 3 nodes by hand, hooks them up into the list "met", traverses and prints it, then cleans up.

```
#include "ll_cnode.h"   /* File: ll_main.cpp */

int main()  // Create the LL-string "met"
{   // Create each of the 3 ll_cnodes
    ll_cnode* mp = new ll_cnode; mp->data = 'm';
    ll_cnode* ep = new ll_cnode; ep->data = 'e';
    ll_cnode* tp = new ll_cnode; tp->data = 't';

    // Hook them up in the required order to create the LL
    mp->next = ep;
    ep->next = tp;
    tp->next = nullptr;
```

```
    // Traverse the LL and print out the data sequentially
    for (ll_cnode* p = mp; p; p = p->next)
        cout << p->data;
    cout << endl;

    // Clean up
    delete mp; delete ep; delete tp; return 0;
}
```

File: ll_create.cpp — two overloaded `ll_create` functions: one creates a single node; the other builds a whole linked list from a C string (with special handling of the head).

```
#include "ll_cnode.h"   /* File: ll_create.cpp */
// Create a ll_cnode and initialize its data
ll_cnode* ll_create(char c)
{
    ll_cnode* p = new ll_cnode; p->data = c; return p;
}

// Create a linked list of ll_cnodes with the contents of a char array
ll_cnode* ll_create(const char s[])
{
    if (s[0] == NULL_CHAR) // Empty linked list due to empty C string
        return nullptr;

    ll_cnode* head = ll_create(s[0]); // Special case with the head

    ll_cnode* p = head; // p is the working pointer
    for (int j = 1; s[j] != NULL_CHAR; ++j)
    {
        p->next = ll_create(s[j]); // Link current cnode to the new \
   cnode
        p = p->next; // p now points to the new ll_cnode
    }

    return head; // The WHOLE linked list can be accessed from the \
   head
}
```

File: ll_length.cpp — counts the nodes by traversing the list with a working pointer.

```
#include "ll_cnode.h" /* File: ll_length.cpp */

int ll_length(const ll_cnode* head)
{
    int length = 0;
    for (const ll_cnode* p = head; p != nullptr; p = p->next)
        ++length;
    return length;
}
```

File: ll_print.cpp — traverses the list and prints each node's data.

```
#include "ll_cnode.h"   /* File: ll_print.cpp */

void ll_print(const ll_cnode* head)
{
    for (const ll_cnode* p = head; p != nullptr; p = p->next)
        cout << p->data;
    cout << endl;
}
```

File: ll_search.cpp — linear search for a character; returns a non-const pointer so the caller may modify the found node.

```
#include "ll_cnode.h"   /* File: ll_search.cpp */

// The returned pointer may be used to change the content
// of the found ll_cnode. Therefore, the return type
// should not be const ll_cnode*.

ll_cnode* ll_search(ll_cnode* head, char c)
{
    for (ll_cnode* p = head; p != nullptr; p = p->next)
    {
        if (p->data == c)
            return p;
    }

    return nullptr;
}
```

12.5.4 LL-String: Insertion and Deletion

To insert a new character so that it becomes the n-th character (counted from zero):

1. Create the new node.
2. Find the node after which the new node is to be added.
3. Set the new node's `next` to the found node's `next`.
4. Set the found node's `next` to the new node.

Special case: insert at the beginning (`n == 0` or the list is empty).

File: ll_insert.cpp — insertion into the LL-string; appends to the end if `n` exceeds the current length.

```
#include "ll_cnode.h"   /* File: ll_insert.cpp */

// To insert character c to the linked list so that after insertion,
// c is the n-th character (counted from zero) in the list.
// If n > current length, append to the end of the list.

void ll_insert(ll_cnode*& head, char c, unsigned n)
{
    // STEP 1: Create the new ll_cnode
    ll_cnode* new_cnode = ll_create(c);

    // Special case: insert at the beginning
    if (n == 0 || head == nullptr)
    {
        new_cnode->next = head;
        head = new_cnode;
        return;
    }

    // STEP 2: Find the node after which the new node is to be added
    ll_cnode* p = head;
    for (int position = 0;
         position < n-1  &&  p->next != nullptr;
         p = p->next, ++position)
         ;

    // STEP 3,4: Insert the new node between
    //           the found node and the next node
    new_cnode->next = p->next; // STEP 3
    p->next = new_cnode;       // STEP 4
}
```

Note that `head` is passed by reference (`ll_cnode*&`) because inserting at the beginning changes the head itself.

To delete a character from the list:

1. Find the item to be deleted (advancing `prev` and `current` together).
2. Bypass the found item; special case: deleting the first item updates `head`.
3. Free up the memory of the deleted item with `delete`.

File: ll_delete.cpp — deletes the first occurrence of a character; does nothing if it is not found.

```
#include "ll_cnode.h"   /* File: ll_delete.cpp */
// To delete the character c from the linked list.
// Do nothing if the character cannot be found.
void ll_delete(ll_cnode*& head, char c)
{
    ll_cnode* prev = nullptr; // Point to previous ll_cnode
    ll_cnode* current = head; // Point to current ll_cnode

    // STEP 1: Find the item to be deleted
    while (current != nullptr && current->data != c)
    {
        prev = current;      // Advance both pointers
        current = current->next;
    }

    if (current != nullptr)  // Data is found
    {   // STEP 2: Bypass the found item
        if (current == head) // Special case: delete the first item
            head = head->next;
        else
            prev->next = current->next;

        delete current;      // STEP 3: Free up the memory of the \
   deleted item
    }
}
```

File: ll_delete_all.cpp — deletes the whole linked list by recursion: first delete the remaining nodes, then delete the current node and reset `head` to `nullptr`.

```
#include "ll_cnode.h"   /* File: ll_delete_all.cpp */

// To delete the WHOLE linked list, given its head by recursion.
void ll_delete_all(ll_cnode*& head)
{
    if (head == nullptr) // An empty list; nothing to delete
        return;

    // STEP 1: First delete the remaining nodes
    ll_delete_all(head->next);

    // For debugging: this shows you what are deleting
    cout << "deleting " << head->data << endl;

    delete head;         // STEP 2: Then delete the current nodes
    head = nullptr;      // STEP 3: To play safe, reset head to nullptr
}
```

File: ll_test.cpp — a test driver exercising create, length, print, search, insert, delete and delete_all.

```
#include "ll_cnode.h"   /* File: ll_test.cpp */
int main()
{
    ll_cnode* ll_string = ll_create("met");
    cout << "length of ll_string = " << ll_length(ll_string) << endl;
    ll_print(ll_string);
```

```
    ll_print(ll_search(ll_string, 'e'));

    cout << endl << "After inserting 'a'" << endl;
    ll_insert(ll_string, 'a', 2); ll_print(ll_string);
    cout << endl << "After deleting 'e'" << endl;
    ll_delete(ll_string, 'e'); ll_print(ll_string);
    cout << endl << "After deleting 'm'" << endl;
    ll_delete(ll_string, 'm'); ll_print(ll_string);

    cout << endl << "After inserting 'e'" << endl;
    ll_insert(ll_string, 'e', 9); ll_print(ll_string);
    cout << endl << "After deleting 't'" << endl;
    ll_delete(ll_string, 't'); ll_print(ll_string);
    cout << endl << "After deleting 'e'" << endl;
    ll_delete(ll_string, 'e'); ll_print(ll_string);
    cout << endl << "After deleting 'a'" << endl;
    ll_delete(ll_string, 'a'); ll_print(ll_string);

    cout << endl << "After deleting 'z'" << endl;
    ll_delete(ll_string, 'z'); ll_print(ll_string);
    cout << endl << "After inserting 'h'" << endl;
    ll_insert(ll_string, 'h', 9); ll_print(ll_string);
    cout << endl << "After inserting 'o'" << endl;
    ll_insert(ll_string, 'o', 0); ll_print(ll_string);

    ll_delete_all(ll_string);
    return 0;
}
```

12.5.5 Other Common Variants of Linked List

- **Circular linked list**: the `next` pointer of the last node points back to the first node.
- **Doubly linked list**: each node has 2 link pointers — one to the next node and one to the previous node (both ends have `nullptr`).

12.5.6 Another Dynamic Data Structure: Binary Tree

- An important dynamic data structure in CS is tree. CS's tree actually looks like an inverted physical tree.
- In particular, any node of a **binary tree** has 2 sub-trees (children): left sub-tree (child) and right sub-tree (child).
- The topmost node is called the root; nodes whose children are both `nullptr` are the leaves.

  The node definition (`btree.h`) is shown later in Further Reading III.

12.6 Array as a Pointer

12.6.1 Pointer Arithmetic

- A pointer variable supports 2 arithmetic operations: `+`, `-`.
- If you have `<type> x; <type>* xp = &x;`, then
  - `xp + N == &x + sizeof(<type>)` × `N`.
  - `xp - N == &x - sizeof(<type>)` × `N`.
- The result of pointer arithmetic should be a valid address, otherwise, dereferencing it may lead to segmentation fault!

File: pointer-math.cpp — pointer arithmetic on a `double` pointer; adding/subtracting N moves the address by $N\times$ `sizeof(double)`. Also shows that dereferencing arbitrary addresses (e.g. around `0x14`) usually leads to runtime error.

```
#include <iostream>     /* File: pointer-math.cpp */
using namespace std;

int main()
{
    double x = 2.3;     // double is 8-byte
    double* xp = &x;    // xp points to x
    cout << &x << endl << xp + 2 << endl << xp - 2 << endl;

    // Nothing disallows you from assigning an integer value
    // to a pointer variable. Hexadecimal numbers start with 0x.
    int* yp = reinterpret_cast<int*>(0x14);
    cout << yp + 1 << endl << yp - 1 << endl;

    // Since addresses around 0x14 may not be accessible to you
    // Dereferencing them usually leads to runtime error
    cout << *(yp + 1) << endl << *(yp - 1) << endl;
    return 0;
}
```

12.6.2 The Duality of Array and Pointer

- Just like that the `head` of a linked list is a pointer to the first element of the list, the array identifier can also be interpreted as a pointer to the first array element.
- In fact, the array identifier can be treated as a const pointer.
- Thus, the variable `x` in `int x[3];` from the pointer perspective, is like `int* const`.

Any pointer pointing to an array can be used to access all elements of the array instead of the original array identifier.

File: array-by-another-pointer.cpp — modifies the array through pointer `y` using array syntax `y[j]`, then prints it through `x`.

```
#include <iostream>     /* File: array-by-another-pointer.cpp */
using namespace std;

int main()
{
    int x[] = { 11, 22, 33, 44 };
    int* y = x; // Both y and x point to the 1st element of array

    // Modify the array through pointer y
    for (int j = 0; j < sizeof(x)/sizeof(int); ++j)
        y[j] += 100;

    // Print the array through pointer x
    for (int j = 0; j < sizeof(x)/sizeof(int); ++j)
        cout << x[j] << endl;
    return 0;
}
```

- Using pointer arithmetic, you may "move" a pointer to point to any array element.
- Dereferencing a pointer to an array element then obtains the element — and you can use it as either lvalue or rvalue.
- Again, if `int x[] = {11,22,33}; int* xp = x;`, then we have:

Element Address	Element Value
`xp == x == &x[0]`	`*xp == *x == x[0] == 11`
`xp+1 == x+1 == &x[1]`	`*(xp+1) == *(x+1) == x[1] == 22`
`xp+2 == x+2 == &x[2]`	`*(xp+2) == *(x+2) == x[2] == 33`

And by definition, numerically, we have `&x == x == &x[0]`.

File: print-array-by-pointer.cpp — prints an `int` array by advancing a pointer through it.

```
#include <iostream> /* File: print-array-by-pointer.cpp */
using namespace std;
int main()
{
    int x[] = { 11, 22, 33, 44 };
    for (int* xp = x, j = 0; j < sizeof(x)/sizeof(int); ++j, ++xp)
        cout << *xp << endl;
    return 0;
}
```

File: print-char-array-by-pointer.cpp — prints a C string by advancing a pointer until the null character `'\0'` is met.

```
#include <iostream> /* File: print-char-array-by-pointer.cpp */
using namespace std;
int main()
{
    char s[] = "hkust";
    for (const char* sp = s; *sp != '\0'; ++sp)
        cout << *sp << endl;
    return 0;
}
```

12.6.3 Creation and Destruction of a Dynamic Array

Syntax: new a dynamic array.

```
<type>* <pointer-variable> =
    new <type> [<integer-expression>];

int array_size; cin >> array_size;  // Unknown till runtime
int* x = new int [array_size];
for (int j = 0; j < array_size; ++j)
    x[j] = j;   // Actually a pointer but treated like an array
```

Syntax: delete a dynamic array.

```
delete [] <pointer-variable>;

delete [] x;    // x is now a dangling pointer
x = nullptr;    // x is now a nullptr pointer
```

delete[] for Dynamic Arrays

Memory allocated by `new <type>[n]` must be released by `delete []`, not plain `delete`. After deletion the pointer dangles; reset it to `nullptr`.

Example: File: dynamic-point-array.cpp — creates a dynamic array of `Point` whose size is read at runtime, computes pairwise distances, then deallocates it.

```
#include <iostream>     /* File: dynamic-point-array.cpp */
#include "point.h"
using namespace std;

int main()
{
    void print_distance(const Point*, const Point*);
    int num_points;
    cout << "Enter the number of points : "; cin >> num_points;
```

```
    Point* point = new Point [num_points]; // Dynamic array of points

    for (int j = 0; j < num_points; ++j)   // Input the points
    {
        cout << "Enter the x & y coordinates of point #" << j << " : \
   ";
        cin >> point[j].x >> point[j].y;
    }

    for (int i = 0; i < num_points; ++i) // Compute distance between \
   2 points
        for (int j = i+1; j < num_points; ++j)
            print_distance(point+i, point+j);

    delete [] point;   // Deallocate the dynamic array of points
    return 0;
} /* g++ dynamic-point-array.cpp point-distance.cpp */
```

This program is compiled together with `point-distance.cpp` (shown earlier); note `print_distance(point+i, point+j)` passes element addresses via pointer arithmetic.

12.6.4 C String as a Char Pointer

File: palindrome.cpp — recursive palindrome check on a C string using two `const char*` pointers that move towards each other.

```
#include <iostream>     /* File: palindrome.cpp */
using namespace std;

bool palindrome(const char* first, const char* last)
{
    if (first >= last)
        return true;
    else if (*first != *last)
        return false;
    else
        return palindrome(first+1, last-1);
}

int main() {
    const int MAX_LINE_LEN = 255;
    char s[MAX_LINE_LEN+1];
    while (cin.getline(s, MAX_LINE_LEN+1, '\n')// Input ends in \
   newline
        || cin.gcount() == MAX_LINE_LEN)  // Input ends with max \
   #chars
        cout << boolalpha << palindrome(s, s+strlen(s)-1) << endl;
    return 0;
}
```

File: const-char-pointer.cpp — a string literal should be held by a `const char*`; the same brace-literal style is illegal for other pointer types.

```
#include <iostream>     /* File: const-char-pointer.cpp */
using namespace std;

int main()
{
    const char* s1 = "creative"; // Correct way!
    char* s2 = "smart";          // Error if g++ -pedantic-errors

    /* This only works for char pointer.
     * E.g., the following are illegal
```

```
     *     const int* x1 = {1, 2, 3};
     *     int* x2 = {1, 2, 3};
     */
    cout << s1 << endl;
    cout << s2 << endl;
    return 0;
}
```

12.7 Multi-dimensional Array and Pointer

12.7.1 Dynamic 2D Array: Allocation and Operations

To create a 2D `int` array with M rows and N columns at runtime:

1. Allocate a 1D array of M `int*` (int pointers).
2. For each of the M elements, create another 1D array of N `int` (integers), and set the former to point to the latter.

The result is held by an `int**` variable `x`, where `x[j]` points to the j-th row.

Question: Can you generalize this to 3D, 4D, ..., arrays?

Example: File: 2d-dynamic-array-main.cpp — reads the numbers of rows and columns, creates a dynamic 2D array, fills and prints it, deletes it and resets the pointer.

```
#include <iostream>     /* File: 2d-dynamic-array-main.cpp */
using namespace std;

int** create_matrix(int, int);
void print_matrix(const int* const*, int, int);
void delete_matrix(const int*const *, int, int);

int main()
{
    int num_rows, num_columns;
    cout << "Enter #rows followed by #columns: ";
    cin >> num_rows >> num_columns;
    int** matrix = create_matrix(num_rows, num_columns);

    // Dynamic array elements can be accessed like static array \
   elements
    for (int j = 0; j < num_rows; ++j)
        for (int k = 0; k < num_columns; ++k)
            matrix[j][k] = 10*(j+1) + (k+1);

    print_matrix(matrix, num_rows, num_columns);
    delete_matrix(matrix, num_rows, num_columns);
    matrix = nullptr;      // Avoid dangling pointer
    return 0;
} /* g++ 2d-dynamic-array-main.cpp 2d-dynamic-array-functions.cpp */
```

File: 2d-dynamic-array-functions.cpp — `create_matrix` allocates the row-pointer array first then each row; `delete_matrix` deallocates in the reverse order.

```
#include <iostream>     /* File: 2d-dynamic-array-functions.cpp */
using namespace std;

int** create_matrix(int num_rows, int num_columns) {
    int** x = new int* [num_rows];     // STEP 1
    for (int j = 0; j < num_rows; ++j) // STEP 2
        x[j] = new int [num_columns];
    return x;
}
```

```
void print_matrix(const int* const* x, int num_rows, int \
   num_columns) {
    for (int j = 0; j < num_rows; ++j)
    {
        for (int k = 0; k < num_columns; ++k)
            cout << x[j][k] << '\t';
        cout << endl;
    }
}

void delete_matrix(const int*const * x, int num_rows, int \
   num_columns) {
    for (int j = 0; j < num_rows; ++j) // Delete is done in reverse \
   order
        delete [] x[j];                // (compared with its creation)
    delete [] x;
}
```

Create Forward and Delete Backward

A dynamic 2D array is created in two steps — first the array of row pointers, then each row — and must be deleted in the reverse order: each row first, then the row-pointer array.

12.8 Further Reading 0: Relation between Dynamic 2D Array & Pointer

File: 2d-dynamic-array-and-pointer.cpp — prints the addresses involved in a dynamic 3×4 array to reveal the relation between `x`, `x[j]` and the rows.

```
#include <iostream>     /* File: 2d-dynamic-array-and-pointer.cpp */
using namespace std;

int main()
{
    // Dynamically create an array with 3 rows, 4 columns
    int** x = new int* [3];     // STEP 1
    for (int j = 0; j < 3; j++) // STEP 2
        x[j] = new int [4];

    cout << endl << "Info about x:" << endl;
    cout << "sizeof(x) :\t" << sizeof(x) << endl << endl;
    cout << "x\t\t"    << "&x[0]\t\t"    << "&x[0][0]" << endl;
    cout << x << '\t'  << &x[0]  << '\t' << &x[0][0] << endl << endl;
    cout << "&x[j]\t\t"          << "x[j]\t\t"
         << "&x[j][0]" << '\t'   << "x+j" << endl;

    for (int j = 0; j < 3; j++)
        cout << &x[j]    << '\t' << x[j] << '\t'
             << &x[j][0] << '\t' << x+j  << endl;
    return 0;
}
```

Sample output:

```
Info about x:
sizeof(x) :     8

x               &x[0]           &x[0][0]
0x14ea5010      0x14ea5010      0x14ea5030

&x[j]           x[j]            &x[j][0]        x+j
0x14ea5010      0x14ea5030      0x14ea5030      0x14ea5010
```

```
0x14ea5018      0x14ea5050      0x14ea5050      0x14ea5018
0x14ea5020      0x14ea5070      0x14ea5070      0x14ea5020
```

Notice that, numerically, we have

- `x == &x[0] != &x[0][0]` ⇒ `x` points to `x[0]` (and not `x[0][0]` as in static 2D array).
- `&x[j] == x+j` ⇒ a proof of the pointer arithmetic.
- `x[j] == &x[j][0]` ⇒ `x[j]` points to the first element of the j-th row.

12.9 Further Reading I: Arguments for the `main()` Function

12.9.1 `main()` Function Arguments

- Up to now, you write the main function header as `int main()` or `int main(void)`.
- In fact, the general form of the main function allows variable number of arguments (overloaded function):

```
int main(int argc, char** argv)
int main(int argc, char* argv[])
```

- `argc` gives the actual number of arguments.
- `argv` is an array of `char*`, each pointing to a character string.
- e.g. `g++ -o a.out a.cpp` calls the main function of the `g++` program with 3 additional commandline arguments. Thus, `argc = 4`, and `argv[0] = "g++"`, `argv[1] = "-o"`, `argv[2] = "a.out"`, `argv[3] = "a.cpp"`.

Example: File: 2d-dynamic-array-main-with-argv.cpp — same matrix program but the numbers of rows and columns come from the command line via `atoi(argv[1])` and `atoi(argv[2])`.

```
#include <iostream> /* File: 2d-dynamic-array-main-with-argv.cpp */
using namespace std;
int** create_matrix(int, int);
void print_matrix(const int* const*, int, int);
void delete_matrix(int**, int, int);

int main(int argc, char** argv)
{
    if (argc != 3)
    { cerr << "Usage: " << argv[0] << " #rows #columns" << endl; \
   return -1; }

    int num_rows = atoi(argv[1]);
    int num_columns = atoi(argv[2]);
    int** matrix = create_matrix(num_rows, num_columns);

    // Dynamic array elements can be accessed like static array \
   elements
    for (int j = 0; j < num_rows; ++j)
        for (int k = 0; k < num_columns; ++k)
            matrix[j][k] = 10*(j+1) + (k+1);

    print_matrix(matrix, num_rows, num_columns);
    delete_matrix(matrix, num_rows, num_columns);
    matrix = nullptr; // Avoid dangling pointer
    return 0;
} /* g++ 2d-dynamic-array-main-with-argv.cpp \
   2d-dynamic-array-functions.cpp */
```

12.10 Further Reading II: Array of Pointers to Structures

12.10.1 Array of Pointers to Structures

- You may create an array of basic data types as well as user-defined data types, or pointers to them.
- Thus, you may have an array of struct objects (`Student_Record sr[3];`), or an array of pointers to struct objects (`Student_Record* srp[3];` where each element holds e.g. `&sr[0]`).

The earlier `sort-student-record.cpp` (shown in Part III) sorts by swapping the struct objects themselves.

12.10.2 Advantage of Indirect Addressing

- During a sorting procedure, in general, many array items are swapped.
- When 2 items are swapped, 3 copy actions are required.
- When the array items are big — say, 1MB — objects, the copying actions may take substantial amount of computation and time.
- A common solution is to make use of indirect addressing and to sort using the pointers to the objects instead.
- The size of pointers is fixed, independent of the objects they point to. For a 32-bit CPU, it is 4 bytes; for a 64-bit CPU, it is 8 bytes.
- When 2 items are sorted and swapped by their pointers, the 3 copy actions involve only copying 4-byte pointers (for 32-bit CPU and 8-byte pointers for 64-bit CPU) which are independent of the size of items they point to.

12.10.3 Sorting by Pointers to Struct Objects

File: sort-student-record-ptr.cpp — builds an array of pointers to the records and sorts the pointers instead of the records.

```
#include "student-record.h" /* File: sort-student-record-ptr.cpp */
void swap_SR_ptr(Student_Record*&, Student_Record*&);
void print_student_record(const Student_Record*);

void sort_3SR_by_id_by_ptr(Student_Record* srp[])
{
    if (srp[0]->id > srp[1]->id) swap_SR_ptr(srp[0], srp[1]);
    if (srp[0]->id > srp[2]->id) swap_SR_ptr(srp[0], srp[2]);
    if (srp[1]->id > srp[2]->id) swap_SR_ptr(srp[1], srp[2]);
}

int main()
{
    Student_Record sr[] = {
        { "Adam", 12000, 'M', CSE, { 2006 , 1 , 10 } },
        { "Bob", 11000, 'M', MATH, { 2005 , 9 , 1 } },
        { "Cathy", 10000, 'F', ECE, { 2009 , 6 , 20 } } };

    Student_Record* srp[] = { &sr[0], &sr[1], &sr[2] }; // Array of \
   pointers
    sort_3SR_by_id_by_ptr(srp);

    for (int j = 0; j < sizeof(srp)/sizeof(Student_Record*); ++j)
        print_student_record(srp[j]);
    return 0;
} /* g++ sort-student-record-ptr.cpp \
   student-record-ptr-functions.cpp */
```

File: student-record-ptr-functions.cpp — `swap_SR_ptr` swaps two pointers (passed by reference); the records themselves never move.

```
#include <iostream> /* File: student-record-ptr-functions.cpp */
#include "student-record.h"
using namespace std;

// Swap 2 Student_Record's by their pointers
void swap_SR_ptr(Student_Record*& srp1, Student_Record*& srp2)
{
    Student_Record* temp = srp1; srp1 = srp2; srp2 = temp;
}

void print_date(const Date* date)
{
    cout << date->year << '/' << date->month << '/' << date->day << \
   endl;
}

void print_student_record(const Student_Record* x)
{
    cout << endl;
    cout.width(12); cout << "name: "   << x->name << endl;
    cout.width(12); cout << "id: "     << x->id << endl;
    cout.width(12); cout << "gender: " << x->gender << endl;
    cout.width(12); cout << "dept: "   << dept_name[x->dept] << endl;
    cout.width(12); cout << "entry date: "; print_date(&x->entry);
}
```

Sort by Pointers Instead of Objects

Swapping two large objects costs 3 copies of the whole object; swapping their pointers costs only 3 pointer copies (4 or 8 bytes each), independent of the object size. The actual objects never move — only the pointers move.

12.10.4 Another Way of Implementing Pointer by Index

- The principle of "sort-by-pointers" is that the actual objects in an array do not move. Instead, their pointers move to indicate their positions during and after sorting.
- Before we have C++ pointers, one may implement the same concept by using a separate array of object indices.
- In a similar fashion, one sort the actual objects by manipulating their indices (which are conceptually equivalent to the pointers).

File: sort-student-record-by-index.cpp — sorts the records by manipulating an array of indices instead of pointers.

```
#include "student-record.h" /* File: \
   sort-student-record-by-index.cpp */
void swap_SR_index(int&, int&);
void print_student_record(const Student_Record&);

void sort_3SR_by_id_by_index(const Student_Record sr[], int index[])
{
    if (sr[index[0]].id > sr[index[1]].id) swap_SR_index(index[0], \
   index[1]);
    if (sr[index[0]].id > sr[index[2]].id) swap_SR_index(index[0], \
   index[2]);
    if (sr[index[1]].id > sr[index[2]].id) swap_SR_index(index[1], \
   index[2]);
}
```

```
int main()
{
    Student_Record sr[] = {
        { "Adam", 12000, 'M', CSE, { 2006 , 1 , 10 } },
        { "Bob", 11000, 'M', MATH, { 2005 , 9 , 1 } },
        { "Cathy", 10000, 'F', ECE, { 2009 , 6 , 20 } } };

    int index[ ] = { 0, 1, 2 }; // Array of indices of student records
     sort_3SR_by_id_by_index(sr, index);

    for (int j = 0; j < sizeof(index)/sizeof(int); ++j)
        print_student_record(sr[index[j]]);
    return 0;
} // g++ sort-student-record-by-index.cpp \
   student-record-by-index-functions.cpp
```

File: student-record-by-index-functions.cpp — the index-based helpers; note that here the records are passed by const reference.

```
#include <iostream> /* File: student-record-by-index-functions.cpp */
#include "student-record.h"
using namespace std;

// Swap 2 Student_Record's by their indices
void swap_SR_index(int& index1, int& index2)
{
    int temp = index1; index1 = index2; index2 = temp;
}

void print_date(const Date& date)
{
    cout << date.year << '/' << date.month << '/' << date.day << endl;
}

void print_student_record(const Student_Record& x)
{
    cout << endl;
    cout.width(12); cout << "name: "   << x.name << endl;
    cout.width(12); cout << "id: "     << x.id << endl;
    cout.width(12); cout << "gender: " << x.gender << endl;
    cout.width(12); cout << "dept: "   << dept_name[x.dept] << endl;
    cout.width(12); cout << "entry date: "; print_date(x.entry);
}
```

12.11 Further Reading III: Dynamic Data Structure: (Binary) Tree

12.11.1 Binary Tree: Definition, Creation, Deletion and Printing

File: btree.h — the `btree_node` struct with two child pointers, plus declarations of the tree functions.

```
#include <iostream>     /* File: btree.h */
using namespace std;

struct btree_node       // A node in a binary tree
{
    int data;
    btree_node* left;   // Left sub-tree or called left child
    btree_node* right;  // Right sub-tree or called right child
};
```

```
// Function declarations
btree_node* create_btree_node(int data);
void delete_btree(btree_node*& tree) ;
void print_btree(const btree_node* tree, int depth = 0);
```

File: btree-main.cpp — builds a small binary tree node by node, prints it, deletes its left sub-tree and prints it again.

```
#include "btree.h"      /* File: btree-main.cpp */

int main()
{
    btree_node* root = create_btree_node(10); // Create root node

    root->left = create_btree_node(8);   // Create the left sub-tree
    root->left->left = create_btree_node(5);
    root->left->right = create_btree_node(9);

    root->right = create_btree_node(15); // Create the right sub-tree
    root->right->left = create_btree_node(12);
    root->right->right = create_btree_node(17);

    print_btree(root);  // Print the resulting tree

    delete_btree(root->left);             // Delete the left sub-tree
    cout << "\n\n\n"; print_btree(root); // Print the resulting tree
    return 0;
}
```

File: btree-create-delete.cpp — `create_btree_node` allocates and initializes a node; `delete_btree` recursively deletes the left and right sub-trees before deleting the root.

```
#include "btree.h"      /* File: btree-create-delete.cpp */

btree_node* create_btree_node(int data)
{
    btree_node* node = new btree_node;
    node->data = data;
    node->left = node->right = nullptr;
    return node;
}

void delete_btree(btree_node*& root) // By recursion
{
    if (root == nullptr) return;     // Base case

    delete_btree(root->left);        // Recursion on the left subtree
    delete_btree(root->right);       // Recursion on the right subtree
    delete root;
    root = nullptr;
}
```

File: btree-print.cpp — prints the tree rotated by 90°: right sub-tree first, then the node indented by its depth, then the left sub-tree.

```
#include "btree.h"      /* File: btree-print.cpp */

void print_btree(const btree_node* root, int depth)
{
    if (root == nullptr)   // Base case
        return;

    print_btree(root->right, depth+1);// Recursion: right subtree
```

```
    for (int j = 0; j < depth; j++)   // Print the node data
        cout << '\t';
    cout << root->data << endl;

    print_btree(root->left, depth+1); // Recursion: left subtree
}
```

13 C++ Class

13.1 What is a C++ Class?

13.1.1 From Scattered Data to C++ Class

- Pieces of information, even belonging to the same object, are scattered around.
- All functions are global and are created to work on data.
- Pieces of information that belong to the same object are collected and wrapped in a `struct`.
- All functions are still global and are created to work on `struct`s.
- Example: 3 student records, each wrapping `name`, `dept` and `id` together.
- Then functions are not global anymore.
- However, the function codes of different objects of the same `struct` are the same.
- Aren't the duplicate functions a waste?
- Factor out the common functions so that the compiler generates only one copy of machine codes for each function.
- But functions are "`struct`-specific" — they can only be called by objects of the intended `struct`. Now you get a class!
- So a class consists of private individual data (one copy per object) plus public common functions (one shared copy of machine code).

13.1.2 C++ Class: Data + Operations + Access Control

- C++ `struct` allows you to create new complex data types consisting of a collection of generally heterogeneous objects.
- However, a basic data type like `int`, besides having a value, also supports a set of operations: `+`, `-`, ×, /, `%`, ≫ (for input), and ≪ (for output).
- `struct` is inherited from the language C, and C++ generalizes the idea to `class`.

Class = Data + Operations + Access Control

Class = data member + member functions + access control. A class allows you to create "smart" objects that support a set of operations (c.f. a remote control). A class is also known as an abstract data type (ADT).

- **Data members**: just like data members in a `struct`.
- **Member functions**: a set of functions that work only for the objects of the class, and can only be called by them.
- In addition, C++ allows access control to each data member and member function:
  - `public`: accessible to any functions (both member functions of the class or other functions).
  - `private`: accessible only to member functions of the class ⇒ enforce information hiding.
  - `protected`: (actually it is more complicated; left to COMP2012).
- The internal electronic components are like data members.
- Each button is like a member function, which when pressed (c.f. calling a function), will execute a function of the remote control, e.g. next channel, volume up or down,

etc.

13.1.3 Example: The student_record Class and Access Control

File: student-record.h — defines a `student_record` class with private data members (`gender`, `id`, `name`), public accessor member functions (marked `const` so they won't modify data members) and public mutator member functions (`set` and `copy`).

```
#include <cstring>  /* File: student-record.h */
const int MAX_NAME_LEN = 32;

class student_record
{
  private:
    char gender;
    unsigned int id;
    char name[MAX_NAME_LEN];

  public:
    // ACCESSOR member functions: const => won't modify data members
    char get_gender() const { return gender; }
    unsigned int get_id() const { return id; }
    void print() const
         { cout << name << endl << id << endl << gender << endl; }

    // MUTATOR member functions
    void set(const char my_name[], unsigned int my_id, char my_gender)
         { strcpy(name, my_name); id = my_id; gender = my_gender; }
    void copy(const student_record& r) { set(r.name, r.id, \
   r.gender); }
};
```

File: student-record-test.cpp — creates 2 static `student_record` objects, sets their data members through the public interface, and lets `amy` copy `bob`'s identity. The static objects are destroyed automatically at the end of `main()`. To compile: `g++ student-record-test.cpp`.

```
#include <iostream>     /* File: student-record-test.cpp */
using namespace std;
#include "student-record.h"

int main()
{
    student_record amy, bob; // Create 2 static student_record objects

    amy.set("Amy", 12345, 'F');   // Put values to their data members
    bob.set("Bob", 34567, 'M');

    cout << amy.get_id() << endl; // Get and print some data member
    amy.copy(bob);           // Amy wants to fake with Bob's identity
    amy.print();

    return 0;
    // Amy and Bob are static object, which will be destroyed
    // at the end of the function --- main() here --- call.
}   /* To compile: g++ student-record-test.cpp */
```

File: access-control.cpp — demonstrates that public member functions can be called from `main()`, while accessing the private data member `degree` directly from outside the class is a compile-time error.

```
class temperature
{
  private:
```

```
    double degree;      // Internally it is always saved in Kelvin
  public:
    ...
    double kelvin() const;      // Read temperature in Kelvin
    void set(double d, char s); // Set the value of degree
};

int main()
{
    temperature x;
    x.set(20, CELSIUS); // OK: set() is public
    cout << x.degree;   // Error: cannot access private member degree
    return 0;
}
```

Private Members Are Not Accessible from Outside

Inside a non-member function such as `main()`, calling a `public` member function like `x.set(20, CELSIUS)` is fine, but reading or writing a `private` data member like `x.degree` is a compile error. Access control is enforced by the compiler.

13.2 OOP!

13.2.1 Manipulate Similar Objects by a C++ Class

- All cars have attributes (data members) such as make, model, size, color, etc. They all are driven in a similar way (member functions) such as braking, pedaling, etc. So it is natural to create them as objects of a class called '`Car`'.
- In general, create a C++ class for objects of the same kind.

13.2.2 Example: The temperature Class

File: temperature.h — defines a `temperature` class whose data is internally always saved in Kelvin, with 2 constructors, 3 accessors (read the temperature in Kelvin/Celsius/Fahrenheit) and 1 mutator `set`.

```
#include <iostream>     /* File: temperature.h */
#include <cstdlib>
using namespace std;
const char KELVIN = 'K', CELSIUS = 'C', FAHRENHEIT = 'F';

class temperature
{
  private:
    double degree;      // Internally it is always saved in Kelvin
  public:
    // CONSTRUCTOR member functions
    temperature();      // Default constructor
    temperature(double d, char s);

    // ACCESSOR member functions: don't modify data
    double kelvin() const;      // Read temperature in Kelvin
    double celsius() const;     // Read temperature in Celsius
    double fahrenheit() const;  // Read temperature in Fahrenheit

    // MUTATOR member functions: will modify data
    void set(double d, char s);
};
```

File: temperature_constructors_accessors.cpp — implements the constructors (the default one sets `degree` to 0.0; the other one delegates to `set`) and the 3 accessors with the unit-conversion formulas.

```
/* File:  temperature_constructors_accessors .cpp */
#include "temperature.h"

// CONSTRUCTOR member functions
temperature::temperature() { degree = 0.0; }
temperature::temperature(double d, char s) { set(d, s); }

// ACCESSOR member functions
double temperature::kelvin() const { return degree; }
double temperature::celsius() const { return degree - 273.15; }
double temperature::fahrenheit() const
                    { return (degree - 273.15)*9.0/5.0 + 32.0; }
```

File: temperature_mutators.cpp — implements `set`, which converts the input to Kelvin according to the scale, reports an error for a bad scale, and checks the integrity of the data (temperature cannot be less than absolute zero).

```
#include "temperature.h" /* File:  temperature_mutators.cpp */

void temperature::set(double d, char s)
{
    switch (s)
    {
        case KELVIN: degree = d; break;
        case CELSIUS: degree = d + 273.15; break;
        case FAHRENHEIT: degree = (d - 32.0)*5.0/9.0 + 273.15; break;

        default: cerr << "Bad temperature scale: " << s << endl;
        exit(-1);
    }

    if (degree < 0.0)   // Check for integrity of data
    {
        cerr << "Temperature less than absolute zero!" << endl;
        exit(-2);
    }
}
```

File: temperature_test.cpp — repeatedly reads a degree and a scale from the user, sets a `temperature` object (constructed with the default constructor) and prints it in all 3 scales.

```
#include "temperature.h" /* File:  temperature_test.cpp */

int main()
{
    char scale;
    double degree;
    temperature x;      // Use the default constructor

    cout << "Enter temperature (e.g., 98.6 F): ";
    while (cin >> degree >> scale)
    {
        x.set(degree, scale);
        cout << x.kelvin() << " K" << endl;     // Print in Kelvin
        cout << x.celsius() << " C" << endl;    // Print in Celsius
        cout << x.fahrenheit() << " F" << endl; // Print in Fahrenheit

        cout << endl << "Enter temperature (e.g., 98.6 F): ";
    };
```

```
    return 0;
}
```

13.2.3 Constructors and Destructors

- An object is constructed when it is
  - defined in a scope (static object on stack).
  - created with the `new` operator (dynamic object on heap).
- An object is destructed when
  - it goes out of a scope (static object).
  - it is deleted with the `delete` operator (dynamic object).
- For "objects" of basic data types (actually they are not objects in C++), their construction and destruction are built into the C++ language.
- For (real) objects of user-defined classes, C++ allows the class developers to write their own construction and destruction functions: constructors and destructor.
- Besides creating an object, a constructor may also initialize its contents.
- A class may have more than 1 constructor (function overloading), but it can only have 1 and only 1 destructor.
- If you do not provide a constructor/destructor, C++ will automatically generate the default constructor/destructor for you.
- The default constructor just reserves an amount of memory big enough for an object of the class.
  - No initialization will take place (except for those with default values in C++11).
- The default destructor just releases the memory acquired by the object.

```
temperature::temperature() { }
temperature::~temperature() { }
```

Default Constructor/Destructor Are Inadequate for Dynamic Data

For a class that contains dynamic data members, the default constructor/destructor are usually inadequate, as they will not create/delete the dynamic data members for you. Also, the default constructor performs no initialization, so data members contain garbage unless initialized.

13.2.4 Contents and Interface; Class Member Functions

- The **data members** are the contents of the objects of a class.
  - Usually they are made `private`.
  - Different objects of a class usually have different contents — different values for their data members.
- The **member functions** represent the interface to the objects of a class.
  - Usually they are made `public`.
  - `private` member functions are for internal use only.
  - Different objects of a class have the same interface!
- Both data members and member functions are members of a class. They are uniformly accessed by the `.` operator.
- An application programmer should
  - only use the public interface provided by the class developer to manipulate objects of a class.
  - a good class design will prevent the application programmer from accessing and modifying the data members directly.

There are at least 4 types of member functions:

1. **constructor**: used to create class object.

2. **destructor**: used to destruct class objects. It is needed only when objects contain dynamic data member(s).
3. **accessor**: `const` functions that inspect data members; they do not modify any data members though.
4. **mutator**: will modify some data member(s).

The member functions may be defined

- inside the class definition in a `.h` header file.
- outside the class definition in a `.cpp` source file. In that case, each function name must be prepended with the class name and the special class scope operator `::`.

⇒ This is preferred so that application programmers won't see the implementation of the functions, and they are only given the `.o` object file of the class for development.

⇒ Information hiding; protecting intellectual property.

Separate Interface from Implementation

Declare member functions inside the class in a `.h` header file, and define them outside the class in a `.cpp` file with the `ClassName::` prefix. Application programmers then only need the header and the `.o` object file — they cannot see the implementation.

13.2.5 Information Hiding Rules

1. DON'T expose data items in a class.
   ⇒ Make all data members `private`.
   ⇒ Class developer should maintain integrity of data members.
2. DON'T expose the difference between stored data and derived data.
   ⇒ The value that an accessor member function returns may NOT be the value of any data member. It is NOT necessary to have an accessor member function for each data member.
3. DON'T expose a class' internal structure.
   ⇒ Application programmers should NOT assume the data structure used for data members.
   ⇒ Class developer may change representation of data members without affecting the application programmers' codes.
4. DON'T expose the implementation details of a class.
   ⇒ Class developer may change algorithm of member functions without affecting the application programmers' codes.

13.2.6 OOP to Maintain Data/State Consistency/Integrity

- Data/State consistency: each time we change the value of `degree` in a `temperature` object, make sure that the new value is valid, since not all temperature values are possible.
- For a bigger object with many data members, changing the value of one member may affect other members.
- A snapshot of the values of all data members of an object represents the state of the object.
- Applications could easily set invalid or nonsensical values, causing a system to crash. OOP forces an application to use the API that performs sanity checks on all values to maintain consistent states of objects.
- Ensuring data/state consistency is one of the major challenges in (large) software projects.
- The problem becomes even more difficult when the program is modified and new constraints are added.

File: non-oop-temperature.cpp — with a plain `struct`, the application can freely assign an impossible value (−1000 degrees Celsius) to `degree`; there is no way to prevent this from happening.

```
#include <iostream>     /* File: non-oop-temperature.cpp */
using namespace std;
const char CELSIUS = 'C', FAHRENHEIT = 'F';

struct temperature
{
    char scale;
    double degree;
};

int main()
{
    temperature x;
    x.scale = CELSIUS;
    x.degree = -1000;   // That is IMPOSSIBLE!!!!
                        // But how can you prevent this to happen?
    return 0;
};
```

13.2.7 Summary: Classes and Objects; Good OOP Design

- A class is a user-defined type representing a set of objects with the same structure and behavior.
- Objects are variables of a class type.
- Instantiation: the process of creating an object of a class is called instantiating an object.
- Each object of a class has its own copies and values of its data members.
- All objects of a class share a common set of member functions.
- To call a function, before we say "call function X" or simply "call X". In OOP, we have to say "invoke method/operation/function X on object Y of class Z".

File: bad-oop.h — a `temperature` class whose public getters/setters simply mirror every data member is basically no different from the plain `struct`: merely wrapping data with accessors/mutators is not good OOP design.

```
struct temperature
{
    char scale;
    double degree;
};

/************** BASICALLY NO DIFFERENCE BETWEEN **************/
/********** THE STRUCT AND CLASS DEFINITIONS HERE ************/

class temperature
{
  public:
    char get_scale() { return scale; }
    double get_degree() { return degree; }
    void set_scale(char s) { scale = s; }
    void set_degree(double d) { degree = d; }

  private:
    char scale;
    double degree;
};
```

Good OOP Is More Than Getters and Setters

Mechanically providing a getter and a setter for every data member defeats information hiding: the class is then basically equivalent to a plain `struct`. Good OOP design hides the representation and maintains data integrity through a meaningful interface

(e.g. the Kelvin-only storage with sanity checks in `temperature::set`).

13.3 Inline Class Member Functions; Classes With Dynamic Data; Interaction Between Classes

13.3.1 Class Member Functions: Two Ways to Define Them

Class member functions are the functions declared inside the body of a class. They can be defined in two ways:

1. Within the class body, then they are inline functions. The keyword `inline` is optional in this case.

   File: inline-fcn.cpp — member functions defined within the class body; they are inline functions even without the `inline` keyword.

```
class temperature
{   ...
    double kelvin() const { return degree; }
    double celsius() const { return degree - 273.15; }
};
```

   File: inline-fcn2.cpp — the same definitions with the optional `inline` keyword written explicitly.

```
class temperature
{   ...
    inline double kelvin() const { return degree; }
    inline double celsius() const { return degree - 273.15; }
};
```

2. Outside the class body, then add the prefix consisting of the class name and the class scope operator `::`.

   File: outside-fcn.cpp — member functions declared inside the class but defined outside it with the `temperature::` prefix (any benefits of doing this?).

```
/* File: temperature.h */
class temperature
{   ...
    double kelvin() const ;
    double celsius() const;
};
```

```
/* File: temperature.cpp */
#include "temperature.h"
...
double temperature::kelvin() const { return degree; }
double temperature::celsius() const { return degree - 273.15; }
```

13.3.2 Inline Functions and Inline Class Member Functions

- Function calls are expensive because when a function is called, the operating system has to do a lot of things behind the scene to make that happen.

  File: inline1.cpp — an ordinary function call: `f(5)` requires a real function call at run time.

```
int f(int x) { return 4*x*x + 9*x + 1; }
int main() { int y = f(5); }
```

- For small functions that are called frequently, it is actually more efficient to unfold the function codes at the expense of program size (both source file and executable).

File: inline2.cpp — the unfolded version of the call above: the expression is computed directly in `main`.

```
int main() { int y = 4*5*5 + 9*5 + 1; }
```

- But functions have the benefit of easy reading, easy maintenance, and type checking by the compiler.
- You have the benefits of both by declaring the function `inline`.

File: inline3.cpp — declaring `f inline` keeps the function-call syntax while suggesting the compiler to unfold the code.

```
inline int f(int x) { return 4*x*x + 9*x + 1; }
int main() { int y = f(5); }
```

inline Is Only a Hint

C++ compilers may not honor your `inline` declaration. The `inline` declaration is just a hint to the compiler, which still has the freedom to choose whether to inline your function or not, especially when it is large!

- Class member functions defined inside the class definition body are automatically treated as inline functions.
- To enhance readability, one may also define them outside the class definition but in the same header file.

File: inline-class-fcn.h (slide: `temperature1.h`) — an inline member function written out with its body inside the class definition.

```
/* File: temperature1.h */
class temperature
{   ...
    inline double
    kelvin() const
    {
        return degree;
    }
};
```

File: inline-class-fcn2.h (slide: `temperature2.h`) — the member function is only declared inside the class; its inline definition is placed outside the class but still in the same header file, prefixed with `temperature::`.

```
/* File: temperature2.h */
class temperature
{   ...
    inline double kelvin() const;
};

inline double
temperature::kelvin() const
{
    return degree;
}
```

13.3.3 Example: The Book Class with Dynamic Data Members

File: book/book.h — a `Book` class written by a class developer with dynamic data members `title` and `author`; it needs a user-defined destructor to `delete []` them, and `set` allocates exact-size char arrays and copies the strings.

```
#include <iostream>      /* File: book.h */
#include <cstring>
using namespace std;
```

```
class Book              // Class definition written by class developer
{
  private:
    char* title;
    char* author;
    int num_pages;
  public:
    Book(int n = 100) { title = author = nullptr; num_pages = n; }
    Book(const char* t, const char* a, int n = 5) { set(t, a, n); }
    ~Book()
    {
        cout << "Delete the book titled \"" << title << "\"" << endl;
        delete [] title; delete [] author;
    }

    void set(const char* t, const char* a, int n)
    {
        title = new char [strlen(t)+1]; strcpy(title, t);
        author = new char [strlen(a)+1]; strcpy(author, a);
        num_pages = n;
    }
};
```

File: book/book.cpp — an app written by an application programmer: creates static and dynamic `Book` objects and a dynamic array of 3 `Book`s, showing when constructors and destructors run.

```
#include "book.h"       /* File: book.cpp */
void create_books()
{
    Book y("Love", "HKUST", 88);
    Book* p = new Book [3];
    p[0].set("book1", "author1", 1);
    p[1].set("book2", "author2", 2);
    p[2].set("book3", "author3", 3);

    delete [] p; cout << endl;
    return;
}

int main()    /* An app written by an application programmer */
{
    Book x("Sapiens", "Y. N. Harari", 1000);
    Book* z = new Book("Outliers", "Gladwell", 300);
    create_books(); cout << endl;
    delete z; cout << endl;
    return 0;
}
```

Output of the program:

```
Delete the book titled "book3"
Delete the book titled "book2"
Delete the book titled "book1"
Delete the book titled "Love"

Delete the book titled "Outliers"

Delete the book titled "Sapiens"
```

delete [] Calls the Destructor on Each Element in Reverse Order

Delete an array of user-defined objects using `delete []`. `delete []` will call the class destructor on each array element in reverse order: the last element first till the first

one. Notice also how the destructor of a static object (e.g. `x`, `y`) is automatically called when it goes out of scope (e.g. when a function returns).

13.3.4 Example: 2 C++ Classes — Bulbs and Lamps

This example consists of 2 classes: `Bulb` and `Lamp`.

- A lamp has at least one light bulb.
- All bulbs of a lamp are the same in terms of price and wattage (power).
- The price of a lamp that is passed to the `Lamp`'s constructor does not include the price of its bulbs, which have to be bought separately.
- One installs bulb(s) onto a lamp by calling its member function `install_bulbs`.

File: bulb-lamp/lamp-test.cpp — creates 2 lamps, installs bulbs of given wattage and price on each, and prints their total power and total price. To compile: `g++ -o lamp-test lamp-test.cpp bulb.cpp lamp.cpp`.

```
#include "lamp.h" /* File: lamp-test.cpp */

int main()
{
    Lamp lamp1(4, 100.5); // lamp1 costs $100.5 and needs 4 bulbs
    Lamp lamp2(2, 200.6); // lamp2 costs $200.6 and needs 2 bulbs

    // Install 4 bulbs of 20 Watts, each costing $30.1 on lamp1
    lamp1.install_bulbs(20, 30.1);
    lamp1.print("lamp1");

    // Install 2 bulbs of 60 Watts, each costing $50.4 on lamp2
    lamp2.install_bulbs(60, 50.4);
    lamp2.print("lamp2");

    return 0;
}
/* To compile: g++ -o lamp-test lamp-test.cpp bulb.cpp lamp.cpp */
```

File: bulb-lamp/bulb.h — the `Bulb` class with private data `wattage` and `price`, two const accessors and one mutator `set`.

```
/* File: bulb.h */

class Bulb
{
  private:
    int wattage;        // A light bulb's power in watt (W)
    float price;        // A light bulb's price in dollars ($)

  public:
    int get_power() const;
    float get_price() const;
    void set(int w, float p); // w = bulb's wattage; p = its price
};
```

File: bulb-lamp/bulb.cpp — the out-of-class definitions of `Bulb`'s member functions, each prefixed with `Bulb::`.

```
/* File: bulb.cpp */

#include "bulb.h"

int Bulb::get_power() const { return wattage; }

float Bulb::get_price() const { return price; }
```

```
void Bulb::set(int w, float p) { wattage = w; price = p; }
```

File: bulb-lamp/lamp.h — the `Lamp` class contains a dynamic array of `Bulb` objects as a data member: an example of one class using another class.

```
#include "bulb.h"       /* File: lamp.h */
class Lamp
{
  private:
    int  num_bulbs; // A lamp MUST have 1 or more light bulbs
    Bulb* bulbs; // Dynamic array of light bulbs installed onto a lamp
    float price; // Price of the lamp, not including its bulbs

  public:
    Lamp(int n, float p);      // n = number of bulbs; p = lamp's \
   price
    ~Lamp();

    int total_power() const;   // Total power/wattage of its bulbs
    float total_price() const; // Price of a lamp PLUS its bulbs

    // Print out a lamp's information; see outputs from our example
    void print(const char* prefix_message) const;

    // All light bulbs of a lamp have the same power/wattage and \
   price:
    // w = a light bulb's wattage; p = a light bulb's price
    void install_bulbs(int w, float p);
};
```

File: bulb-lamp/lamp.cpp — the constructor allocates the dynamic array of bulbs and the destructor releases it; `Lamp`'s member functions call `Bulb`'s member functions on the array elements.

```
#include "lamp.h"       /* File: lamp.cpp */
#include <iostream>
using namespace std;

Lamp::Lamp(int n, float p)
          { num_bulbs = n; price = p; bulbs = new Bulb [n]; }
Lamp::~Lamp() { delete [] bulbs; }
int Lamp::total_power() const
          { return num_bulbs*bulbs[0].get_power(); }
float Lamp::total_price() const
          { return price + num_bulbs*bulbs[0].get_price(); }
void Lamp::print(const char* prefix_message) const
{
    cout << prefix_message << ": total power = " << total_power()
         << "W" << " , total price = $" << total_price() << endl;
}
void Lamp::install_bulbs(int w, float p)
{
    for (int j = 0; j < num_bulbs; ++j)
        bulbs[j].set(w, p);
}
```

13.4 C++ Class Basics, and this Pointer

13.4.1 Structure vs. Class; Name Equivalence

- In C++, structures are special classes and they may have member functions. By default,

```
struct { ... }  is equivalent to  class { public: ... }
class  { ... }  is equivalent to  struct { private: ... }
```

- So the `temperature` class could equally well be written as a `struct` with explicit `private:`/`public:` labels (shown on the slide):

```
#include <iostream>      /* File: temperature.h */
#include <cstdlib>
using namespace std;
const char KELVIN = 'K', CELSIUS = 'C', FAHRENHEIT = 'F';

struct temperature
{
    double degree;
    temperature();
    temperature(double d, char s);
    double kelvin() const;
    double celsius() const;
    double fahrenheit() const;
    void set(double d, char s);
};
```

struct and class Differ Only in Default Access

The only difference between `struct` and `class` in C++ is the default access level: members of a `struct` are `public` by default, while members of a `class` are `private` by default. Both may have member functions, constructors and destructors.

- A class definition introduces a new abstract data type.
- C++ relies on name equivalence (and not structure equivalence) for class types.

```
class X { int a; };
class Y { int a; };
class W { int a; };
class W { int b; }; // Error, double definition

X x;
Y y;
x = y;              // Error: type mismatch
```

Same Structure Does Not Mean Same Type

`X` and `Y` have exactly the same structure, yet they are different types, so `x = y;` is a type-mismatch error. Defining the same class name twice (even identically) is a double-definition error.

13.4.2 Class Data Members

- Data members can be any basic type, or any user-defined types if they are already declared.
- A special case: a class name can be used inside its own definition for a pointer to an object of the class:

```
class Cell
{
    int info;
    Cell* next;
};
```

- A forward declaration of a class `X` can be used in the definition of another class `Y` to define a pointer to `X`:

```
class Cell;          // Forward declaration of Cell

class List
{
    int size;
    Cell* data;      // Points to a (forward-declared) Cell object
    Cell x;          // Error: Cell not defined yet!
};

class Cell           // Definition of Cell
{
    int info;
    Cell* next;
};
```

Forward Declaration Allows Pointers Only

With only a forward declaration of `Cell`, you may declare `Cell*` pointers (or references) inside another class, but not a `Cell` object member — the compiler does not yet know the size of `Cell`.

13.4.3 Class Scope and Scope Operator ::

- C++ uses lexical (static) scope rules: the binding of name occurrences to declarations is done statically at compile time.
- Identifiers declared inside a class definition are under its scope.
- To define the member functions outside the class definition, prefix the identifier with the class scope operator `::`, e.g. `temperature::kelvin()`, `temperature::celsius()`.

```
int height = 10;

class Weird
{
    short height;
    Weird() { height = 5; }
};
```

Which height Is Used?

Q1: Which "`height`" is used in `Weird::Weird()`? Q2: Can we access the global `height` inside the `Weird` class body? (Hint: the class member shadows the global one; the global one is still reachable with the scope operator `::height`.)

13.4.4 this Pointer

- Each class member function implicitly contains a pointer of its class type named "`this`".
- When an object calls the function, `this` pointer is set to point to the object.
- For example, after compilation, the `temperature::set(double d, char s)` function in the `temperature` class will be translated to a unique global function by adding a new argument:

```
void temperature::set(temperature* this, double d, char s)
{
    ...
}
```

- `x.set(d, s)` becomes `temperature::set(&x, d, s)`.

Member Functions Are Shared via the this Pointer

This is why the compiler only needs one copy of the machine code for each member function: the call `x.set(d, s)` is translated into an ordinary function call with the object's address `&x` passed as the hidden `this` argument.

- For `const` member functions, their implicit `this` pointer is a const pointer.
- That is, the `this` pointer in a const member function is pointing to a `const` object.
- Thus, the object can't be modified inside a const member function.
- E.g., after compilation, `temperature::kelvin() const` will be translated to a global function like:

```
void temperature::kelvin(const temperature* this)
{
    ...
}
```

- One of the most common uses of `this` pointer is to resolve name ambiguity.
- If the parameters of the constructors or member functions have the same name as the data members, we can use `this` pointer to distinguish between them.

```
class Weird
{
    short height;
    Weird(int height)
    {
        height = height;
    }
};
```

Does height = height Modify the Data Member?

Q1: Is the data member "`height`" modified in `Weird::Weird(int)`? Q2: How to access the data member "`height`" in `Weird::Weird(int)`? (Hint: the parameter shadows the data member, so `height = height;` only assigns the parameter to itself; use `this->height = height;` to modify the data member.)

The slide rewrites the mutator `temperature::set` so that its parameters have the same names as the data members; `this->` is then used to access the data members unambiguously:

```
#include "temperature.h" /* File: temperature_mutators.cpp */

void temperature::set(double degree, char scale)
{
    switch (scale)
    {
        case KELVIN:
            this->degree = degree; break;
        case CELSIUS:
            this->degree = degree + 273.15; break;
        case FAHRENHEIT:
            this->degree = (degree - 32.0)*5.0/9.0 + 273.15; break;
        default: cerr << "Bad temperature scale: " << scale << endl;
        exit(-1);
    }

    if (this->degree < 0.0)
    {
        cerr << "Temperature less than absolute zero!" << endl;
        exit(-2);
    }
}
```

14 Stack & Queue

14.1 Data Structures

- Computer science is the study of how to process information (data) efficiently using computers.
- A **data structure** helps store, organize, and manipulate data in a particular way so that they can be processed efficiently by computers.
- Different applications require different data structures.
- Examples: array, linked list, (binary) tree, stack, queue, etc.

Stack and Queue Are Restricted Access Structures

Stack and queue let you insert and remove items at the ends only, not in the middle.

14.2 Part I: Stack

14.2.1 Stack: How it Works

Consider a pile of cookies.

- **More cookies:** new cookies are added on top, one at a time.
- **Fewer cookies:** cookies are consumed one at a time, starting at the top.

As an ADT (abstract data type), insertions and removals of items on a stack are based on the **last-in first-out (LIFO)** policy.

Stack ADT

- **Data:** an ordered list of data/items.
- **Major operations:**
  - `top`: get the value of the top item
  - `push`: add a new item to the top
  - `pop`: remove an item from the top

14.2.2 The int_stack Class

The class `int_stack` uses an array `data` of size `BUFFER_SIZE` to store the items, plus an integer `top_index` recording the index of the top item (it starts from 0 and equals -1 when the stack is empty). Accessor member functions are declared `const` because they do not modify the data members.

File: int-stack.h — declaration of the `int_stack` class (array-based stack of `int`).

```
#include <iostream>     /* File: int-stack.h */
#include <cstdlib>
using namespace std;
const int BUFFER_SIZE = 20;

class int_stack
{
  private:
    int data[BUFFER_SIZE];  // Use an array to store data
    int top_index;          // Starts from 0; -1 when empty

  public:
    // CONSTRUCTOR member functions
    int_stack();            // Default constructor

    // ACCESSOR member functions: const => won't modify data members
    bool empty() const;     // Check if the stack is empty
```

```
    bool full() const;      // Check if the stack is full
    int size() const;       // Give the number of data currently \
   stored
    int top() const;        // Retrieve the value of the top item

    // MUTATOR member functions
    void push(int);         // Add a new item to the top of the stack
    void pop();             // Remove the top item from the stack
};
```

14.2.3 Application: Decimal to Binary Conversion

e.g., $26_{(10)} = 11010_{(2)}$.

Algorithm to convert $N_{(10)} = M_{(2)}$:

1. Step 1: divide N by 2 successively.
2. Step 2: each time `push` the remainder onto a stack.
3. Step 3: print the answer by `pop`-ping the stack successively.

Why a Stack Works Here

Repeated division by 2 produces the binary digits from the least significant bit to the most significant bit. Pushing them onto a stack and then popping prints them in reverse — exactly the correct order.

File: decimal2binary.cpp — converts a positive decimal number to a binary number using an `int_stack`.

```
#include "int-stack/int-stack.h" /* File: decimal2binary.cpp */

int main()  // Convert +ve decimal number to binary number using an \
   stack
{
    int_stack a;
    int x, number;

    while (cin >> number)
    {   // Conversion: decimal to binary
        for (x = number; x > 0; x /= 2)
            a.push(x % 2);

        // Print a binary that is stored on a stack
        cout << number << "(base 10) = ";
        while (!a.empty())
        {
            cout << a.top();
            a.pop();
        }
        cout << "(base 2)" << endl;
    }

    return 0;
} // Compile: g++ -o decimal2binary -Lint-stack decimal2binary.cpp \
   -lintstack
```

14.2.4 Implementing the int_stack Class

The default constructor creates an empty stack by setting `top_index` to -1. `empty()`, `full()` and `size()` are simple checks on `top_index`. `top()` returns `data[top_index]` when the stack is non-empty; otherwise it prints a warning to `cerr` and calls `exit(-1)`.

File: int-stack1.cpp — default constructor and accessor member functions of `int_stack`.

```
#include "int-stack.h"  /* File: int-stack1.cpp */

          /***** Default CONSTRUCTOR member function *****/
int_stack::int_stack() { top_index = -1; } // Create an empty stack

          /***** ACCESSOR member functions *****/
// Check if the int_stack is empty
bool int_stack::empty() const { return (top_index == -1); }

// Check if the int_stack is full
bool int_stack::full() const { return (top_index == BUFFER_SIZE-1); }

// Give the number of data currently stored
int int_stack::size() const { return top_index + 1; }

// Retrieve the value of the top item
int int_stack::top() const
{
    if (!empty())
        return data[top_index];

    cerr << "Warning: Stack is empty; can't retrieve any data!" << \
   endl;
    exit(-1);
}
```

top on an Empty Stack

Calling `top()` when the stack is empty does not return a meaningful value: the implementation prints a warning to `cerr` and terminates the program with `exit(-1)`. Always check `empty()` before `top()`.

Mutators. `push(x)` stores `x` at `data[++top_index]` (note the pre-increment) if the stack is not full; `pop()` simply does `--top_index` if the stack is not empty. Both print an error message to `cerr` and call `exit(-1)` when the operation is impossible.

File: int-stack2.cpp — mutator member functions `push` and `pop` of `int_stack`.

```
#include "int-stack.h"  /* File: int-stack2.cpp */

        /***** MUTATOR member functions *****/
void int_stack::push(int x) // Add a new item to the top of the stack
{
    if (!full())
        data[++top_index] = x;
    else
    {
        cerr << "Error: Stack is full; can't add (" << x << ")!" << \
   endl;
        exit(-1);
    }
}

void int_stack::pop()       // Remove the top item from the stack
{
    if (!empty())
        --top_index;
    else
    {
        cerr << "Error: Stack is empty; can't remove any data!" << \
   endl;
        exit(-1);
```

```
    }
}
```

push on Full and pop on Empty

- `push` on a full stack: error "Stack is full; can't add (x)!" then `exit(-1)`.
- `pop` on an empty stack: error "Stack is empty; can't remove any data!" then `exit(-1)`.
- Also note `pop()` does not return the removed value; use `top()` first if you need it.

Example: File: int-stack-test.cpp — exercises `push`/`pop` and prints the stack size, top item, and the `empty()`/`full()` status after each operation (note the use of `boolalpha` to print `true`/`false`).

```
#include "int-stack.h" /* File: int-stack-test.cpp */

void print_stack_info(const int_stack& s)
{
    cout << "No. of data currently on the stack = " << s.size() << \
   "\t";
    if (!s.empty())
        cout << "Top item = " << s.top();
    cout << endl << "Empty: " << boolalpha << s.empty()
         << "\t\t" << "Full: " << boolalpha << s.full() << endl << \
   endl;
}

int main()
{
    int_stack a; print_stack_info(a);
    a.push(4);   print_stack_info(a);
    a.push(15);  print_stack_info(a);
    a.push(26);  print_stack_info(a);
    a.push(37);  print_stack_info(a);
    a.pop();     print_stack_info(a);
    a.push(48);  print_stack_info(a);
    a.push(59);  print_stack_info(a);

    return 0;
} /* compile: g++ -L. -o int-stack-test int-stack-test.cpp \
   -lintstack */
```

14.3 Part II: Queue

14.3.1 Queue: How it Works

Consider the case when people line up for tickets.

- **More people:** new customers join the back of a queue, one at a time.
- **Fewer people:** the customer at the front buys a ticket and leaves the queue.

As an ADT, insertions and removals of items on a queue are based on a **first-in first-out (FIFO)** policy.

Queue ADT

- **Data:** an ordered list of data/items.
- **Major operations:**
  - `front`: get the value of the front item
  - `enqueue`: add a new item to the back

– `dequeue`: remove an item from the front

14.3.2 Circular Queue of int Data

In an array-based queue, the slot freed by `dequeue` at the front would be wasted if the back index only moved forward. A **circular queue** treats the array as a ring: indices wrap around to 0 after reaching `BUFFER_SIZE - 1`, so all slots can be reused.

The class `int_queue` keeps:

- `data[BUFFER_SIZE]`: array storing the items;
- `num_items`: number of items currently on the queue;
- `first`: index of the first (front) item, starting from 0.

File: int-queue.h — declaration of the `int_queue` class (circular queue of `int`).

```
#include <iostream>     /* File: int-queue.h */
#include <cstdlib>
using namespace std;
const int BUFFER_SIZE = 5;

class int_queue // Circular queue
{
  private:
    int data[BUFFER_SIZE]; // Use an array to store data
    int num_items;         // Number of items on the queue
    int first;             // Index of the first item; start from 0

  public:
    // CONSTRUCTOR member functions
    int_queue();           // Default constructor

    // ACCESSOR member functions: const => won't modify data members
    bool empty() const;    // Check if the queue is empty
    bool full() const;     // Check if the queue is full
    int size() const;      // Give the number of data currently stored
    int front() const;     // Retrieve the value of the front item
    // MUTATOR member functions
    void enqueue(int);     // Add a new item to the back of the queue
    void dequeue();        // Remove the front item from the queue
};
```

Example: File: int-queue-test.cpp — exercises `enqueue`/`dequeue` on a circular queue of `BUFFER_SIZE` 5 and prints the queue size, front item, and the `empty()`/`full()` status after each operation.

```
#include "int-queue.h" /* File: int-queue-test.cpp */

void print_queue_info(const int_queue& a) {
    cout << "No. of data currently on the queue = " << a.size() << \
   "\t";
    if (!a.empty()) cout << "Front item = " << a.front();
    cout << endl << "Empty: " << boolalpha << a.empty();
    cout << "\t\t" << "Full: " << boolalpha << a.full() << endl << \
   endl;
}

int main() {
    int_queue a;    print_queue_info(a);
    a.enqueue(4);   print_queue_info(a);
    a.enqueue(15);  print_queue_info(a);
    a.enqueue(26);  print_queue_info(a);
```

```
    a.enqueue(37); print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    a.enqueue(48); print_queue_info(a);
    a.enqueue(59); print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    a.dequeue();   print_queue_info(a);
    return 0;
} /* compile: g++ -L. -o int-queue-test int-queue-test.cpp \
   -lintqueue */
```

14.3.3 Implementing the int_queue Class

The default constructor creates an empty queue by setting `first = 0` and `num_items = 0`. The queue is empty when `num_items == 0` and full when `num_items == BUFFER_SIZE`. `front()` returns `data[first]` when the queue is non-empty; otherwise it prints a warning and calls `exit(-1)`.

File: int-queue1.cpp — default constructor and accessor member functions of `int_queue`.

```
#include "int-queue.h"  /* File: int-queue1.cpp */

          /***** Default CONSTRUCTOR member function *****/
// Create an empty queue
int_queue::int_queue() { first = 0; num_items = 0; }

          /***** ACCESSOR member functions *****/
// Check if the int_queue is empty
bool int_queue::empty() const { return (num_items == 0); }

// Check if the int_queue is full
bool int_queue::full() const { return (num_items == BUFFER_SIZE); }

// Give the number of data currently stored
int int_queue::size() const { return num_items; }

// Retrieve the value of the front item
int int_queue::front() const
{
    if (!empty())
        return data[first];

    cerr << "Warning: Queue is empty; can't retrieve any data!" << \
   endl;
    exit(-1);
}
```

Distinguishing Empty from Full

In a circular queue the front index alone cannot tell "empty" from "full" (both may have `first` pointing at a valid slot). That is why `int_queue` stores a separate counter `num_items`: empty means `num_items == 0`, full means `num_items == BUFFER_SIZE`.

Mutators.

- `enqueue(x)`: the new item goes to the back of the queue, whose index is `(first+num_items) % BUFFER_SIZE`; then `num_items` is incremented. Error and `exit(-1)` if the queue is full.
- `dequeue()`: the front advances by one with wrap-around, `first = (first+1) % BUFFER_SIZE`; then `num_items` is decremented. Error and `exit(-1)` if the queue is empty.

Wrap-Around with the % Operator

The modulo operator `% BUFFER_SIZE` is what makes the queue circular: whenever an index reaches the end of the array it wraps back to 0, so freed slots at the front are reused.

File: int-queue2.cpp — mutator member functions `enqueue` and `dequeue` of `int_queue`.

```
#include "int-queue.h" /* File: int-queue2.cpp */

void int_queue::enqueue(int x)  // Add a new item to the back of the \
   queue
{
    if (!full())
    {
        data[(first+num_items) % BUFFER_SIZE] = x;
        ++num_items;
    } else {
        cerr << "Error: Queue is full; can't add (" << x << ")!" << \
   endl;
        exit(-1);
    }
}

void int_queue::dequeue()       // Remove the front item from the \
   queue
{
    if (!empty())
    {
        first = (first+1) % BUFFER_SIZE;
        --num_items;
    } else {
        cerr << "Error: Queue is empty; can't remove any data!" << \
   endl;
        exit(-1);
    }
}
```

front on an Empty Queue

Just like `top()` on a stack, calling `front()` on an empty queue prints a warning and terminates with `exit(-1)`. Check `empty()` first.